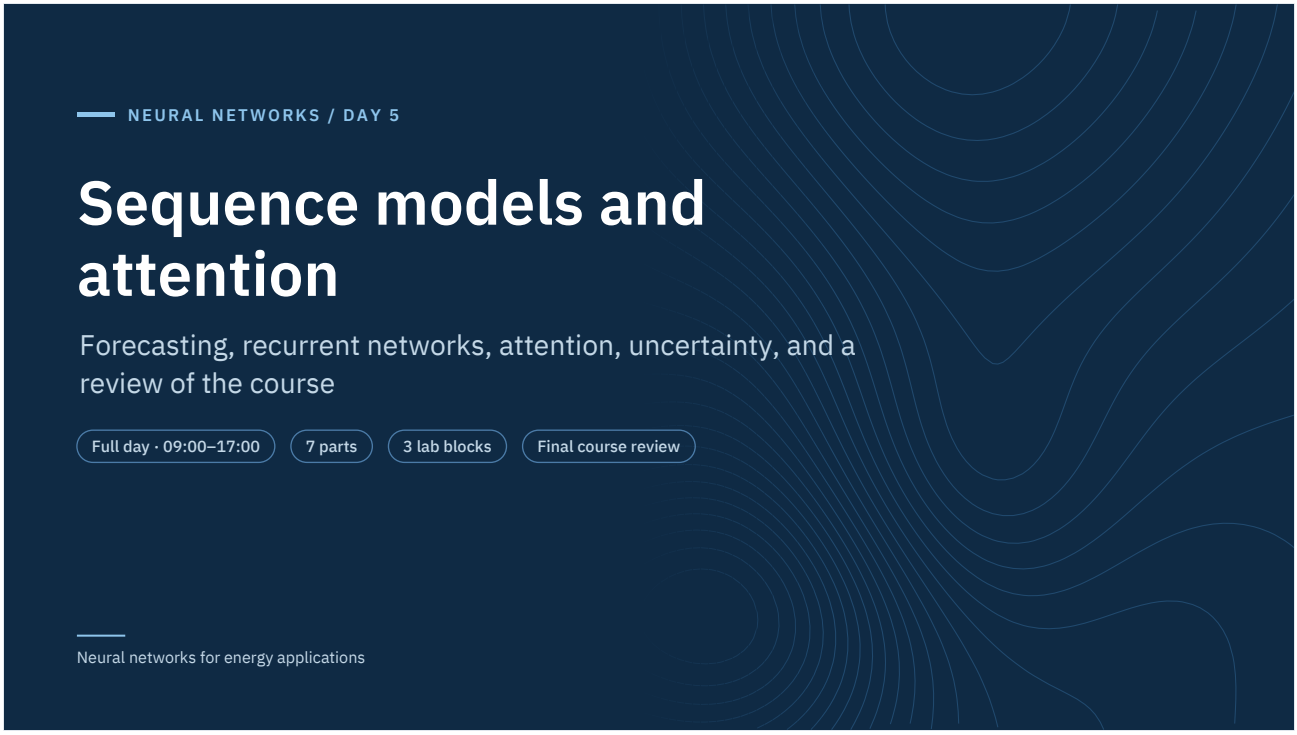
Sequence models and attention

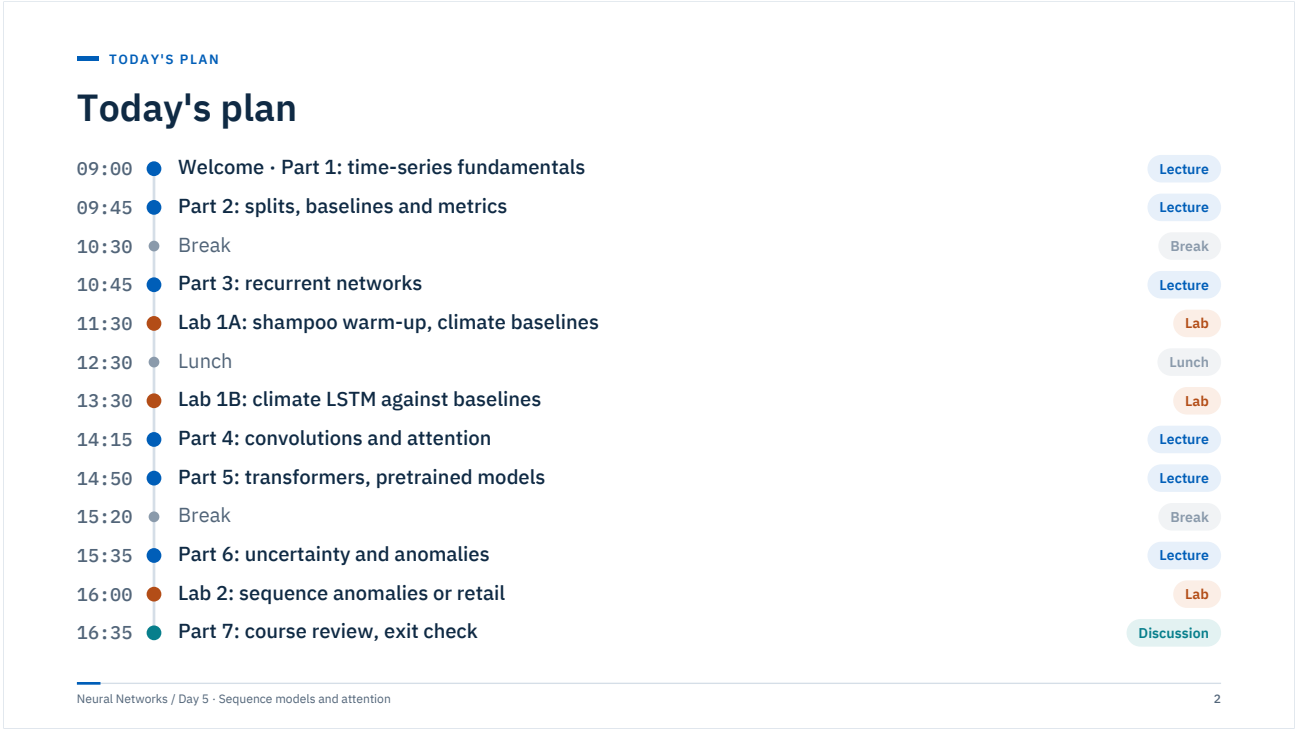


Welcome to the last day of the neural networks course. Today connects everything from Days 1 to 4 to data that arrive in order: load, weather, production, pump and drilling sensors.

Prerequisites are Days 1 and 2, especially the chain rule, tensor shapes and the training loop. Day 4 helps when we meet one-dimensional and dilated convolutions, but it is not required for the forecasting and evaluation material.

One idea runs through the whole day: every forecast is defined by its origin, the moment at which it is made. Keep asking "what was known at that moment?" whenever a model, a feature or a score appears. [2 min]

Today's plan



The morning builds the evaluation habits and the recurrent models; the first laboratory straddles lunch so that longer training runs can finish while people eat. The afternoon covers convolutions, attention, transformers, uncertainty and anomaly detection, then closes with a review of the whole course.

Before class, check on one laptop that the notebooks open and that torch, pandas and scikit-learn import. Notebook 04 reads an Excel file and also needs openpyxl, which is listed in requirements-labs.txt. The Jena file is 42 MB and three of the four retail files exceed 100 MB, so copy them locally rather than over a slow network share.

Timing is tight in Parts 5 and 6. If the morning runs long, shorten the transformer-family and application slides, not the exercises. [1 min]

What you will be able to do

LEARNING OUTCOMES

What you will be able to do

By the end of the day you will be able to:

1. Build forecast windows whose inputs are known at the origin, and count the samples.
2. Evaluate a forecast with a time-respecting split, strong baselines and per-horizon metrics.
3. Explain how RNN, LSTM and GRU states update, and check their tensor shapes in PyTorch.
4. Compute a small attention example, apply a causal mask and describe a transformer block.
5. Produce and check quantile forecasts and reconstruction-based anomaly scores.
6. Choose a model family for a new energy problem and defend its evaluation design.

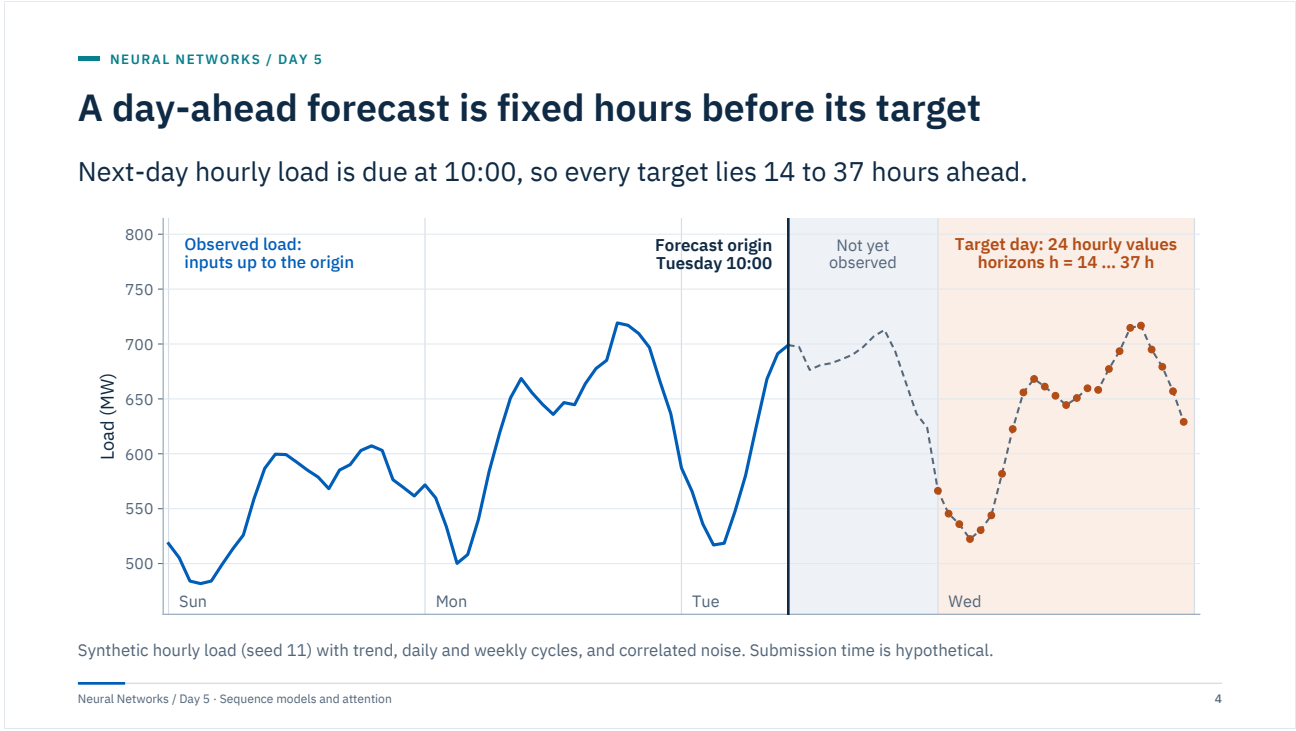
Neural Networks / Day 5 · Sequence models and attention 3

Read the outcomes aloud and point out that each is an observable skill: you will count windows, compute baseline errors, write down shapes, calculate attention weights and check interval coverage. None of the outcomes is "train the biggest model".

Outcomes 1 and 2 are the foundation of the whole day and are practised in Lab 1. Outcomes 3 and 4 are the architecture content. Outcome 5 is practised in Lab 2. Outcome 6 is the final course review, where you will defend a model choice for a stated energy task.

Ask the room which of these they have already done at work, and which feels least familiar. Use the answer to decide where to slow down. [2 min]

A day-ahead forecast is fixed hours before its target



This is the motivating case for the day. A utility or a trader must submit 24 hourly values for Wednesday before 10:00 on Tuesday. The forecast origin is 10:00 Tuesday. The first target, 00:00 Wednesday, is 14 hours ahead; the last, 23:00 Wednesday, is 37 hours ahead.

Point at the grey band: the hours between 10:00 and midnight are not targets, but they are also not known. A training window that uses the observed 20:00 Tuesday load as an input would look excellent in validation and be impossible in operation.

Ask: what is the horizon for 18:00 Wednesday? The answer is 32 hours. What inputs are legitimate? Load observed up to 10:00, the calendar for Wednesday, and any weather forecast issued before 10:00.

Everything later today, from baselines to LSTMs to attention, will be judged against this contract. [3 min]

A forecast is indexed by its origin

PRACTICE

4 min Pairs

A forecast is indexed by its origin

At 12:00 you must predict air temperature at 18:00 today.

1. Which measurements are already known, and what is the horizon?
2. Can you use tomorrow's observed maximum temperature?
3. How do known calendar features differ from future sensor readings?

Neural Networks / Day 5 · Sequence models and attention 5

Give pairs three minutes, then take answers.

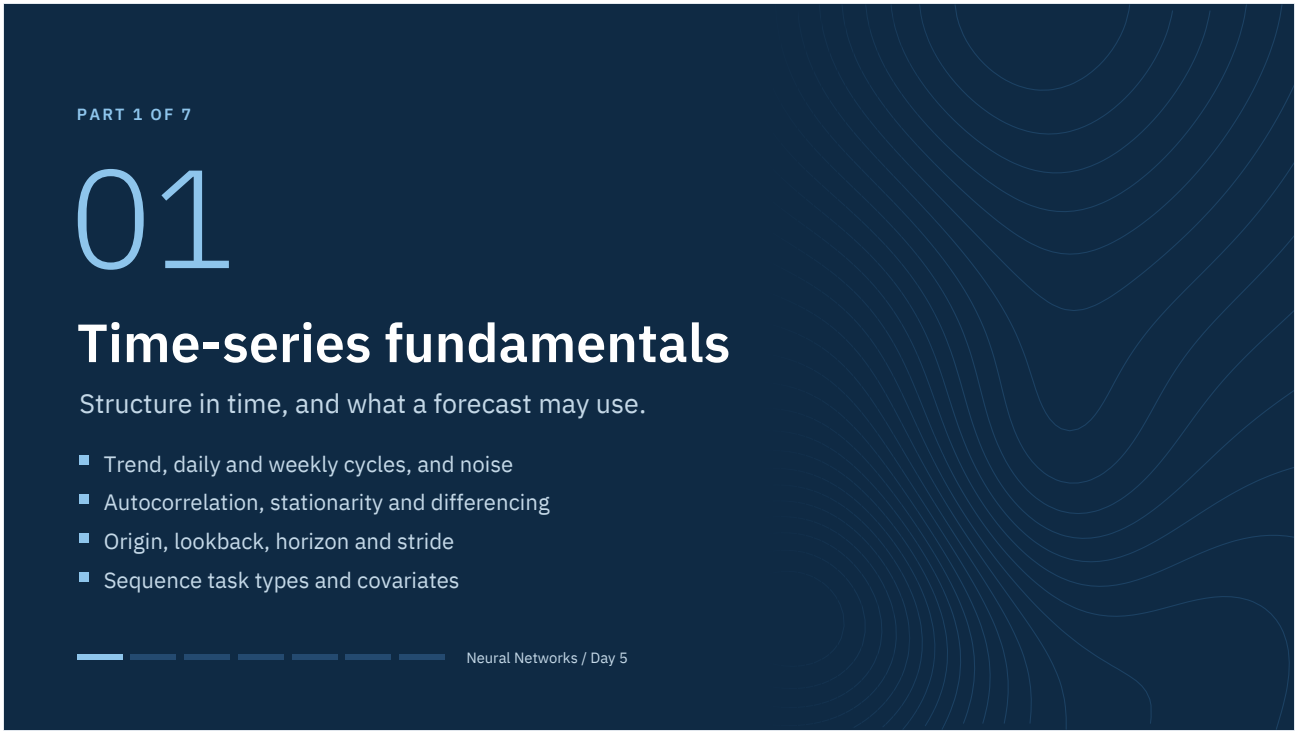
Answers. Known: every measurement that has arrived by 12:00, not merely those time-stamped before 12:00; a sensor with a 30-minute delay contributes values up to about 11:30. The horizon is 6 hours.

Tomorrow's maximum is unknown at 12:00, so using it is leakage, even if it sits in the same row of a spreadsheet.

Calendar values such as hour of day, weekday and public holidays are known for any future time and are legitimate future covariates. Realized future weather is not. A weather forecast from an external model is legitimate only if it was issued before 12:00; use the vintage that existed at the origin, not a later revised run.

Common error: treating "the row for 18:00" as the unit of prediction. The unit is the pair (origin, target time). [4 min]

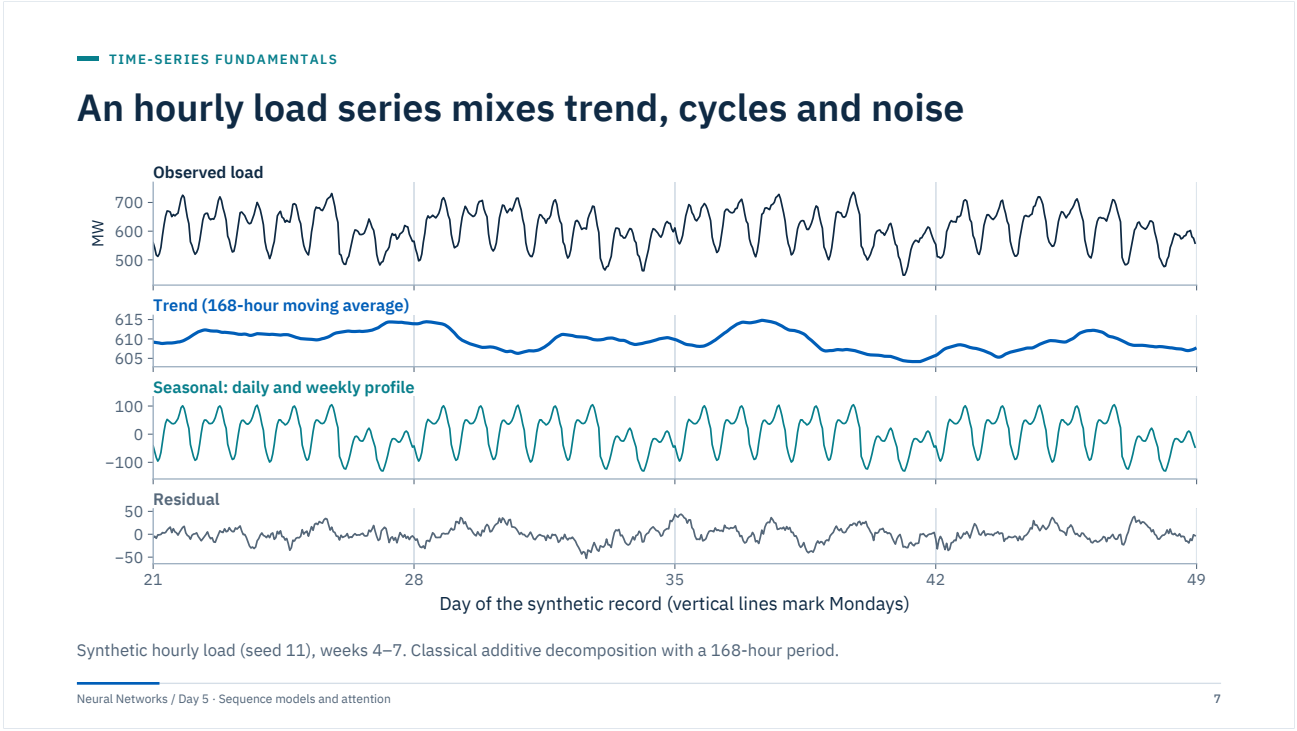
Time-series fundamentals



This part gives the vocabulary for the rest of the day. We use a synthetic hourly electrical load series throughout because its structure is known exactly, which lets us check every number on the slides. The labs then use real course data.

The part ends with an exercise that counts the windows produced by the climate notebook, so learners arrive at Lab 1 already knowing what shapes to expect. [1 min]

An hourly load series mixes trend, cycles and noise



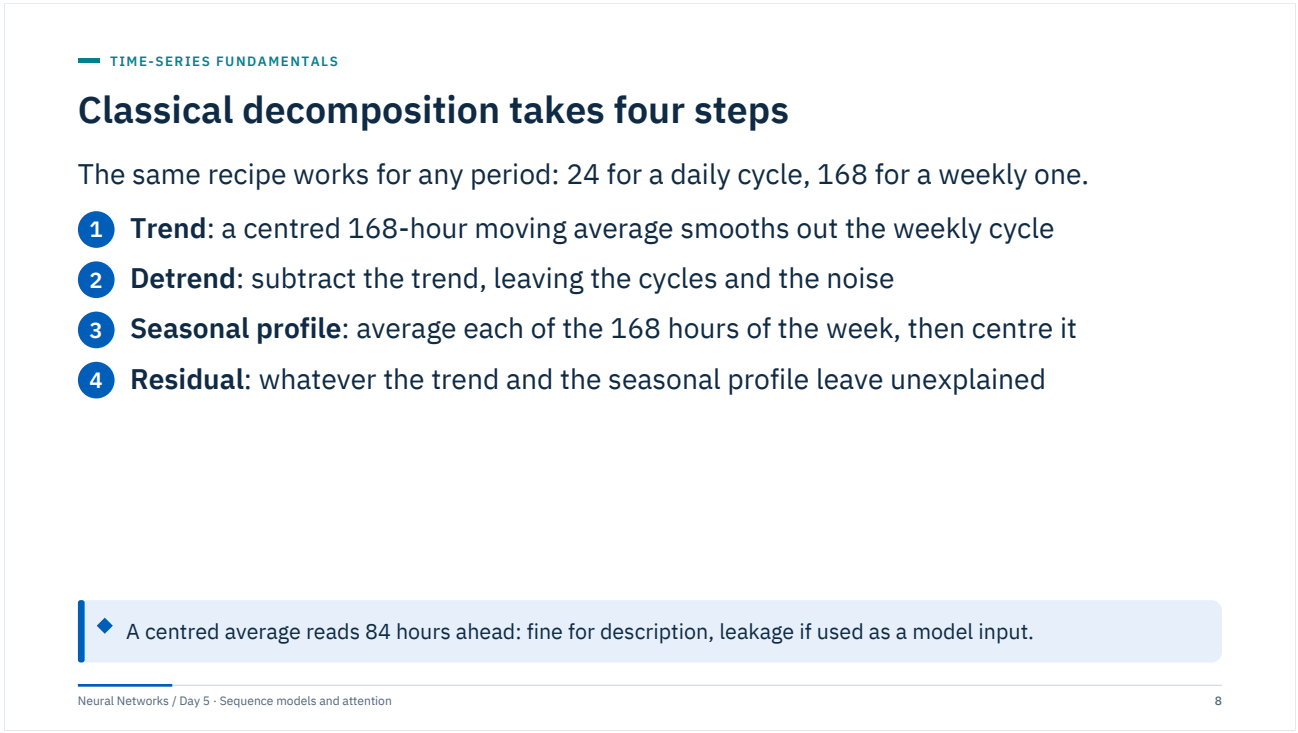
Walk down the panels. The observed load has a daily double peak, lower weekends and a slow drift. The trend panel is a 168-hour moving average. In these four weeks it stays between about 605 and 615 MW: the generator's steady rise of 0.6 MW per day is offset here by the falling part of a slow nine-week swing, and the noise adds the wiggles. Over the whole twelve-week record the level rises by about 60 MW, as the differencing slide shows. The seasonal panel repeats every week: five weekday shapes and two lower weekend shapes. The residual is what is left.

Point out that the residual is not white noise: it wanders for hours at a time. The generator uses autocorrelated noise, as real feeders do, because weather and customer behaviour persist. A decomposition describes a series; it does not remove short-term memory.

Additive decomposition assumes the seasonal swing has the same size at any level. If the daily swing grows with the level, use a multiplicative form or decompose the logarithm.

Ask: what would a heat wave look like in these panels? Mostly in the trend and the residual, because it is not part of the repeating weekly profile. [3 min]

Classical decomposition takes four steps



Explain the moving average for an even period: a 2 × 168 average gives half weight to the two end points so the window is centred. The seasonal profile has 168 values, one per hour of the week, averaged over all weeks after detrending, then shifted to have mean zero.

Robust alternatives such as STL (seasonal-trend decomposition using loess) handle outliers and slowly changing seasonal shapes better; they are available in statsmodels, which is not installed in the course environment.

Stress the callout. The centred trend at 10:00 Tuesday uses load up to 22:00 Friday. That is perfect for understanding a series and wrong for a feature. If you want trend or seasonal features in a forecast model, compute them causally, from data up to the origin, and fit any profile on the training period only. [2 min]

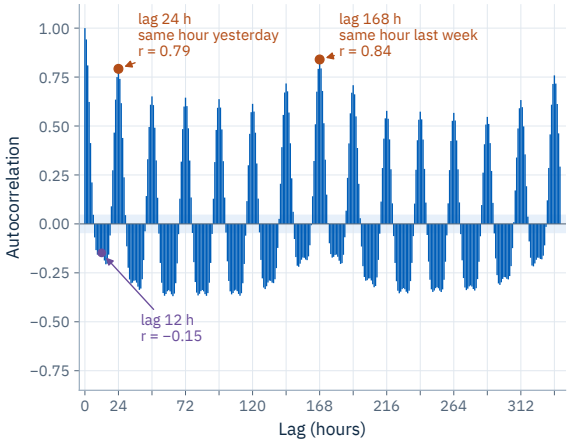
Autocorrelation reveals daily and weekly memory

TIME-SERIES FUNDAMENTALS

Autocorrelation reveals daily and weekly memory

$$r_k = \frac{\sum_t (y_t - \bar{y})(y_{t+k} - \bar{y})}{\sum_t (y_t - \bar{y})^2}$$

- r_k correlates the series with itself k steps later.
- Peaks at 24 and 168 hours: yesterday and last week.
- Negative at 12 hours: the opposite phase of the day.



Synthetic hourly load (seed 11, 2,016 hours). Band: approximate 95% range for white noise.

Define the autocorrelation function: the correlation between the series and a copy shifted by k steps. Values computed for this series: r(1) = 0.94, r(12) = −0.15, r(24) = 0.79, r(168) = 0.84 and r(336) = 0.76.

Why is lag 168 higher than lag 24? The same hour last week is also the same weekday, so weekday and weekend shapes line up. Lag 24 compares Monday with Sunday and Saturday with Friday once a week.

Practical consequences: a model that should use the weekly pattern directly needs a lookback of at least 168 hours, or explicit lag-168 and calendar features. The shaded band is only a rough guide; it assumes white noise, and neighbouring autocorrelations of a real series are themselves correlated.

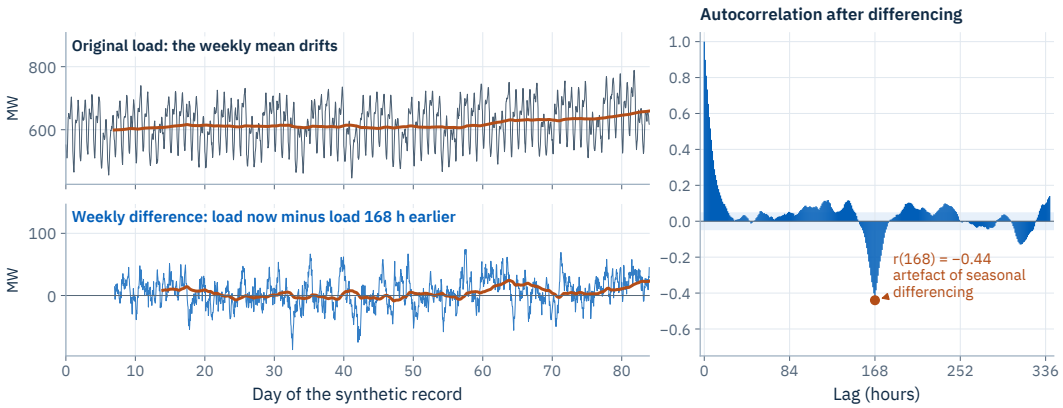
The slow decay of the peaks comes from the trend: a trending series is correlated with itself at almost every lag. That leads to the next slide. [3 min]

Differencing removes trend and seasonality

TIME-SERIES FUNDAMENTALS

Differencing removes trend and seasonality

Weakly stationary: mean, variance and autocorrelation stay the same over time.



Synthetic hourly load (seed 11). Orange: 168-hour rolling means. Right: autocorrelation of the weekly difference.

Neural Networks / Day 5 · Sequence models and attention 10

The original rolling mean drifts, so the series is not stationary. The weekly difference, load now minus load at the same hour last week, removes both the trend and the whole weekly profile. Its rolling mean no longer climbs: it fluctuates around 5.5 MW, the average rise of the level in one week.

Read the right panel with the numbers: after differencing, r(1) = 0.90 because short-term memory remains, r(24) = 0.05 because the daily cycle is gone, and r(168) = −0.44. That negative spike is an artefact: differencing noise at lag 168 creates it. Over-differencing produces such artefacts, so difference only as much as needed.

Why it matters for neural networks: models learn relationships from the training period; if the level drifts, a later period can fall outside what was seen. Differencing, per-window normalization or level features help. Every transform must be fitted on training data and inverted before scoring in the original units. [3 min]

Read the autocorrelation before choosing a lookback

PRACTICE

4 min Pairs

Read the autocorrelation before choosing a lookback

For the hourly load: r(1) = 0.94, r(12) = −0.15, r(24) = 0.79, r(168) = 0.84.

1. Which seasonal-naive forecast does this suggest, and why not lag 24?
2. Why is r(12) negative?
3. For the 10:00 day-ahead case, is lag 24 even available for 18:00 tomorrow?

Answers.

1. The weekly seasonal naive: predict each hour with the same hour one week earlier. Lag 168 has the highest seasonal correlation because it matches the weekday. Lag 24 fails at every weekday-to-weekend change.

2. A 12-hour shift aligns the evening peak with the night trough, so high values pair with low values.

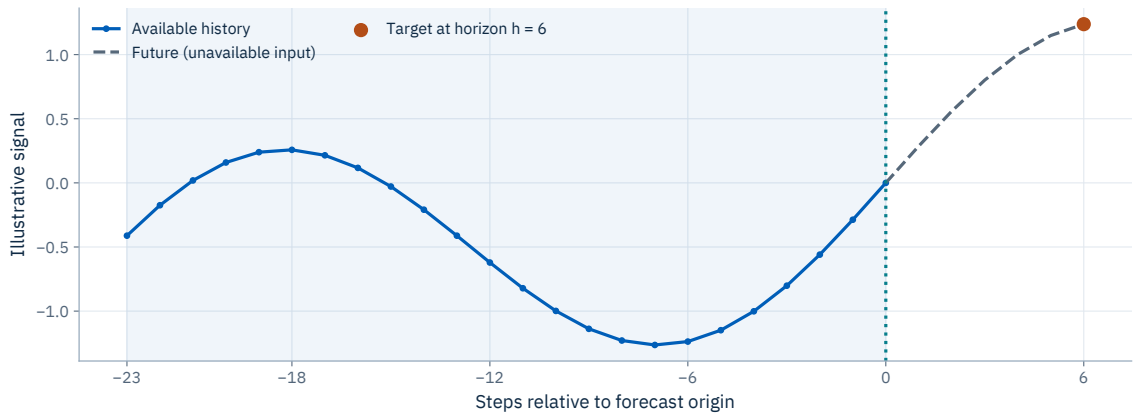
3. No. The target 18:00 Wednesday is 32 hours after the 10:00 Tuesday origin, so lag 24 points at 18:00 Tuesday, which is after the origin and not yet observed. The daily seasonal naive must fall back to 18:00 Monday, lag 48. In general a lag k is usable for horizon h only if k ≥ h. This is the most common leakage in day-ahead feature engineering.

Follow-up question: which lookback would you give a sequence model? At least 168 hours, or a shorter window plus explicit lag-168 and calendar features. [4 min]

A window fixes origin, lookback and horizon

TIME-SERIES FUNDAMENTALS

A window fixes origin, lookback and horizon



Schematic single-origin forecast: lookback 24 observations, horizon 6 steps. Inputs end at the origin; the target lies h steps later.

Neural Networks / Day 5 · Sequence models and attention 12

Define the three quantities we will use all day. The lookback L is the number of past steps given to the model. The horizon h is how many steps after the origin the target lies. The stride s is the distance between consecutive origins when we cut many windows from one series.

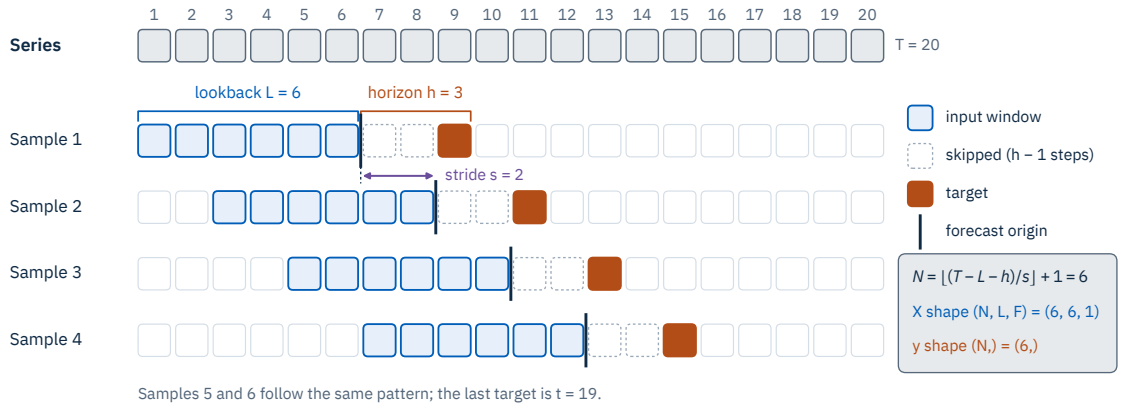
Make sure the axis is read in steps, not hours: with 10-minute data a horizon of 6 steps is one hour. Document the sampling interval next to L and h in every notebook.

Irregular sampling needs explicit treatment: resample to a fixed grid with a documented rule, or give the model the elapsed time between observations as a feature. Gaps must not silently become neighbours. [2 min]

Sliding the origin turns one series into many samples

TIME-SERIES FUNDAMENTALS

Sliding the origin turns one series into many samples



Samples 5 and 6 follow the same pattern; the last target is t = 19.

Schematic with exact indexing: T = 20, L = 6, h = 3 and s = 2 give N = 6 samples.

Neural Networks / Day 5 · Sequence models and attention 13

Walk through sample 1: inputs are steps 1 to 6, the origin is the end of step 6, steps 7 and 8 are skipped, and the target is step 9. Sample 2 starts two steps later. Sample 6 has inputs 11 to 16 and target 19; a seventh sample would need a target at step 21, beyond the end, so step 20 is never a target and there are six samples: N = floor((T − L − h) / s) + 1 = floor(11 / 2) + 1 = 6.

The notebooks use stride 1, so neighbouring windows share all but one input step. The number of windows is therefore far larger than the amount of independent information. Shuffling windows inside the training set is fine for stochastic gradient descent; shuffling across the split boundary is not.

Horizon conventions differ. In the climate notebook, forecast_horizon = 12 means the target is 12 steps after the last input, at index i + L + h − 1. Always check with one printed example; off-by-one errors are common and silently turn a forecast into a nowcast. [3 min]

A window builder in a few lines of NumPy

TIME-SERIES FUNDAMENTALS

A window builder in a few lines of NumPy

```python
import numpy as np
from numpy.lib.stride_tricks import sliding_window_view

def make_windows(series, L, h, s=1):
    """Inputs end at each origin; the target is h steps later."""
    X = sliding_window_view(series[:len(series) - h], L)[::s]
    y = series[L - 1 + h::s][:len(X)]
    return X[..., None], y            # (N, L, 1) and (N,)

X, y = make_windows(np.arange(1.0, 21.0), L=6, h=3, s=2)
print(X.shape, y.shape)
print(X[0, :, 0], y[0])
```

OUTPUT

```
(6, 6, 1) (6,)
[1. 2. 3. 4. 5. 6.] 9.0
```

Build windows after the chronological split, separately for each asset.

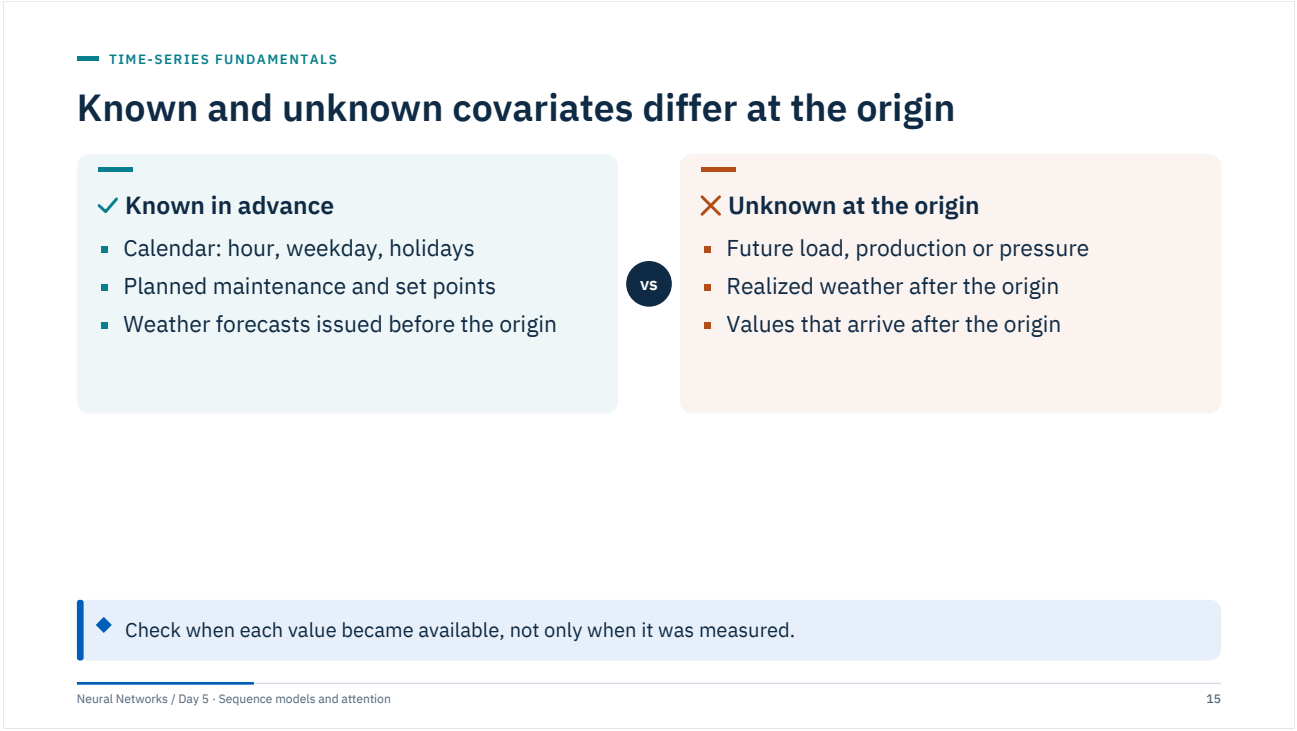
Run through the code with the values 1 to 20 so each number is its own time index. The inputs stop h steps before the end so every window has a target. The first window is 1 to 6 and its target is 9, three steps after the origin, exactly as in the previous diagram.

sliding_window_view returns a read-only view without copying data, which matters for the 70,000-row climate series. Copy before modifying.

For several features, pass a two-dimensional array and slide along time only, or loop as the notebooks do; the notebook helpers are slower but easier to read.

Common errors: building windows on the full series and splitting afterwards, which creates windows whose inputs sit in training and whose targets sit in validation; and forgetting that each well, feeder or pump needs its own windows so that no window joins the end of one asset to the start of another. [2 min]

Known and unknown covariates differ at the origin

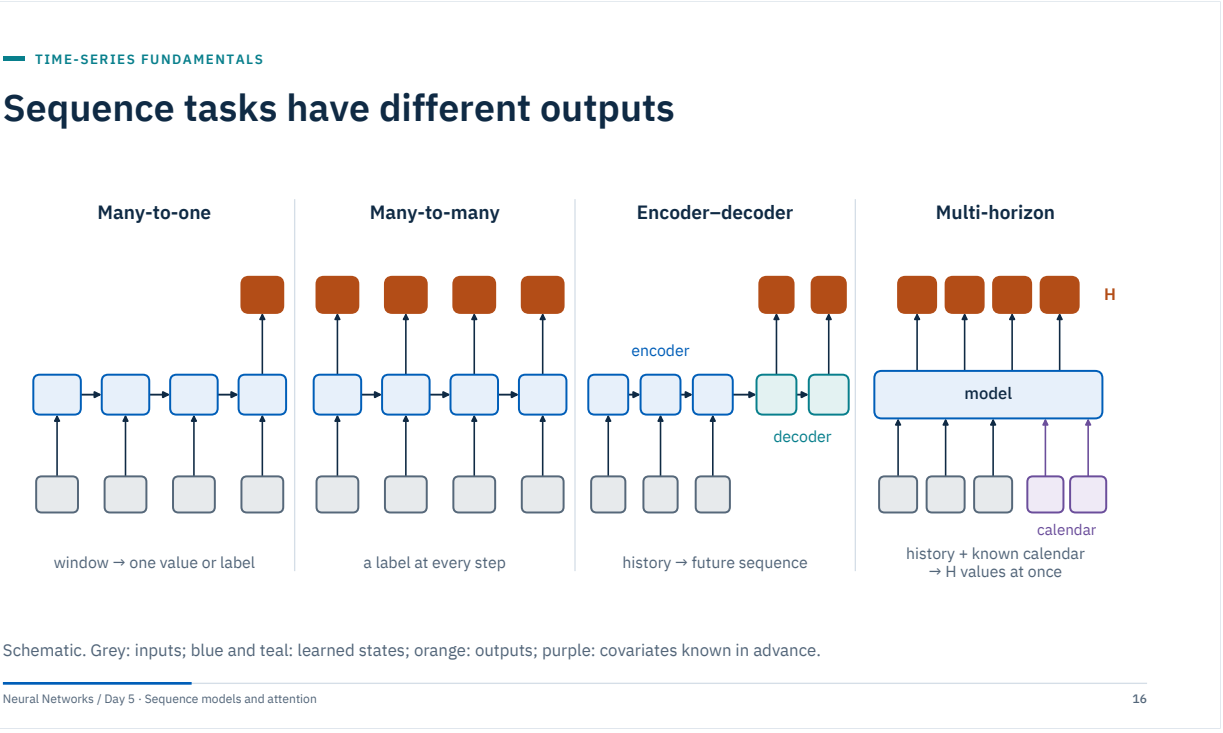


Covariates are the extra inputs besides the target's own history. Known-in-advance covariates are legitimate for every future time step: the calendar, scheduled shutdowns, planned choke or pump set points. A numerical weather forecast is legitimate if it was issued before the origin; store its issue time.

Unknown covariates can only enter as history up to the origin. Two traps are common in operational data. First, latency: SCADA or laboratory values may arrive hours after the measurement time. Second, revision: data are corrected later, and the corrected archive is not what the model would have seen. Use the vintage available at the origin when you can.

Ask the room for one example from their own data where the availability time differs from the timestamp. Typical answers include laboratory fluid analyses and daily allocated production. [2 min]

Sequence tasks have different outputs



Four patterns cover most energy tasks. Many-to-one maps a history window to one future value or event label; the Lab 1 models are many-to-one. Many-to-many produces a label at every position, for example the drilling state at each second. Encoder–decoder reads an input sequence and produces a possibly different-length output sequence. Multi-horizon forecasting maps history plus known future covariates to a vector of future values in one pass, as in the day-ahead case.

A sequence classification task does not automatically forecast anything. Define whether a label describes the current window, for example "gas lock present now", or a future outcome, "gas lock within the next hour". The second is a forecast and needs a horizon and a gap between inputs and label.

Sequence-to-sequence models may need different masking and decoder inputs from one-step regressors; we return to this with teacher forcing in Part 5. [2 min]

Count the windows before you train

PRACTICE

5 min · Individual

Count the windows before you train

The climate lab uses hourly data, lookback L = 72 and horizon h = 12.

1. How many windows does a 1,000-hour segment give with stride 1? With stride 6?
2. The training segment has 49,064 hours. How many training windows?
3. Why does the notebook build windows separately inside each split?

Neural Networks / Day 5 · Sequence models and attention 17

Answers.

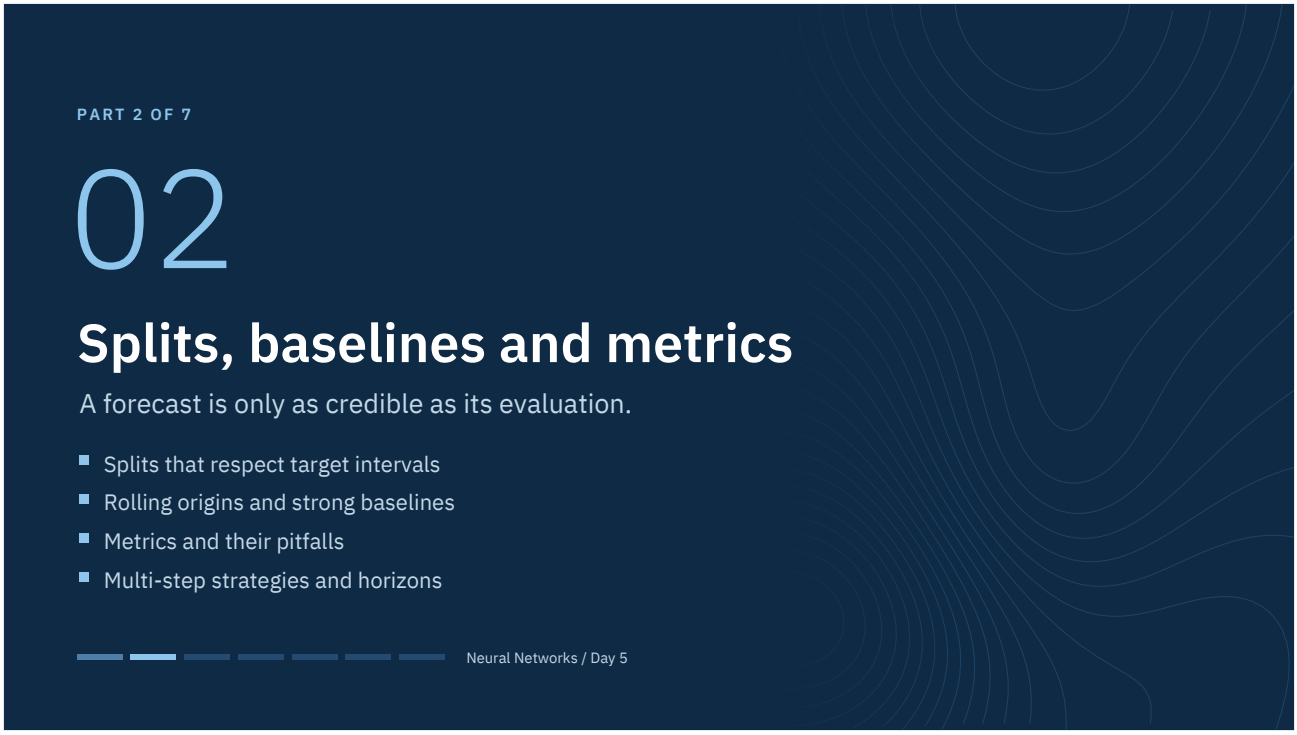
1. N = T − L − h + 1 = 1,000 − 72 − 12 + 1 = 917 with stride 1. With stride 6: floor(916 / 6) + 1 = 152 + 1 = 153.

2. 49,064 − 72 − 12 + 1 = 48,981 training windows. The validation and test segments give 13,935 and 6,927 windows. These counts come from the course file: 420,551 ten-minute rows become 70,092 hourly rows, split 70/20/10 in time.

3. Building windows per split guarantees that no training window has inputs or a target inside the validation period, and that the scaler fitted on training rows is the only one applied. The cost is a few lost windows at each boundary, which is the honest price of the split.

Remind learners that 48,981 overlapping windows share almost all their inputs; the effective sample size is much smaller, which is why validation curves can be noisy. [5 min]

Splits, baselines and metrics



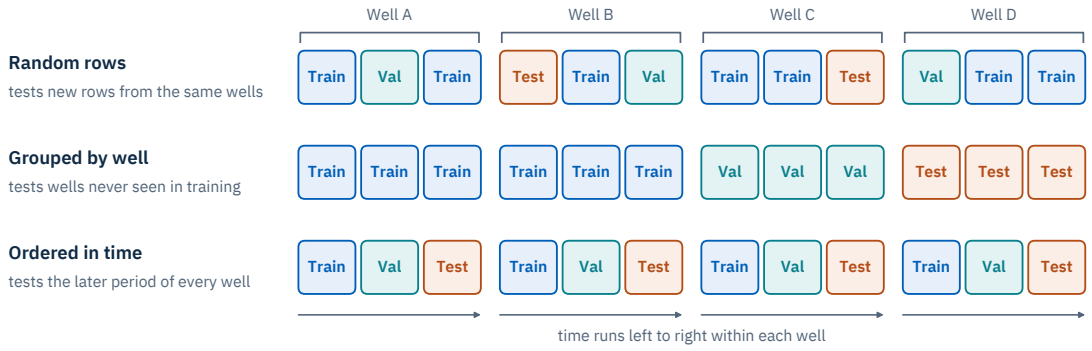
This part is the heart of the day, and it applies to every model family including the ones we have not met yet. Several learners will have seen splits and baselines in the machine learning course; here the twist is time: targets extend into the future and neighbouring samples overlap.

By the end of the part learners should be able to specify, for any forecasting task, the split, the origins, the baselines and the metrics before any network is trained. [1 min]

Time needs a different split from rows or wells

SPLITS, BASELINES AND METRICS

Time needs a different split from rows or wells



Schematic partition assignments: a time split holds out the later period of every well. The machine learning course covers these splits in more depth.

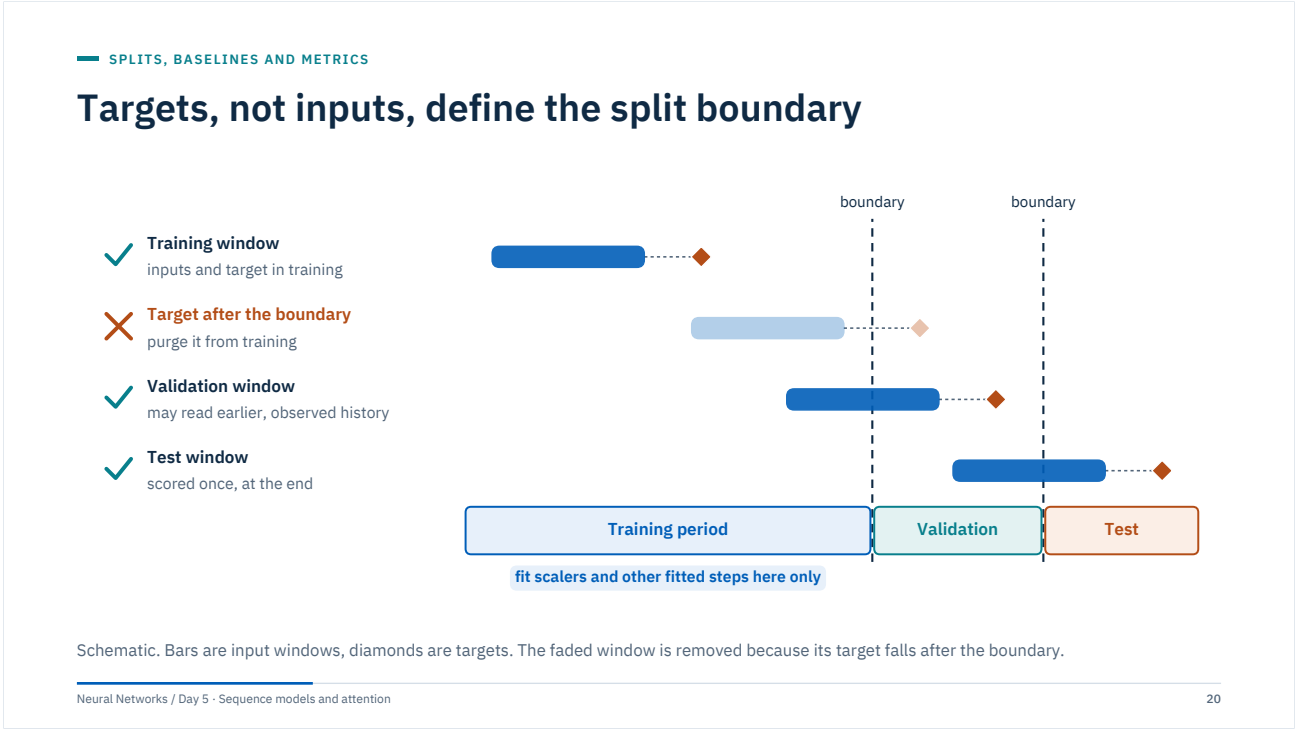
Neural Networks / Day 5 · Sequence models and attention 19

A quick recap; the machine learning course covers these splits in more depth. Random rows test new rows from wells the model has already seen. Grouped splits test wells never seen in training. Ordered splits test the future after the training period.

Sequence forecasting almost always needs the ordered split, and often both: a production forecast for new wells in a later period needs wells and time held out together.

Ask: which split matches "forecast next month's rate for the wells we operate today"? Ordered in time, with all wells present in training. Which matches "forecast the first year of a well we have not drilled"? Grouped by well, and ideally later in time as well. [2 min]

Targets, not inputs, define the split boundary

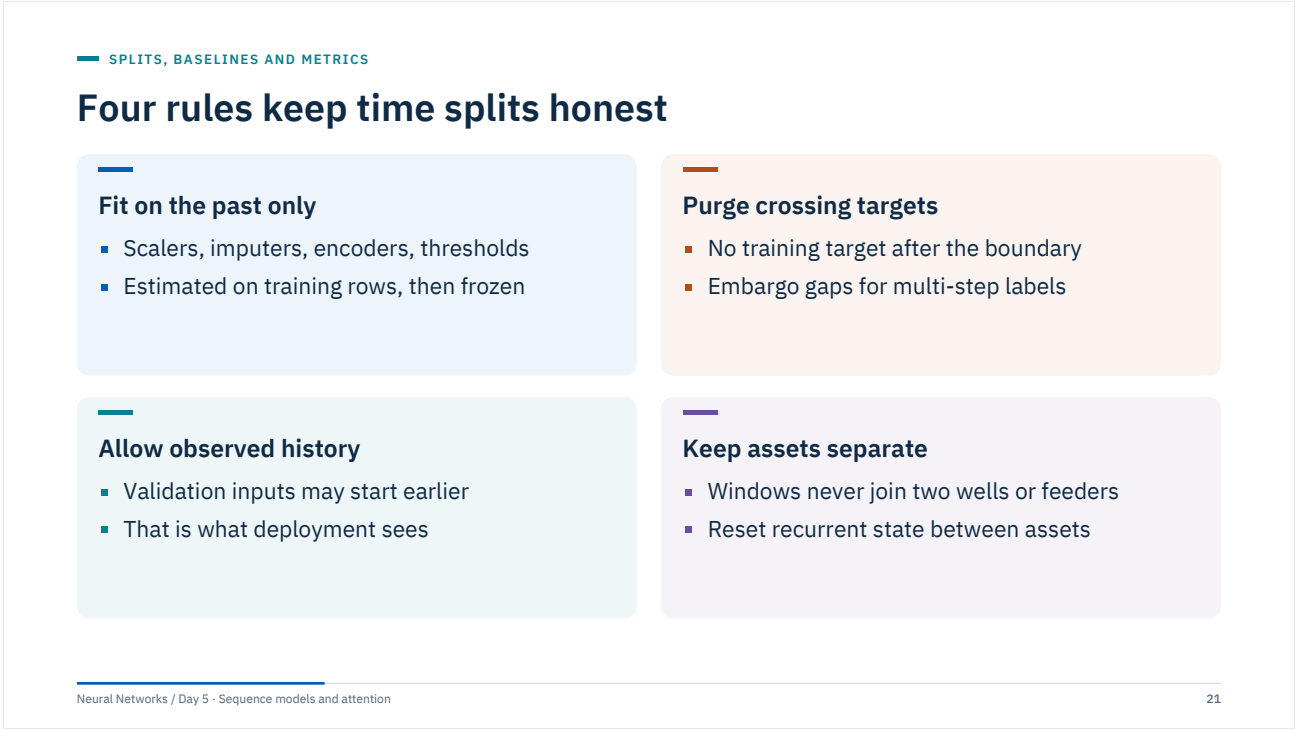


Do not teach that every overlap of input history is leakage. A validation window at 09:00 on the first validation day may legitimately read training-period observations; that is exactly what the deployed model will see. Leakage happens when fitting uses future targets or inputs that would not be available.

The dangerous windows are training windows whose targets lie after the boundary. Purge them: the last training origin is the boundary minus the horizon. For multi-step targets, or labels computed over an interval such as a 24-hour average, add an embargo gap of at least the label length.

Fitted preprocessing, including scalers, imputers and any learned threshold, is estimated on the training period only and then applied unchanged to validation and test. [2 min]

Four rules keep time splits honest



Summarize the previous diagram as four rules learners can check in any notebook.

Fit on the past only: this includes less obvious fitted steps such as a missing-value fill computed from the median, a clipping range, or the anomaly threshold in Lab 2.

Purge crossing targets: with horizon h, the last h training origins before the boundary would have targets in validation.

Allow observed history: it is valid to use historical observations known at a validation origin. Excluding them would make validation harder than deployment and misreport performance.

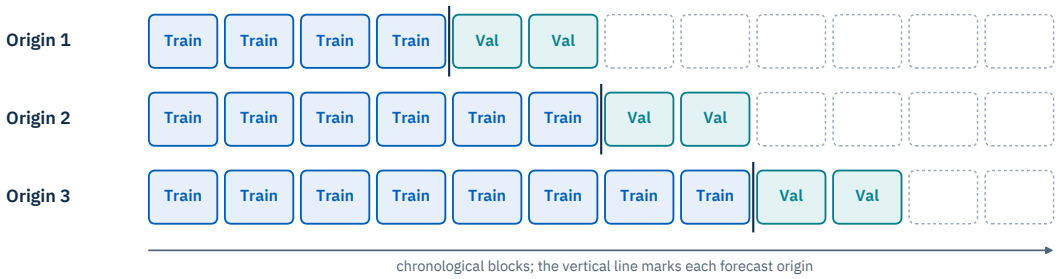
Keep assets separate: windows are built per asset, and a recurrent state carried across batches must be reset between assets.

Ask: which of the four rules does the Lab 1 climate notebook implement, and how? It splits first, fits the scaler on the training rows and builds windows per split. [2 min]

Rolling-origin evaluation repeats the real question

SPLITS, BASELINES AND METRICS

Rolling-origin evaluation repeats the real question



Schematic expanding-window evaluation. Each fold forecasts from information available at its origin; keep a final test period outside all folds.

Neural Networks / Day 5 · Sequence models and attention 22

A single validation block gives one estimate that depends on one season. Rolling-origin evaluation, sometimes called time-series cross-validation, repeats the forecast from several origins, each time training on the past and scoring the next block.

Refit preprocessing within each fold whenever the protocol retrains. Report the mean and the spread across origins; a model that wins on average but loses badly in winter may be unusable.

Keep a final test block after all folds and use it once. A fixed-origin forecast and an updating rolling forecast are different operational scenarios; label which one you report.

Expanding windows keep all past data; sliding windows of fixed length adapt faster to regime changes. Both are legitimate if they match how the model will be retrained in operation. [2 min]

Strong baselines can be very simple

SPLITS, BASELINES AND METRICS

Strong baselines can be very simple

Baseline	Forecast of the value h steps ahead	Informative when
Persistence	The last observed value	Slowly changing signals
Seasonal naive	Same hour, one season earlier	Repeating daily or weekly cycles
Moving average	Mean of the last k values	A noisy level without cycles
Exponential smoothing	A level updated with weight α	A level that drifts slowly
Lag model	Linear or boosted on lags and calendar	A tabular benchmark

Score every model on the same origins, targets and horizons.

◆ A model that cannot beat these has not yet earned its complexity.

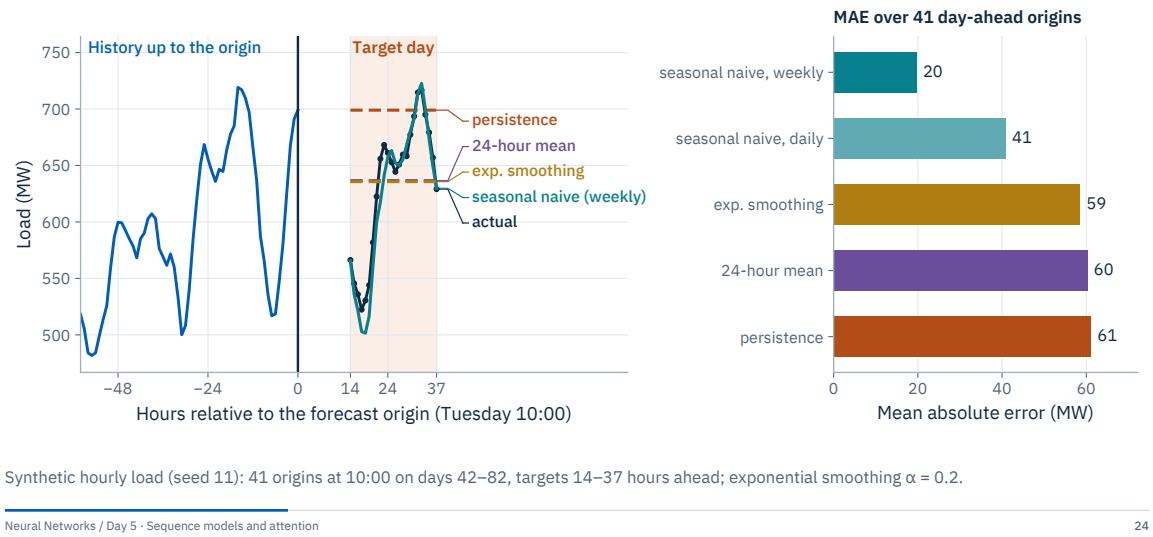
Neural Networks / Day 5 · Sequence models and attention 23

Introduce the baselines every sequence model must beat. Persistence repeats the last value. Seasonal naive repeats the value one season earlier, using an older season when the recent one is not yet observed. The moving average is the mean of the last k values. Simple exponential smoothing updates a level as $\ell(t) = \alpha\, y(t) + (1 - \alpha)\, \ell(t - 1)$ and forecasts that level for every horizon. The lag model is a linear or gradient-boosted model on engineered lags and calendar features, the strongest tabular benchmark.

Do not compare one model's one-step forecast with another's long-horizon recursive forecast. A naive baseline can be very hard to beat on short or highly persistent series.

Seasonal variants of exponential smoothing (Holt–Winters) add trend and seasonal components; they are described in Hyndman and Athanasopoulos, Forecasting: Principles and Practice. [2 min]

Seasonal naive wins at day-ahead horizons



Left: one origin, Tuesday 10:00. Persistence repeats the 10:00 value all day; the 24-hour mean and exponential smoothing are flat lines at the average level; the weekly seasonal naive reproduces Wednesday's shape from last Wednesday and tracks the actual closely.

Right: mean absolute error over 41 origins. Weekly seasonal naive 19.7 MW, daily seasonal naive 40.9 MW, exponential smoothing 58.5 MW, 24-hour mean 60.3 MW, persistence 61.2 MW.

The daily naive loses to the weekly one on the days around the weekend. For hours after 10:00 it must fall back two days, because yesterday's value at that hour is not yet known, so Monday and Tuesday afternoon are copied from weekend days, and Saturday and Sunday afternoon from weekdays. Midweek the two naive forecasts score about the same.

Ask: an LSTM reaches 25 MW on this task. Is that good? No. It is worse than a one-line baseline, however impressive its training curve looks. [3 min]

Exponential smoothing by hand

PRACTICE

3 min · Individual

Exponential smoothing by hand

Simple exponential smoothing with $\alpha = 0.5$ starts from a level $\ell_0 = 100$ MW.

1. The next two observations are 110 and 90 MW. Compute ℓ_1 and ℓ_2.
2. What is the forecast for 1 hour ahead, and for 24 hours ahead?
3. What does the method become when α approaches 1?

Answers.

1. $\ell_1 = 0.5 \times 110 + 0.5 \times 100 = 105$ MW. $\ell_2 = 0.5 \times 90 + 0.5 \times 105 = 97.5$ MW.

2. Simple exponential smoothing produces a flat forecast: 97.5 MW for every horizon, one hour or twenty-four hours ahead. That is why it lost on the previous slide: a seasonal series needs seasonal components.

3. With α close to 1 the level equals the latest observation, so the method becomes persistence. With α close to 0 the level barely moves and approaches a long-run average.

Follow-up: the weights on past observations are α, $\alpha(1-\alpha)$, $\alpha(1-\alpha)^2$, and so on, a geometric decay. That is the same "fraction retained" idea as the LSTM forget gate later this morning. [3 min]

Forecast metrics answer different questions

SPLITS, BASELINES AND METRICS

Forecast metrics answer different questions

Metric	Definition	Watch out for
MAE	Mean absolute error	In target units; robust to single misses
RMSE	Root of the mean squared error	Dominated by the largest misses
MAPE	Mean of absolute error ÷ actual	Undefined at zero, explodes near zero
MASE	MAE ÷ in-sample naive MAE	Below 1 beats the naive method

Report errors by horizon and by asset, not only one average.

◆ Choose the metric before seeing any results, and use it for every model.

Neural Networks / Day 5 · Sequence models and attention 26

MAE is in the units of the target, megawatts or degrees, and is easy to explain. RMSE penalizes large misses more, which suits applications where one large error is costly, such as reserve shortfalls. The two answer different questions, and the model that minimizes one does not always minimize the other.

MAPE is popular with managers because it is a percentage, but it divides by the actual value: it is undefined when the actual is zero and explodes when the actual is small. Wind and solar output, intermittent retail demand and shut-in wells all contain zeros.

MASE, the mean absolute scaled error, divides the MAE by the in-sample MAE of a naive forecast. It is scale-free and defined when the series contains zeros.

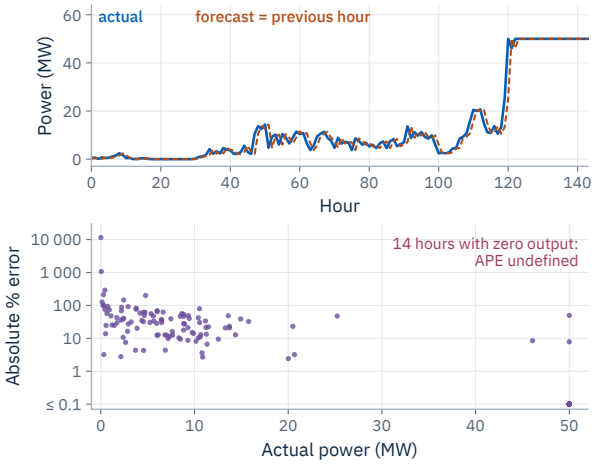
Whatever the metric, break it down by horizon, asset and regime. A single average hides the hours that matter. [2 min]

MAPE explodes when actual values approach zero

SPLITS, BASELINES AND METRICS

MAPE explodes when actual values approach zero

- One forecast: MAE = 1.7 MW, median APE = 23%.
- MAPE = 130%, driven by a few low-output hours.
- 14 zero-output hours cannot enter MAPE at all.



◆ For wind and solar, divide MAE by installed capacity instead.

Synthetic 50 MW wind farm (seed 3), 144 hours; forecast = previous hour's output.

Neural Networks / Day 5 · Sequence models and attention 27

Top panel: the forecast is simply the previous hour, and it tracks the actual reasonably. Bottom panel: absolute percentage error against the actual output. The same size of error is 10% at 20 MW and several hundred percent at 0.3 MW.

Numbers for this window: MAE 1.70 MW over 144 hours; median absolute percentage error 23%; MAPE over the 130 hours with non-zero output 130%; 14 hours had exactly zero output, where MAPE is undefined and software either crashes, returns infinity or silently drops them.

Capacity-normalized MAE is the usual alternative for renewables: 1.70 / 50 = 3.4% of capacity. The retail data in Lab 2 option B have many zero-sales days, so MAPE is unusable there too.

Ask: why might a team prefer MAPE anyway? Because it looks comparable across assets. MASE or capacity-normalized errors give that comparability without the division by zero. [2 min]

MASE compares your error with a naive forecast

SPLITS, BASELINES AND METRICS

MASE compares your error with a naive forecast

$$\mathrm{MASE} = \frac{\mathrm{MAE}}{\frac{1}{n-m}\sum_{t=m+1}^{n} |y_t - y_{t-m}|}$$

- The denominator uses the training period only.
- m = 1 scales by persistence; m = 24 or 168 by a seasonal naive.
- MASE below 1 means better than that naive method, on average.
- Scale-free, and defined when the series contains zeros.

◆ Example: MAE 18 MW against an in-sample weekly-naive MAE of 24 MW gives MASE = 0.75.

Neural Networks / Day 5 · Sequence models and attention 28

Hyndman and Koehler proposed the mean absolute scaled error in 2006 to fix the problems of percentage errors. The numerator is the MAE of your forecast on the evaluation period. The denominator is the MAE that the naive method with season length m achieved in-sample, on the n observations of the training period.

Interpretation: 0.8 means your errors are 20% smaller than the naive method's typical in-sample error. With m = 1 that is a one-step error, so multi-step forecasts can have MASE above 1 and still be useful; compare MASE across models at the same horizon, not against the number 1 alone.

Choose m to match the strongest simple baseline: for hourly load, m = 168 is a demanding scale.

Common error: computing the denominator on the test period. That leaks information and makes MASE depend on the evaluation window. [2 min]

Reference: https://doi.org/10.1016/j.ijforecast.2006.03.001

Score one forecast four ways

PRACTICE

5 min Pairs

Score one forecast four ways

Wind power (MW). Actual: 4, 2, 0.5, 6. Forecast: 3, 3, 1.5, 5. Training one-step naive MAE: 2.25 MW.

1. Compute MAE and RMSE.
2. Compute MAPE. Which hour dominates it?
3. Compute MASE and interpret it.

Neural Networks / Day 5 · Sequence models and attention 29

Answers.

1. Errors (actual – forecast) are 1, −1, −1, 1. MAE = 1.0 MW. RMSE = √((1 + 1 + 1 + 1) / 4) = 1.0 MW. Equal because all errors have the same size.

2. Absolute percentage errors: 1/4 = 25%, 1/2 = 50%, 1/0.5 = 200%, 1/6 = 16.7%. MAPE = (25 + 50 + 200 + 16.7) / 4 = 72.9%. The 0.5 MW hour contributes more than the other three together, although its absolute error is the same.

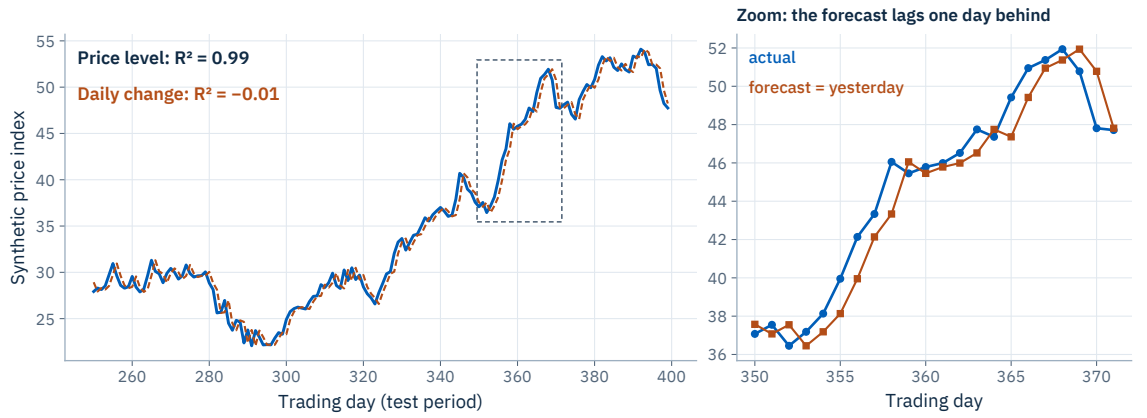
3. MASE = 1.0 / 2.25 = 0.44. The forecast's errors are less than half of the typical one-step naive error on the training period, which is a strong result.

The training series used for the denominator was 3, 5, 4, 6, 2 MW: absolute one-step changes 2, 1, 2 and 4, mean 2.25. Ask a pair to verify this on the board. [5 min]

A price model can look perfect by copying yesterday

SPLITS, BASELINES AND METRICS

A price model can look perfect by copying yesterday



Synthetic Gaussian random walk (seed 21), test days 250–399; the forecast is simply the previous day's value.

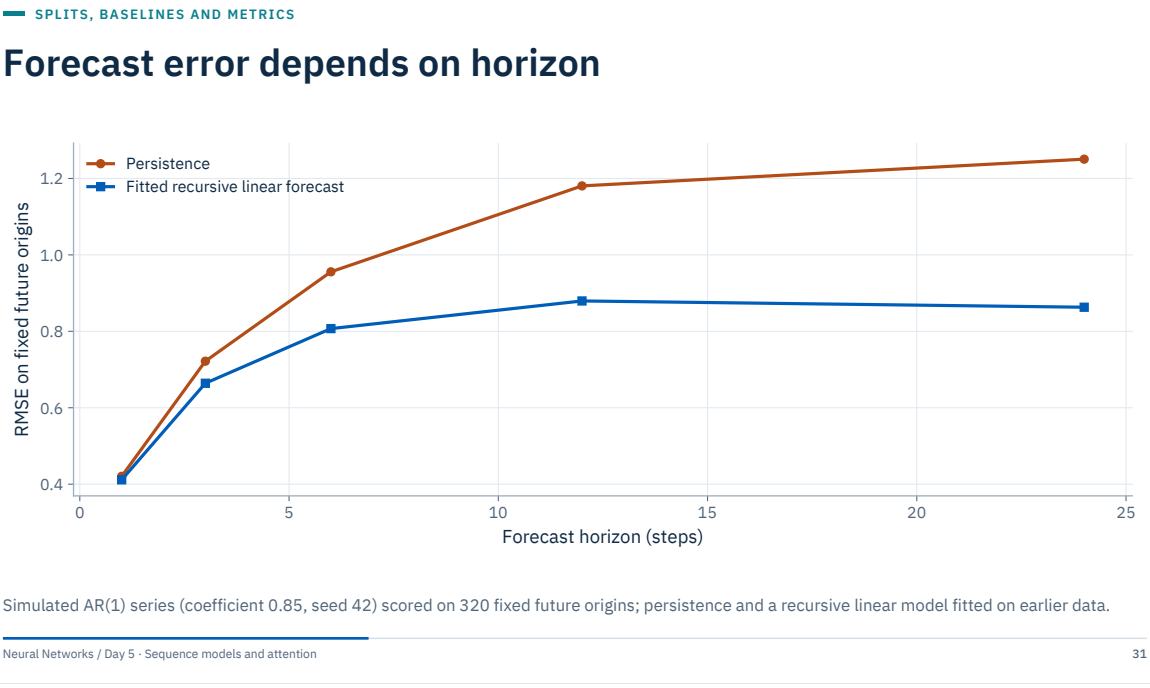
Neural Networks / Day 5 · Sequence models and attention 30

This slide prepares the caution in notebook 05 (stock prediction). The "forecast" is yesterday's value. On price levels it looks almost perfect: $R^2 = 0.99$. On daily changes, the thing a trader or planner actually needs, $R^2 = -0.01$: no skill at all. The zoom shows the tell-tale sign, a forecast that lags the actual by one step.

The same illusion appears in energy with highly persistent series: reservoir pressure, cumulative production, tank levels. A neural network trained on levels often learns to output something close to the last input, and a line plot over hundreds of days hides the lag.

Defences: always compare with persistence on the same days; evaluate changes or returns when the level is persistent; look at a zoomed window; and remember the notebook's own caveats about efficient markets. The stock notebook is a method demonstration, not evidence of trading value. [2 min]

Forecast error depends on horizon



This computed simulation illustrates reporting errors by horizon. Persistence keeps getting worse as the horizon grows, while the fitted model levels off once its forecast has relaxed to the mean. The gap between them changes with horizon: at one step they are almost equal, at 12 and 24 steps the fitted model is clearly better because persistence ignores the pull back towards the mean.

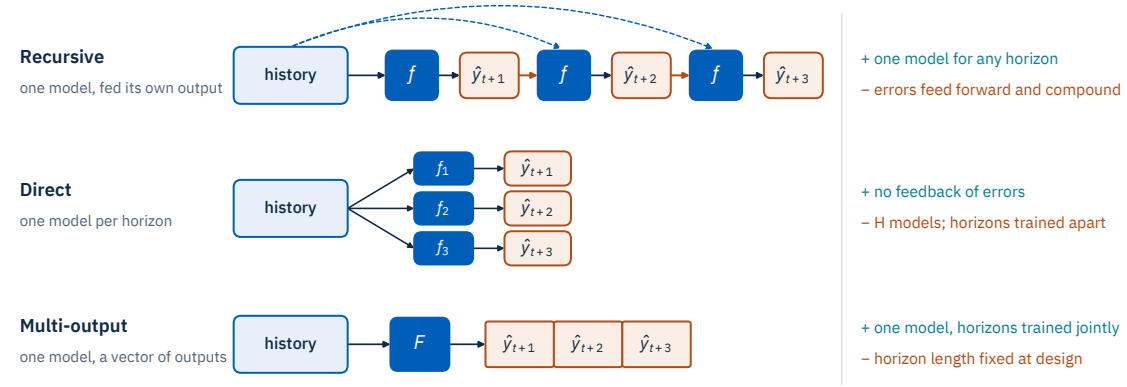
It is not a performance claim about LSTMs, transformers or field data.

Ask: why can an aggregate average hide an unacceptable long-horizon forecast? Because the average over horizons 1 to 24 mixes easy and hard targets; a model can win on average by being excellent at short horizons and useless at the horizons the operator cares about. Report a curve or a table by horizon, with the same origins for every model. [2 min]

Multi-step forecasts need a strategy

SPLITS, BASELINES AND METRICS

Multi-step forecasts need a strategy



Schematic of three strategies for horizons 1–3. Dashed arrows: the recursive model also re-reads the observed history.

Neural Networks / Day 5 · Sequence models and attention 32

Recursive: train one one-step model and feed its own predictions back as inputs to reach longer horizons. One model serves every horizon, but errors feed forward.

Direct: train a separate model for each horizon from the same history. There is no feedback of errors, but you train H models, each seeing only its own target.

Multi-output: one model outputs the whole vector of future values at once; for a neural network this is simply a final layer with H outputs, for example nn.Linear(64, 24) for a day-ahead hourly forecast. Most neural forecasters use this form.

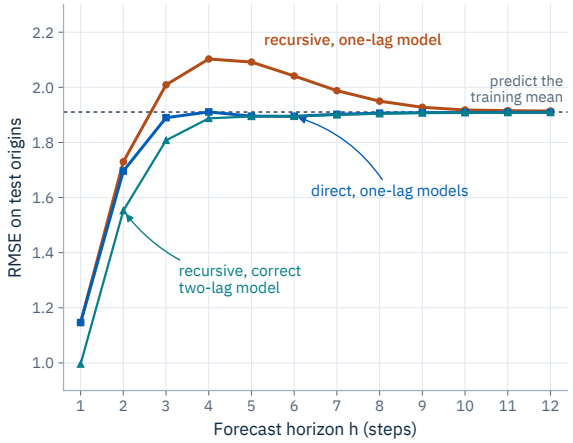
The climate notebook is a direct model for a single horizon of 12 hours. The day-ahead case would suit a multi-output model. Hybrids exist, and sequence-to-sequence decoders are recursive inside the network, which is why teacher forcing matters later. [3 min]

A misspecified recursive model compounds its error

SPLITS, BASELINES AND METRICS

A misspecified recursive model compounds its error

- True process uses two lags; the one-step model uses one.
- From h = 3, recursive errors exceed the mean forecast.
- Direct models level off at the mean-forecast error.
- With the correct two-lag model, recursion works well.



Simulated AR(2) process (seed 8); models fitted on the first 5,000 steps, scored on 1,986 later origins.

The data come from a known oscillating process, y(t) = 1.2 y(t−1) − 0.5 y(t−2) + noise. A one-lag model fitted by least squares has coefficient 0.80. Iterating it four times predicts $0.80^4 = 0.41$ times the current value, but the true correlation at lag 4 is close to zero, so the recursive forecast confidently predicts the wrong thing.

Numbers: at horizon 4 the recursive one-lag model has RMSE 2.10, worse than predicting the training mean (1.91), and it stays above that line at every longer horizon. The direct one-lag models, each fitted to its own horizon, level off at about 1.91. The correct two-lag model used recursively is best up to four steps, 1.00 at one step and 1.89 at four, and ties with the direct models beyond that, where every method approaches the mean forecast.

Lessons: recursion amplifies misspecification; direct or multi-output forecasts avoid feedback; and whatever the strategy, evaluate the full rollout at the horizons that matter, never only the one-step error. [3 min]

Design the evaluation for a wind forecast

PRACTICE

5 min · Pairs

Design the evaluation for a wind forecast

A 50 MW wind farm needs hourly forecasts 1–48 hours ahead, issued every day at 06:00. You have two years of data.

1. Which baselines must a neural model beat?
2. Which metric would you report, broken down how?
3. Sketch the split and the rolling origins.

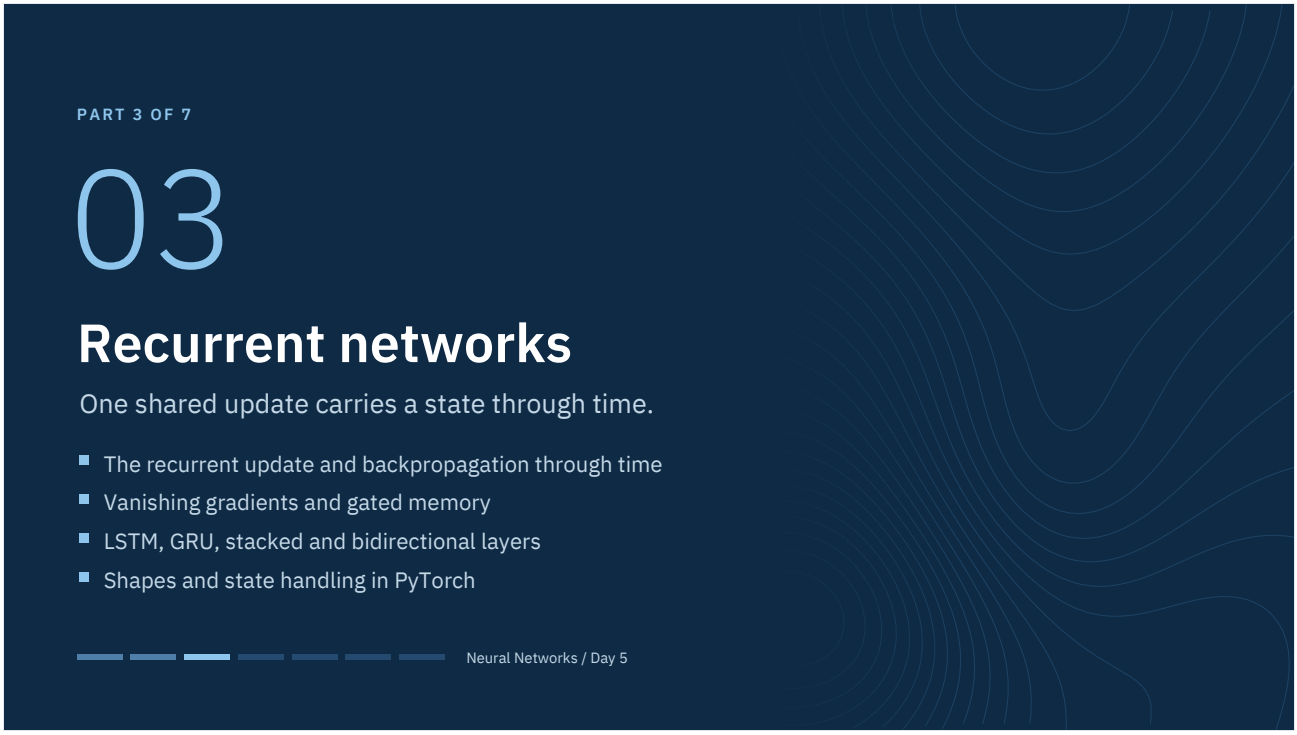
Model answer.

Baselines: persistence, which is strong for the first few hours; a climatological forecast such as the mean output for that hour of day and month; and, if available, a forecast derived from numerical weather prediction through a power curve, the usual industrial benchmark. A daily seasonal naive is weak for wind because the daily cycle is weak.

Metric: MAE and RMSE divided by the 50 MW capacity, reported for each horizon from 1 to 48 and by season. No MAPE: there are many zero-output hours. If intervals are produced, add coverage and pinball loss.

Split: train on months 1–15; rolling origins every day at 06:00 over months 16–20, retraining monthly if that is the deployment plan; final test months 21–24, used once. Fit scalers per fold on training data only, purge the last 48 hours before each boundary, and never shuffle across time. [5 min]

Recurrent networks



Connect recurrence to what learners already know. A dense network from Day 1 has a fixed number of inputs. A recurrent network processes a variable number of steps with the same parameters, by carrying a state from one step to the next.

The state is a learned summary of the history, not a perfect memory of every past observation. Everything in this part is about how well that summary can preserve what matters, and how gradients reach old inputs during training.

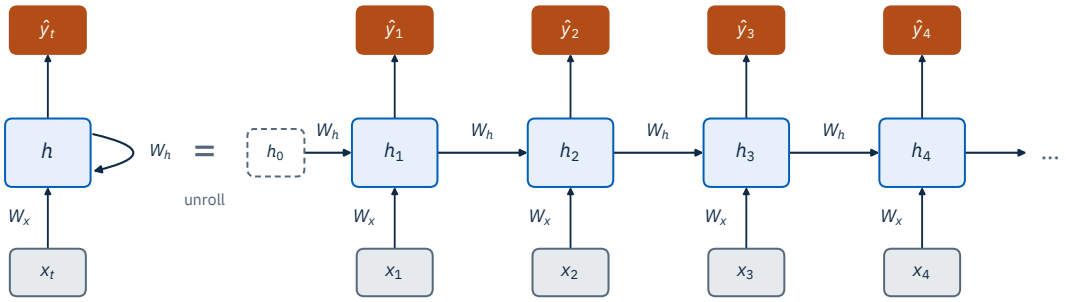
The part ends with a shape and parameter check on the exact model used in Lab 1, which starts straight afterwards. [1 min]

One recurrent cell is reused at every step

RECURRENT NETWORKS

One recurrent cell is reused at every step

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$



Schematic. The same weights W_x, W_h and bias b are applied at every time step; h_0 is usually zeros.

Neural Networks / Day 5 · Sequence models and attention 36

The current input and the previous hidden state determine the new state through one shared update. Unrolling the loop shows that a recurrent network over 24 steps is a very deep network with tied weights.

An output head maps a chosen state to a prediction. For many-to-one forecasting, as in the labs, use the state after the last valid input. For padded sequences, the last array position may be padding rather than the final observation, so track lengths explicitly or use packed sequences.

Tensor layout matters: with batch_first=True the input is (batch, time, features). The hidden state has size H regardless of the window length, which is why the same model can read windows of different lengths.

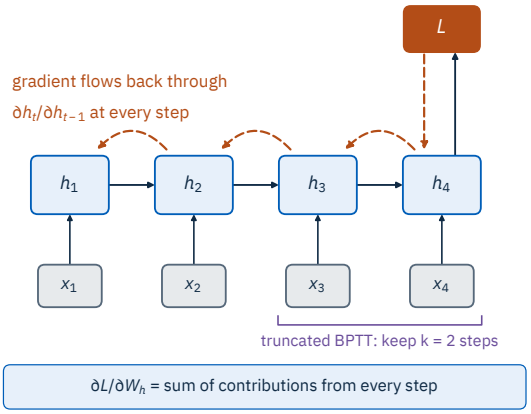
Ask: how many parameters does this cell have for F inputs and H hidden units? H·F + H·H + H with one bias; PyTorch stores two bias vectors, giving H·F + H·H + 2H. [3 min]

Backpropagation through time follows the unrolled graph

RECURRENT NETWORKS

Backpropagation through time follows the unrolled graph

- Unroll the shared recurrence over the training window.
- Gradients for W_h add up contributions from every step.
- Long products of derivatives can shrink or amplify.
- Truncated BPTT limits history to manage compute.



Schematic for a many-to-one RNN; dashed arrows carry gradients backwards through the recurrence.

Neural Networks / Day 5 · Sequence models and attention 37

Backpropagation through time is ordinary backpropagation, from Day 1, applied to the unrolled graph. Because the same W_h appears at every step, its gradient is the sum of the contributions from each step.

To reach step 1 from a loss at step 4, the gradient is multiplied by the derivative of each state with respect to the previous one, three times. That product is the source of vanishing and exploding gradients, on the next slide.

Truncated backpropagation stops the gradient after k steps. It is an optimization choice that limits compute and memory, and it can limit learning of dependencies longer than k.

In PyTorch, detach the state at intended boundaries, for example hidden = hidden.detach() when you carry state between batches, rather than accidentally retaining the computation graph across the whole data set. Reset states between unrelated assets. [3 min]

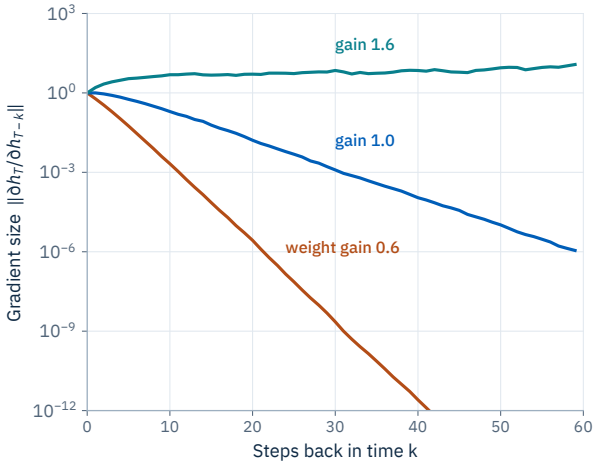
Gradients shrink geometrically through time

RECURRENT NETWORKS

Gradients shrink geometrically through time

$$\frac{\partial h_T}{\partial h_{T-k}} = \prod_{j=T-k+1}^{T} \operatorname{diag}(1 - h_j^2)\, W_h$$

- Each step multiplies the gradient by a Jacobian.
- Gain 0.6: about 3 × 10⁻⁶ of the signal after 20 steps.
- Even gain 1.0 decays, because tanh slopes are below 1.



Computed for a 32-unit tanh RNN with random inputs (seed 4); median over 30 runs.

Neural Networks / Day 5 · Sequence models and attention 38

This is a real computation, not a sketch: random recurrent matrices with orthogonal structure, scaled by a gain, driven by random inputs; we multiply the actual Jacobians and take the spectral norm.

Values after 20 steps: gain 0.6 gives 2.7 × 10⁻⁶, gain 1.0 gives 0.016, gain 1.6 gives about 5. The derivative of tanh is at most 1 and usually smaller, so even a perfectly norm-preserving weight matrix loses signal. With larger gains the norms can grow; saturation limits the growth in this example, but exploding gradients do occur in practice.

Remedies: gradient clipping handles explosions, for example torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0). Careful initialization, from Day 2, helps at the start of training. Gated architectures address the decay, which is the next slide.

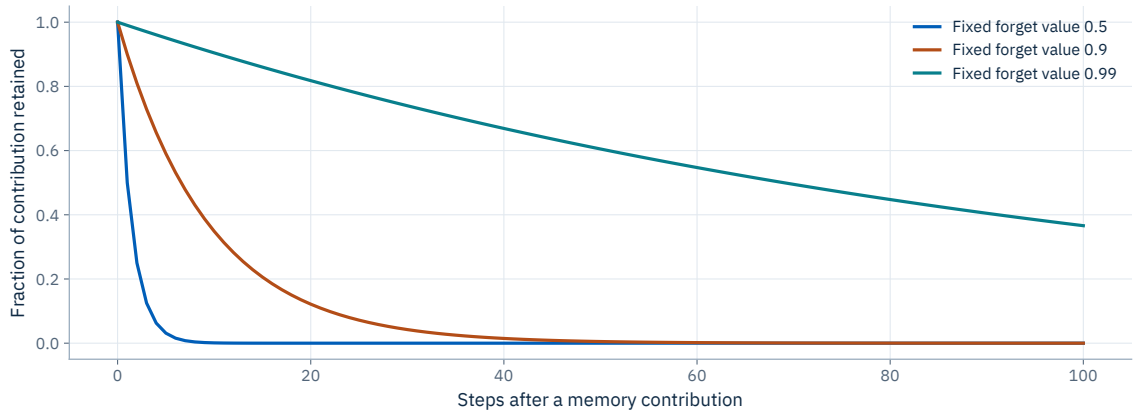
References: Bengio, Simard and Frasconi (1994) and Pascanu, Mikolov and Bengio (2013). [3 min]

Reference: https://arxiv.org/abs/1211.5063

Gated memory offers a more direct path

RECURRENT NETWORKS

Gated memory offers a more direct path



Simplified retained-signal magnitude under fixed forget-gate values. Real LSTM gates depend on inputs and state.

Neural Networks / Day 5 · Sequence models and attention 39

Instead of multiplying by a weight matrix and a tanh slope at every step, an LSTM carries a cell state that is multiplied only by a forget gate between 0 and 1. The plot shows the fraction of a contribution that survives: with a forget value of 0.5 it is gone in about 10 steps; with 0.9, 35% remains after 10 steps; with 0.99, 37% remains after 100 steps.

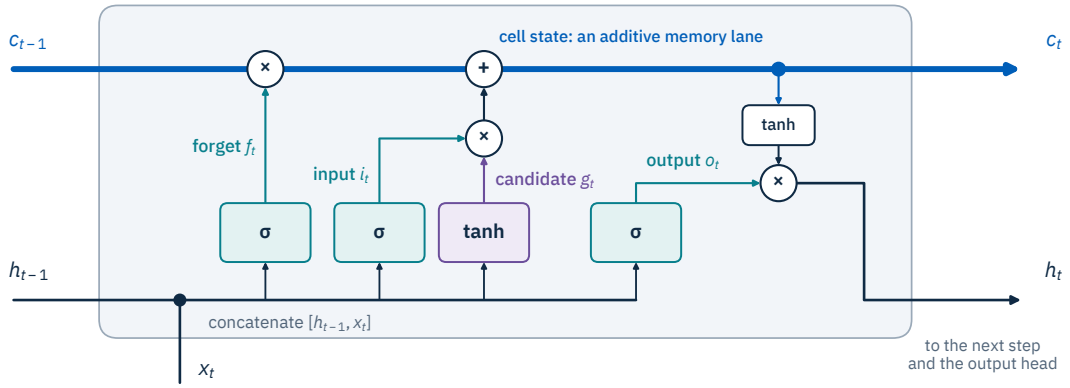
The plot illustrates how a forget factor near one preserves a signal longer. It does not prove that an LSTM automatically learns the right long-term dependencies or avoids all gradient problems; the gates are learned and data-dependent.

Link back to exponential smoothing: a fixed forget value is exactly a geometric weighting of the past. The LSTM learns when to keep and when to overwrite. [2 min]

An LSTM cell adds a protected memory lane

RECURRENT NETWORKS

An LSTM cell adds a protected memory lane



Schematic of one LSTM step. Teal: sigmoid gates; purple: candidate values; blue: cell state. × and + act elementwise.

Neural Networks / Day 5 · Sequence models and attention 40

Walk through the diagram from left to right. The previous hidden state and the current input are concatenated and feed four small layers.

The forget gate f multiplies the old cell state: what to keep. The input gate i multiplies the candidate g: how much new information to write. The sum is the new cell state c(t). The output gate o multiplies tanh of the new cell state to give the hidden state h(t), which goes to the next step and to the output head.

The top lane is the key design idea: from c(t−1) to c(t) the only operations are an elementwise multiplication and an addition, so the local derivative along the lane is just f(t). Gradients along the lane are products of forget gates rather than of weight matrices.

The original LSTM (Hochreiter and Schmidhuber, 1997) had no forget gate; it was added by Gers, Schmidhuber and Cummins in 2000. In PyTorch the four weight blocks are stored stacked in the order i, f, g, o. [4 min]

Four gates define the LSTM update

RECURRENT NETWORKS

Four gates define the LSTM update

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \qquad i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$
$$g_t = \tanh(W_g[h_{t-1}, x_t] + b_g), \qquad o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$
$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \qquad h_t = o_t \odot \tanh(c_t)$$

- Gates use sigmoids (0 to 1); the candidate uses tanh (−1 to 1).
- ⊙ multiplies elementwise; c and h both have H values.

◆ PyTorch stacks the four weight blocks in the order i, f, g, o.

Neural Networks / Day 5 · Sequence models and attention 41

Read the equations with the diagram still in mind. Each gate is a small dense layer on the concatenated vector [h(t−1), x(t)], followed by a sigmoid, so every gate value lies between 0 and 1 for every hidden unit separately. The candidate g uses tanh so that new content can be positive or negative.

The bracket notation is equivalent to PyTorch's separate input and hidden weight matrices: W[h, x] = W_hh h + W_ih x. PyTorch also keeps two bias vectors per gate.

Parameters per layer: 4H(F + H) weights plus 8H biases in PyTorch. For F = 14 inputs and H = 64 units that is 4 × 64 × 78 + 512 = 20,480.

The additive memory path can help preserve gradients, but it is not a guarantee of indefinite memory: if the network learns forget values well below one, memory fades quickly. [3 min]

A memory update by hand

PRACTICE

4 min · Individual

A memory update by hand

Previous cell value = 2. Forget gate = 0.9. Input gate = 0.2. Candidate = −1.

1. Calculate the new cell value.
2. With output gate 0.5, compute the hidden state.
3. With the forget gate fixed at 0.9 and no new input, what fraction of the memory remains after 10 steps?

Answers.

1. $c = 0.9 \times 2 + 0.2 \times (-1) = 1.8 - 0.2 = 1.6$.

2. $h = 0.5 \times \tanh(1.6) = 0.5 \times 0.922 = 0.461$.

3. $0.9^{10} = 0.349$, so about 35% remains, the orange curve on the gated-memory slide.

Explain that in a full model the gates operate elementwise on vectors: unit 7 may keep 99% of its memory while unit 12 forgets almost everything at the same step. That is how an LSTM can track a slow quantity, such as a reservoir pressure trend, and a fast one, such as a pump start, at the same time.

Common error: applying tanh to the cell value before the update, or forgetting that h uses the new cell value, not the old one. [4 min]

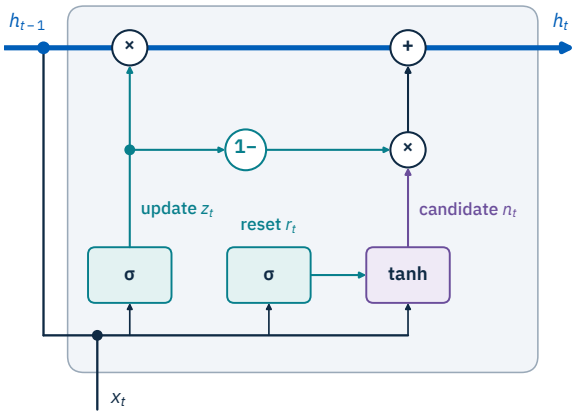
A GRU blends old and new state with one gate

RECURRENT NETWORKS

A GRU blends old and new state with one gate

$$h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}$$

- Update gate *z* blends the candidate with the old state.
- Reset gate *r* scales the old state inside the candidate.
- No separate cell state; three weight blocks, not four.



Schematic of one GRU step in the PyTorch convention.

Neural Networks / Day 5 · Sequence models and attention 43

The gated recurrent unit (Cho and colleagues, 2014) merges the LSTM's cell and hidden states into one. The update gate z decides, per unit, how much of the previous state to keep; the rest comes from the candidate n.

In PyTorch: r = σ(W_ir x + b_ir + W_hr h + b_hr), z likewise, n = tanh(W_in x + b_in + r ⊙ (W_hn h + b_hn)), and h(t) = (1 − z) ⊙ n + z ⊙ h(t−1).

This equation uses PyTorch's update-gate convention. Some texts swap z and 1 − z, so read the equation rather than relying on a gate name. Reset-gate placement also varies between formulations.

A GRU has three weight blocks instead of four, so about 25% fewer parameters than an LSTM of the same width. Do not promise that fewer parameters always give faster or better training; compare them on your own data. [3 min]

RNN, LSTM and GRU trade simplicity for memory

RECURRENT NETWORKS

RNN, LSTM and GRU trade simplicity for memory

Model	State	Parameters (14 inputs, 64 units)	Main trade-off
Vanilla RNN	Hidden state	5,120	Simple; long-range training is hard
LSTM	Hidden and cell states	20,480	More gates and parameters
GRU	One gated hidden state	15,360	A simpler gated alternative
1D CNN or TCN	Local feature maps	Depends on kernels and layers	Parallel over time

Compare them with the same split, target and tuning budget.

Neural Networks / Day 5 · Sequence models and attention 44

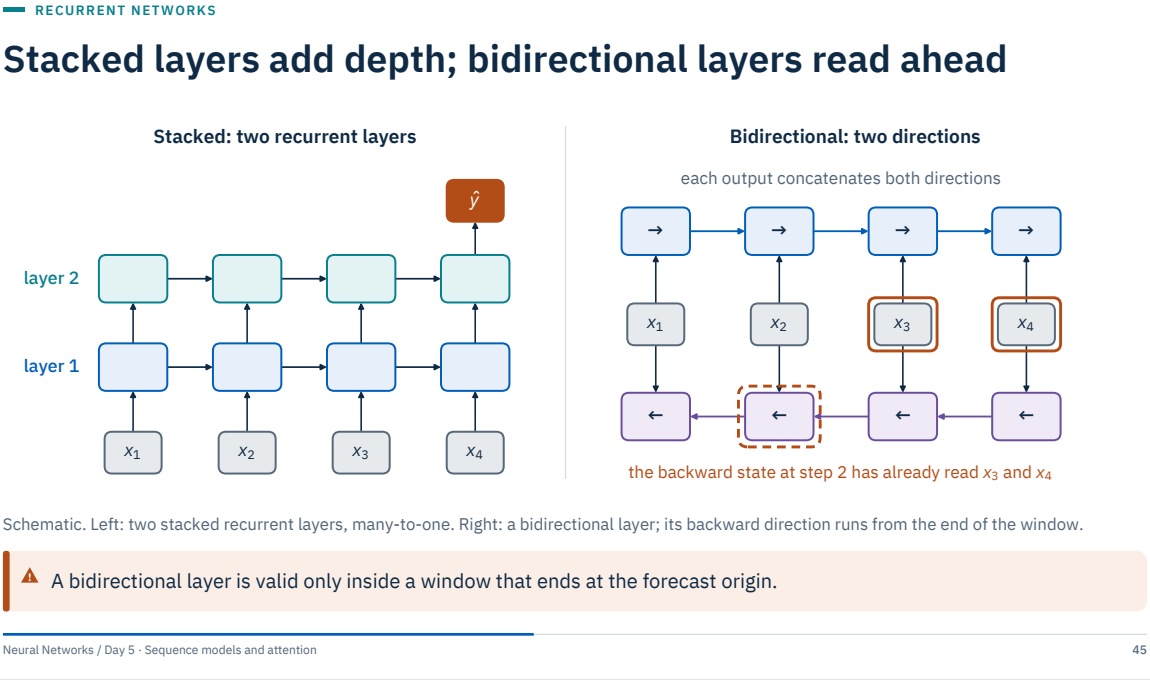
The parameter counts are computed with PyTorch for one layer: the RNN has $H(F + H + 2) = 64 \times 80 = 5{,}120$, the LSTM four times that and the GRU three times.

Use controlled experiments with the same split and forecast target. The ranking between LSTM and GRU varies between data sets; there is no universal winner.

Sequential recurrence limits training parallelism across time: step t cannot start before step t – 1 finishes. A dilated or deep one-dimensional CNN, which we meet in Part 4, can have a broad receptive field without explicit recurrent state and processes all positions in parallel.

Ask: if the GRU and LSTM give the same validation error, which would you deploy? The smaller and faster one, unless you have evidence, such as error on long-memory episodes, that favours the other. [2 min]

Stacked layers add depth; bidirectional layers read ahead



Stacking: nn.LSTM(..., num_layers=2) feeds the output sequence of layer 1 into layer 2. Dropout between layers is set with the dropout argument. The lab model uses two layers of 128 units.

Bidirectional: nn.LSTM(..., bidirectional=True) runs a second LSTM from the end of the sequence to the start and concatenates both outputs, so the output size is 2H. At step 2 the backward state has already read steps 3 and 4.

That is fine when the whole window is known when the prediction is made, for example classifying a completed drilling stand or encoding the history before a forecast origin. It is leakage when the window extends past the origin, or in streaming use where step t must be scored before step t + 1 exists.

Shapes: with bidirectional=True and two layers, h_n has shape (4, batch, H), ordered by layer and direction. [3 min]

A many-to-one LSTM shape check

RECURRENT NETWORKS

A many-to-one LSTM shape check

```
import torch
from torch import nn

rnn = nn.LSTM(input_size=6, hidden_size=32, batch_first=True)
head = nn.Linear(32, 1)
x = torch.zeros(8, 24, 6)                 # (batch, time, features)
sequence, (hidden, cell) = rnn(x)
prediction = head(sequence[:, -1, :])
print(sequence.shape, hidden.shape, prediction.shape)
```

OUTPUT

```
torch.Size([8, 24, 32]) torch.Size([1, 8, 32]) torch.Size([8, 1])
```

For one layer and equal lengths, `sequence[:, -1]` equals `hidden[0]`.

Neural Networks / Day 5 · Sequence models and attention 46

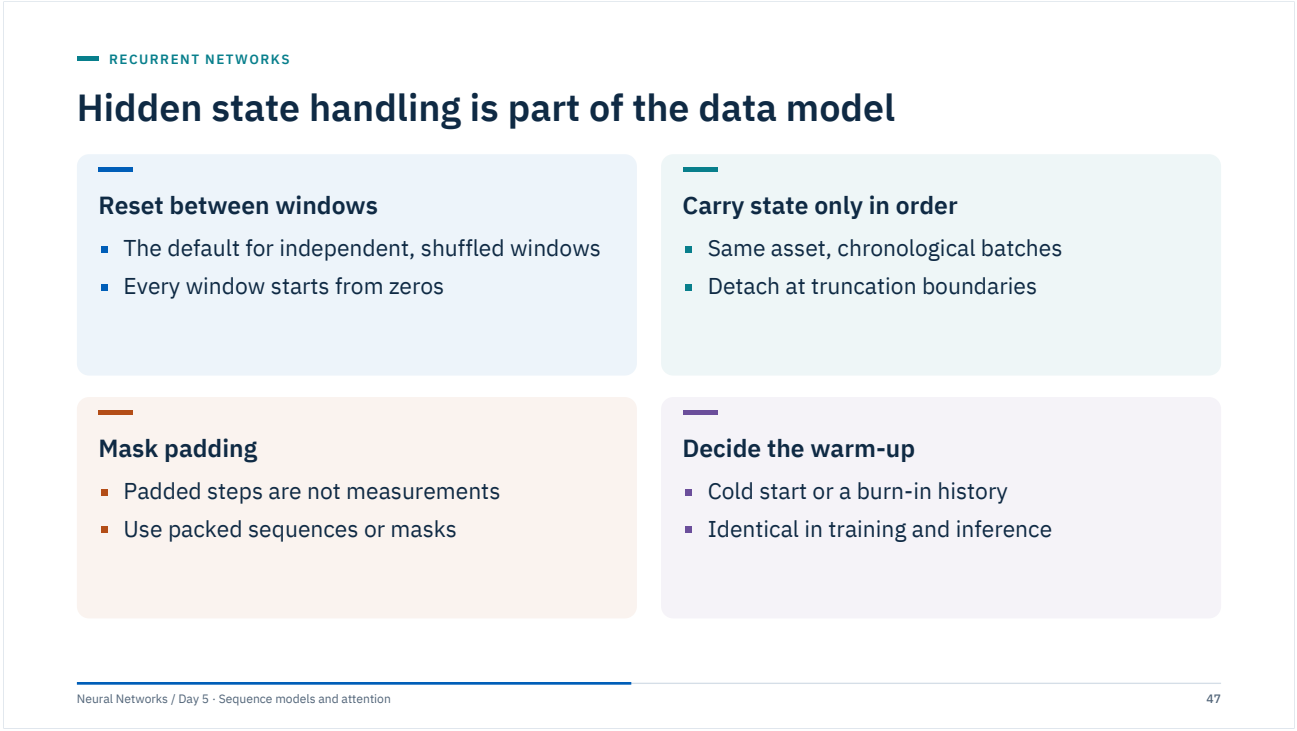
Batch-first input is (B, T, F): 8 windows of 24 steps with 6 features. The output sequence has the hidden state at every step, (8, 24, 32). The final hidden and cell states have shape (num_layers × directions, batch, hidden), here (1, 8, 32). Note that h_n is not batch-first even when batch_first=True, a frequent source of shape bugs.

This equal-length example uses the final output state. We checked that sequence[:, -1] and hidden[0] are identical here; with several layers, sequence holds the top layer only and matches hidden[-1].

Register rnn and head inside one nn.Module, as the notebooks do, or pass both parameter sets to the optimizer explicitly; otherwise the head is never trained.

For forecasting, a bidirectional encoder is valid only over history actually available at the origin. [3 min]

Hidden state handling is part of the data model



A shuffled DataLoader is compatible with independent windows and reset state, which is what all the course notebooks do. It is not compatible with carrying state from one batch to the next: the next batch is a random window from another time or asset.

Stateful training, carrying the state forward, requires batches in chronological order per asset and a detach at every truncation boundary.

Variable-length sequences need masking or torch.nn.utils.rnn.pack_padded_sequence so that padding zeros are not read as measurements. A zero pressure is a real value in many systems.

Define whether inference starts cold, from zeros, or after a warm-up on recent history, and do the same in training. A model trained on warm states and deployed cold will look worse in operation than in validation. [2 min]

Check the lab model before training it

PRACTICE

5 min Pairs

Check the lab model before training it

The climate notebook uses `nn.LSTM(14, 128, num_layers=2, batch_first=True)` and a head 128 → 64 → 1.

1. What are the shapes of the LSTM output and of `h_n` for 256 windows of 72 hours?
2. How many parameters does the first LSTM layer have?
3. Which tensor feeds the head, and why not `h_n[0]`?

Answers, verified in PyTorch.

1. Output (256, 72, 128): top-layer hidden state at every step. h_n (2, 256, 128): the final state of each of the two layers, not batch-first.

2. Layer 1: 4 × 128 × (14 + 128) weights = 72,704, plus 2 × 4 × 128 = 1,024 biases, total 73,728. Layer 2 has 4 × 128 × 256 + 1,024 = 132,096. The whole LSTM has 205,824 parameters; the head adds 8,256 + 65 = 8,321, so the model has 214,145, the number the notebook prints.

3. The notebook uses lstm_out[:, -1, :], the top layer at the last step, which equals h_n[-1]. h_n[0] is the bottom layer's final state, a lower-level summary that the second layer was meant to refine.

Ask: roughly how many training windows per parameter? 48,981 overlapping windows for 214,145 parameters, so early stopping and a validation curve are essential. [5 min]

Lab 1A: a warm-up forecast and two baselines

LABORATORY

60 min

Lab 1A: a warm-up forecast and two baselines

- ☐ Shampoo: build 6-month windows, train the LSTM, compare with the previous month.
- ☐ Climate: load Jena, subsample hourly, split 70/20/10 in time.
- ☐ Fit the scaler on training rows; build 72 h → +12 h windows per split.
- ☐ Score persistence and the 24-hour seasonal naive on validation, in °C.
- ☐ Start the LSTM training before lunch if time allows.

NOTEBOOK

`exercises/01_lstm_shampoo/LSTM_Shampoo_Student.ipynb`

EVIDENCE TO SUBMIT

- Shampoo RMSE next to the naive RMSE
- Window counts for each split
- Two baseline MAEs on validation

Neural Networks / Day 5 · Sequence models and attention 49

Timing: 20 minutes for shampoo, 30 for climate preparation and baselines, 10 to start training. The climate notebook is exercises/02_lstm_climate/LSTM_Climate_Student.ipynb.

Shampoo facts: 36 monthly values, 30 windows of 6 months, 24 for training and 6 for testing. The previous-month forecast has RMSE 173.6 sales units on the 6 test months. Six targets are far too few to rank models; treat the result as a mechanics check, not evidence.

Climate facts from the course file: 420,551 ten-minute rows (2009–2016) become 70,092 hourly rows; the splits give 48,981, 13,935 and 6,927 windows. The notebook first replaces a −9999 logger error code in the two wind-speed columns (four hourly rows, all in the validation period) by the previous valid reading.

Baselines on the scaled windows: persistence is X[:, -1, target_idx], the temperature at the origin. The 24-hour seasonal naive is the temperature at the same clock hour one day before the target, which sits inside the window at index 72 + 12 − 1 − 24 = 59. Invert the scaling with the target column's mean and standard deviation before computing errors in °C. Reference levels on the 13,935 validation windows: persistence MAE 4.21 °C, seasonal naive 2.65 °C.

Common errors: using index 47, which is 24 hours before the origin rather than before the target; fitting the scaler on all rows; comparing baselines and model on different windows. [60 min]

Lab 1B: the climate LSTM against both baselines

LABORATORY

45 min

Lab 1B: the climate LSTM against both baselines

- ☐ Train the multi-feature LSTM with early stopping; plot both loss curves.
- ☐ Score the test windows in °C: LSTM, persistence and seasonal naive.
- ☐ Use identical target timestamps for all three methods.
- ☐ Plot one test week and name one failure period with a hypothesis.
- ☐ Optional: compare a temperature-only model on validation.

NOTEBOOK

`exercises/02_lstm_climate/LSTM_Climate_Student.ipynb`

EVIDENCE TO SUBMIT

- A score table with both baselines first
- Loss curves with the chosen epoch marked
- One failure period and a hypothesis

Neural Networks / Day 5 · Sequence models and attention 50

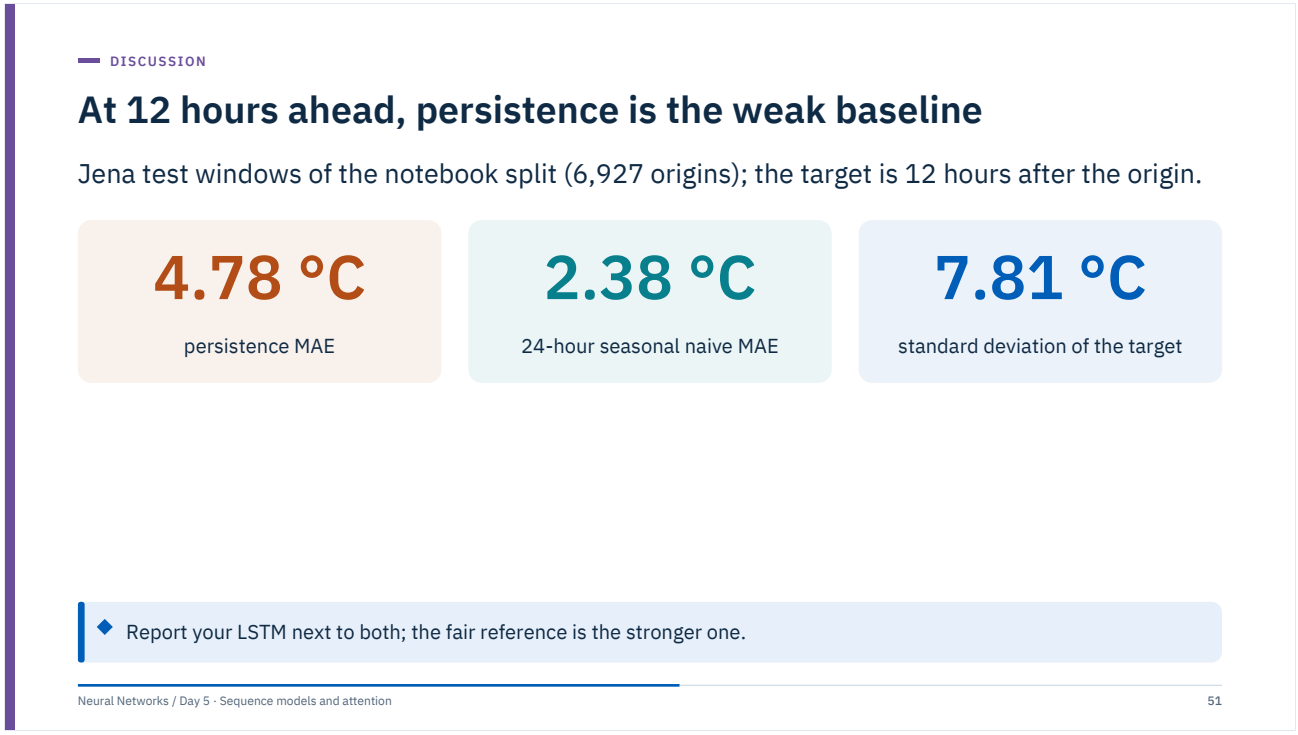
Timing: 30 minutes of work, then the debrief. To fit the slot, each epoch trains on a fresh random 12,288 of the 48,981 training windows, and training stops after at most 15 epochs. On an Apple M3 Max laptop CPU the main model trained in about 2.5 minutes and the optional temperature-only model in about 1.5; older laptops may need twice as long. If laptops are slow, cap training at 10 epochs or skip the optional model; record what you changed, because it changes the evidence.

Early stopping uses the validation loss with patience 5 and restores a deep copy of the best weights. The temperature-only comparison is made on the validation windows. The test windows are used once, in the last section of the notebook; they must not influence the stopping epoch, the choice of inputs or any hyperparameter.

Reference levels computed from the course file with the notebook's split: on the 6,927 test windows persistence has MAE 4.78 °C and RMSE 6.02 °C; the 24-hour seasonal naive has MAE 2.38 °C and RMSE 3.10 °C. Do not promise learners any LSTM number; results vary with hardware, seeds and the subset used.

For the failure period, useful hypotheses include frontal passages with rapid temperature change, snow cover or fog, and sensor gaps. Ask for the evidence that would test the hypothesis, not just the story. [45 min]

At 12 hours ahead, persistence is the weak baseline



Why is persistence so weak here? Twelve hours ahead is the opposite phase of the daily temperature cycle: the value at 06:00 is a poor guess for 18:00. The seasonal naive uses the same clock hour yesterday and halves the error. This mirrors the negative autocorrelation at lag 12 we saw for load.

Collect the groups' LSTM test MAEs on the board next to these two numbers. Discuss: did the multi-feature model beat 2.38 °C? By how much, and is the difference larger than the spread between groups with different seeds? Did anyone's model beat persistence but not the seasonal naive? That model has not yet shown useful skill.

Break the error down by month if time permits: winter errors are usually larger. Compare multi-feature and temperature-only runs.

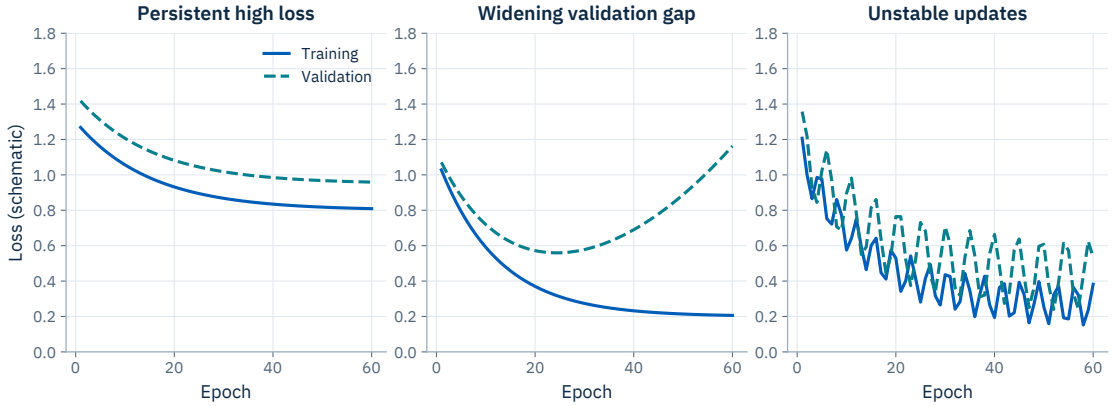
The standard deviation of 7.81 °C gives scale: a model that always predicted the test mean would have an RMSE of about that size. [6 min]

Your loss curves suggest the next experiment

DISCUSSION

Your loss curves suggest the next experiment

Which shape is closest to your climate LSTM, and what would you change first?



Schematic learning curves (recap of Day 2), not measured training runs; each shape admits several diagnoses.

Ask two or three groups to hold up their loss curves and match them to a panel. [4 min]

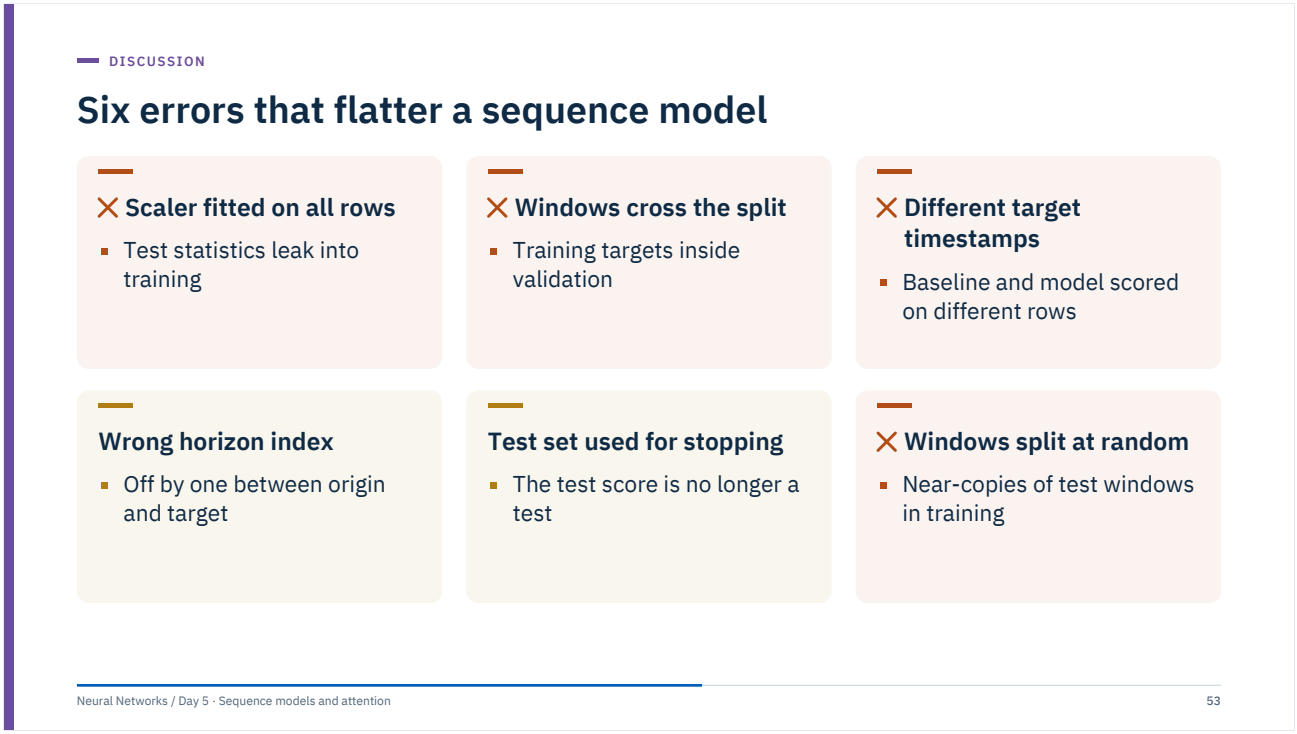
Persistent high loss, both curves flat and high: underfitting or an input problem. Check the scaling, the target index and the learning rate before adding capacity; a model that cannot beat persistence on the training windows has a data or code problem.

Widening validation gap: the model memorizes the training period. With 48,981 overlapping windows this can happen within a few epochs. Early stopping with restored best weights is already in the notebook; add dropout or weight decay, or reduce the hidden size.

Unstable updates: the learning rate is too high for this batch size, or gradients occasionally explode. Lower the learning rate, or clip gradient norms as discussed for recurrent networks this morning.

Remind the room that a validation curve measures only the validation period. A good curve says nothing about the baselines; that comparison is on the previous slide.

Six errors that flatter a sequence model



Ask each group to check their notebook against these six errors. Most groups find at least one.

Scaler on all rows: the notebook fits on training rows only; check that no later cell refits. Windows crossing the split: building windows before splitting creates training windows with validation targets. Different target timestamps: the baseline arrays must be cut from the same windows as the model's targets; lengths that differ by 12 or 72 are the tell-tale sign. Wrong horizon index: print the timestamp of the last input and of the target for one window. Test set used for stopping: early stopping and any model selection use validation only.

Random splits of windows: with stride 1, a test window and its training neighbour share all but one input step, so a random split measures memorization of near-duplicates. The notebooks split in time; keep it that way when you modify them.

Each error makes the model look better than it will be in operation. None of them shows up as a code error; they are found only by reading the evaluation design. [5 min]

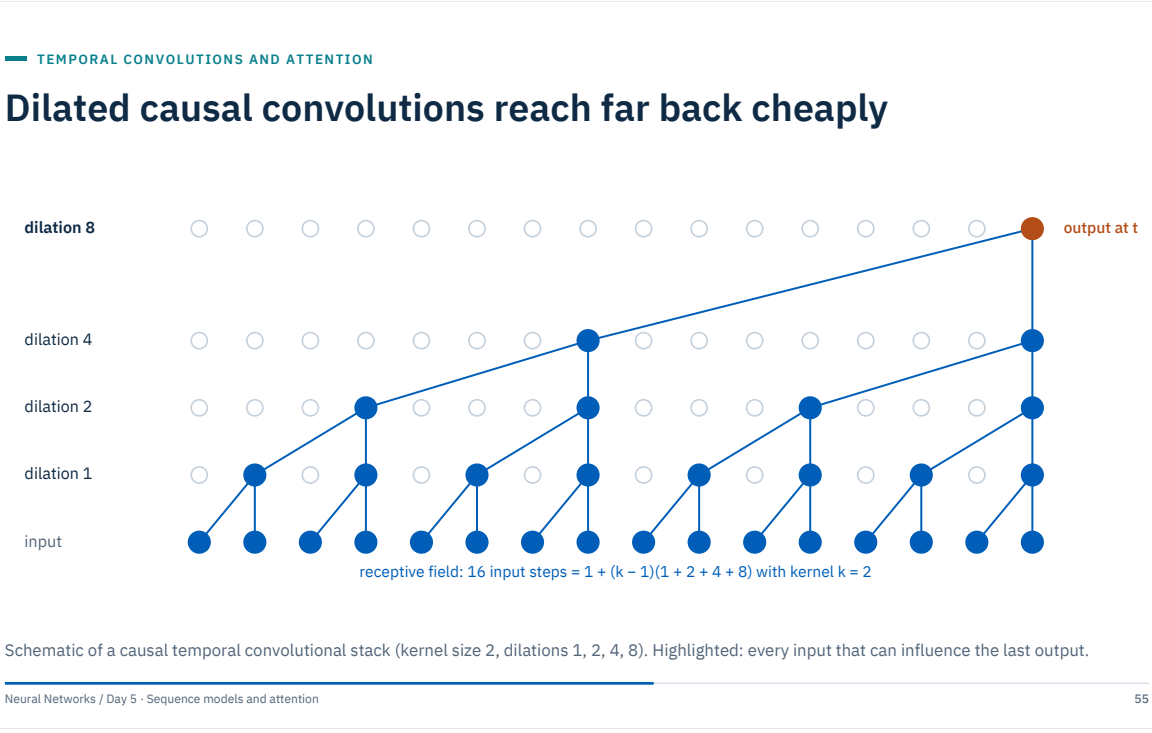
Temporal convolutions and attention



After lunch and the lab, restart with a question: in the LSTM, how many sequential steps separate the first input of a 168-hour window from the output? 167. Both ideas in this part shorten that path.

Temporal convolutions reach far back with a logarithmic number of layers. Attention connects every position to every other position in a single step, at a cost we will quantify in Part 5. [1 min]

Dilated causal convolutions reach far back cheaply



A temporal convolutional network (TCN) stacks one-dimensional convolutions, from Day 4, with two constraints. Causal: the output at time t uses only inputs at t and before, implemented by padding on the left only. Dilated: layer l skips d(l) − 1 inputs between kernel taps, with dilations doubling from layer to layer.

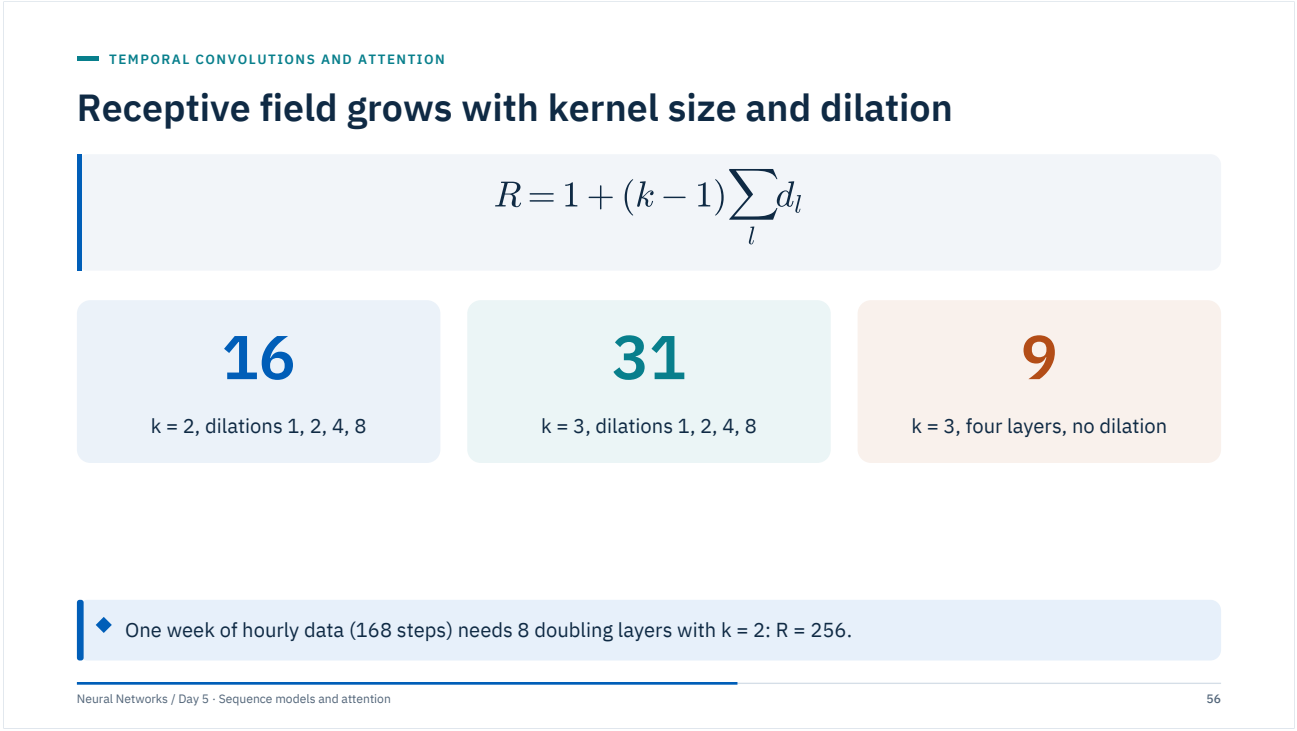
Follow the blue tree down from the orange output: four layers with kernel size 2 cover 16 input steps. Without dilation, four layers would cover only 5.

Advantages: all time steps are computed in parallel during training, gradients pass through a fixed number of layers rather than one step per time step, and the receptive field is explicit. Disadvantage: the model cannot see beyond its receptive field, while an RNN can in principle carry state indefinitely.

References: dilated causal convolutions were popularized by WaveNet (van den Oord and colleagues, 2016); Bai, Kolter and Koltun (2018) compared TCNs with recurrent networks on sequence benchmarks. [3 min]

Reference: https://arxiv.org/abs/1803.01271

Receptive field grows with kernel size and dilation



The receptive field R is how many input steps can influence one output. Each layer with kernel size k and dilation d adds (k − 1) × d steps.

Check the tiles: k = 2 with dilations 1, 2, 4, 8 gives 1 + 1 × 15 = 16. k = 3 with the same dilations gives 1 + 2 × 15 = 31. k = 3 with four undilated layers gives 1 + 2 × 4 = 9.

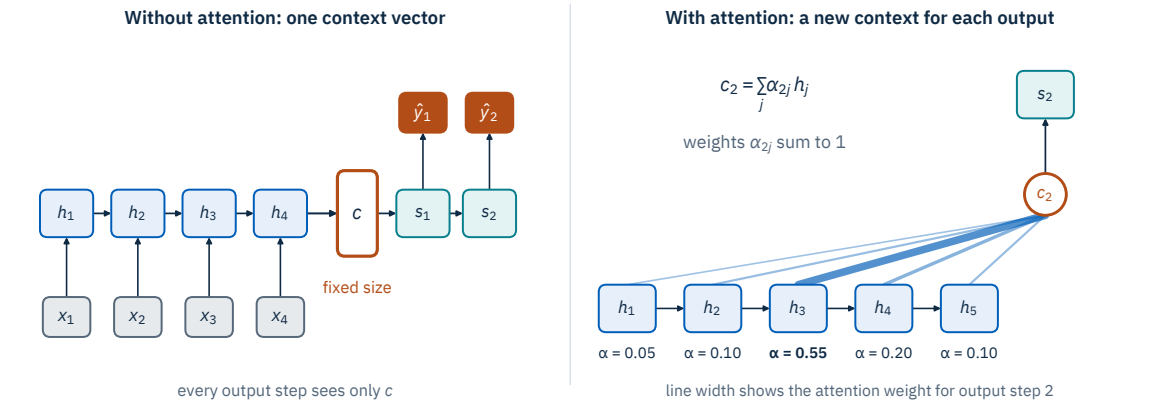
For a weekly context of 168 hourly steps with k = 2 and doubling dilations: seven layers give 1 + 127 = 128, not enough; eight layers give 1 + 255 = 256.

In PyTorch, one causal layer can be written as nn.Conv1d(channels, channels, kernel_size=k, dilation=d) applied after padding the input with (k − 1) × d zeros on the left, for example with torch.nn.functional.pad(x, ((k - 1) * d, 0)). Note that Conv1d expects (batch, channels, time), unlike the batch-first LSTM layout. [2 min]

One context vector is a bottleneck

TEMPORAL CONVOLUTIONS AND ATTENTION

One context vector is a bottleneck



Schematic. Left: an encoder–decoder that passes one fixed-size vector. Right: attention forms a weighted context for each output step; weights are illustrative.

Neural Networks / Day 5 · Sequence models and attention 57

The first sequence-to-sequence models (Sutskever and colleagues, 2014) encoded the entire input into the encoder's final state c and decoded every output from it. Whatever the length of the history, the decoder sees one fixed-size vector: a bottleneck, especially for long inputs.

Bahdanau, Cho and Bengio (2014) let each decoder step look back at all encoder states and form its own context as a weighted sum, with weights that sum to one. In the illustration, output step 2 puts 55% of its weight on h_3.

This is the idea behind all attention mechanisms: instead of forcing all information through one recurrent state, build a context on demand. Do not claim that attention weights are automatically explanations of model reasoning or causal feature importance; we return to this on the next slide. [3 min]

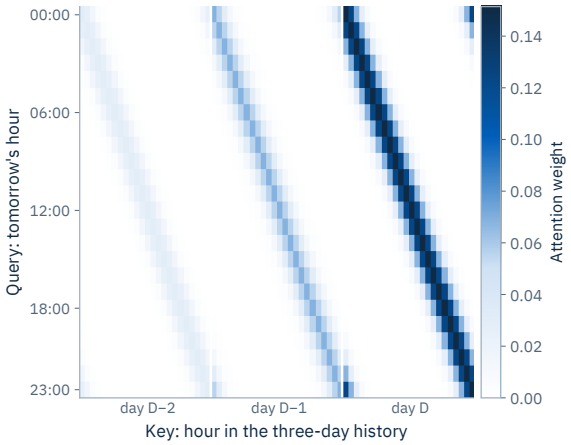
Reference: https://arxiv.org/abs/1409.0473

Attention is a learned soft alignment

TEMPORAL CONVOLUTIONS AND ATTENTION

Attention is a learned soft alignment

- Each row is a probability distribution over the keys.
- Each hour of tomorrow looks at the same clock hour.
- Recent days weigh more: day D gets 61% at 00:00.



◆ Attention weights show where information flowed, not why the output is right.

Hand-built queries and keys (clock-hour encodings, recency penalty); illustrates the mechanism, not a trained model.

Neural Networks / Day 5 · Sequence models and attention 58

This toy example uses the day-ahead setting. Each query is one hour of tomorrow; each key is one hour of the last three days. The similarity score is high when the clock hours match, and slightly lower for older days. After a softmax over the 72 keys, every row sums to one.

The result is a soft alignment: tomorrow's 18:00 looks mostly at 18:00 today, less at 18:00 yesterday, and a little at the neighbouring hours. A trained model learns scores like these from data, if they help reduce the loss.

Why "soft"? A hard alignment would pick one key; soft weights are differentiable, so the whole mechanism trains by backpropagation.

Caution: weights show where information was drawn from, not a causal explanation, and different weight patterns can give the same prediction (Jain and Wallace, 2019, "Attention is not explanation"). [3 min]

Reference: https://arxiv.org/abs/1902.10186

Queries, keys and values have distinct roles

TEMPORAL CONVOLUTIONS AND ATTENTION

Queries, keys and values have distinct roles

Quantity	Role	In self-attention
Query Q	What each position seeks	A projection of the input states
Key K	What each position offers for matching	Another projection of the same states
Value V	Information to aggregate	A third projection of the same states

Cross-attention takes queries from one sequence and keys and values from another.

◆ Each output is a sum of values, weighted by how well its query matches each key.

Neural Networks / Day 5 · Sequence models and attention 59

Use a lookup analogy cautiously: keys and queries are learned vectors, not literal database keys. The output for each query is a weighted sum of value vectors, with weights determined by query–key similarities.

In self-attention all three come from the same sequence through three learned matrices: Q = X W_Q, K = X W_K, V = X W_V. In cross-attention, for example a decoder reading an encoder, the queries come from the decoder and the keys and values from the encoder. The soft-alignment example on the previous slide was cross-attention between tomorrow's hours and the history.

Ask: why separate keys from values? Because what makes a position relevant, for example "this is 18:00 on a weekday", need not be the information you want to copy from it, for example the load value and its recent slope. [2 min]

Scaled dot-product attention in one line

TEMPORAL CONVOLUTIONS AND ATTENTION

Scaled dot-product attention in one line

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

- Divide by $\sqrt{d_k}$ to keep scores in a useful range.
- Softmax runs over the keys, separately for each query.
- The mask M adds $-\infty$ where access is not allowed.
- Each output is a weighted average of value vectors.

◆ PyTorch: `F.scaled_dot_product_attention(q, k, v, is_causal=True)`.

Neural Networks / Day 5 · Sequence models and attention 60

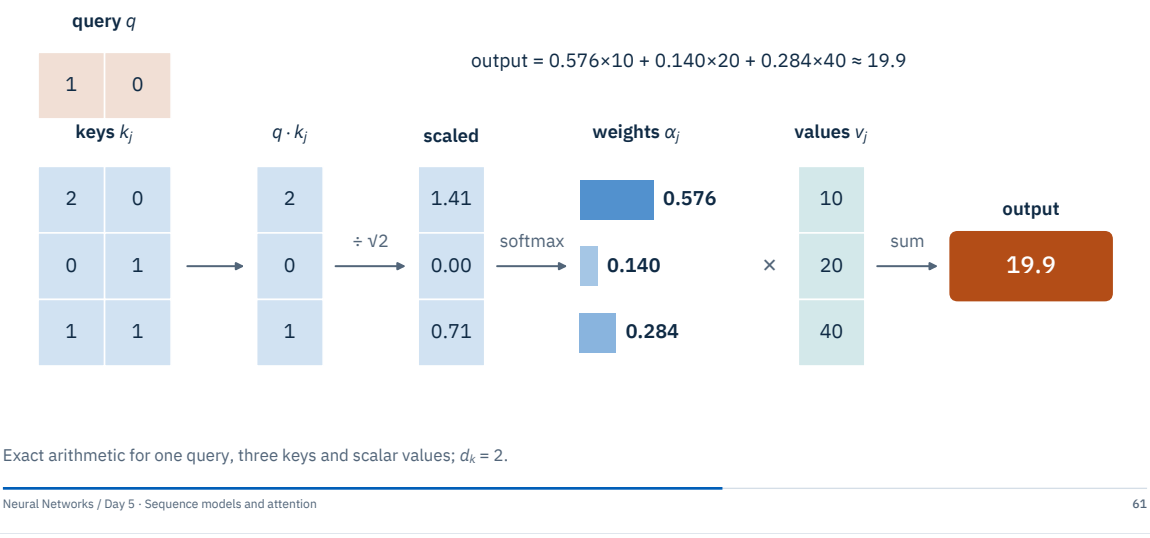
Shapes for one head: Q is T_query × d_k, K is T_key × d_k and V is T_key × d_v. The score matrix QK^T is T_query × T_key, one row per query. The softmax is taken along each row. Multiplying by V gives T_query × d_v.

A causal additive mask uses zero for allowed positions and minus infinity for future positions; after the softmax those positions receive exactly zero weight.

In PyTorch, torch.nn.functional.scaled_dot_product_attention implements the formula and accepts is_causal=True. Mask conventions differ between APIs: for that function a boolean mask marks positions that may be attended to, whereas nn.MultiheadAttention treats True as "not allowed". Verify any mask with a tiny example before trusting it.

Next slide: the whole calculation with numbers. [3 min]

Attention for one query, step by step



Walk left to right. The query is [1, 0]. Dot products with the three keys give scores 2, 0 and 1. Dividing by √2 gives 1.41, 0.00 and 0.71. The softmax turns these into weights 0.576, 0.140 and 0.284, which sum to one. The output is 0.576 × 10 + 0.140 × 20 + 0.284 × 40 = 19.92, about 19.9. If learners round the weights to two decimals first they get 19.8: a useful reminder to round only at the end.

Points to make. The first key is most similar to the query, so its value dominates. The output is a convex combination of the values, so it must lie between 10 and 40. If all scores were equal, the output would be the plain mean, 23.3.

Ask: what happens to the weights if the query is doubled to [2, 0]? Scores double to 4, 0 and 2, the softmax becomes sharper and the output moves towards 10. The scale of the scores controls how peaked the attention is, which is why the √d_k scaling matters. [2 min]

One attention calculation

PRACTICE

4 min · Individual

One attention calculation

A query has scaled scores [0, log 2] for two keys. The values are [10, 20].

1. Calculate the softmax weights.
2. Calculate the attention output.
3. What changes if the second key is masked out?

Answers.

1. Exponentiated scores are $e^0 = 1$ and $e^{\log 2} = 2$, so the weights are 1/3 and 2/3.

2. The output is $(1/3) \times 10 + (2/3) \times 20 = 50/3 \approx 16.67$.

3. Masking the second key adds minus infinity to its score, so its exponential is zero. The weights become [1, 0] and the output is 10.

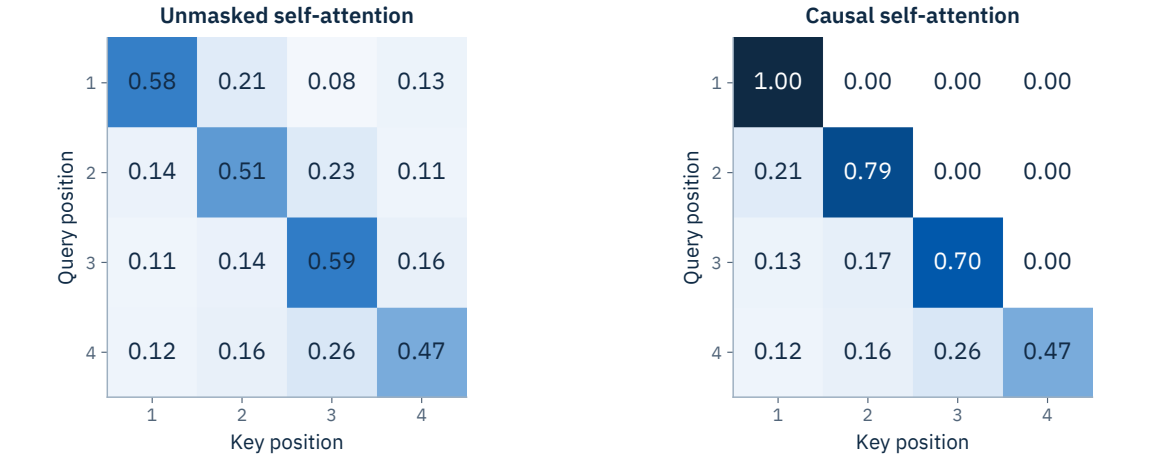
This scalar-value example isolates the operation; in a real model each value is a vector and the same weights combine every component.

Common error: normalizing the scores by their sum instead of applying the exponential first. Scores can be negative, and dividing by their sum gives meaningless weights. [4 min]

A causal mask removes future access

TEMPORAL CONVOLUTIONS AND ATTENTION

A causal mask removes future access



Computed example attention weights with and without an additive causal mask. Every row normalizes over its allowed keys.

Neural Networks / Day 5 · Sequence models and attention 63

Ask which cells must be zero in a causal self-attention matrix. The upper triangle beyond the diagonal: query position 2 may attend to positions 1 and 2 but not 3 and 4.

Compare the rows. Row 4 is unchanged because the last position may see everything. Row 1 must put all its weight on itself. Masking does not simply delete weights; the remaining weights are renormalized, so row 2 changes from 0.51 on itself to 0.79.

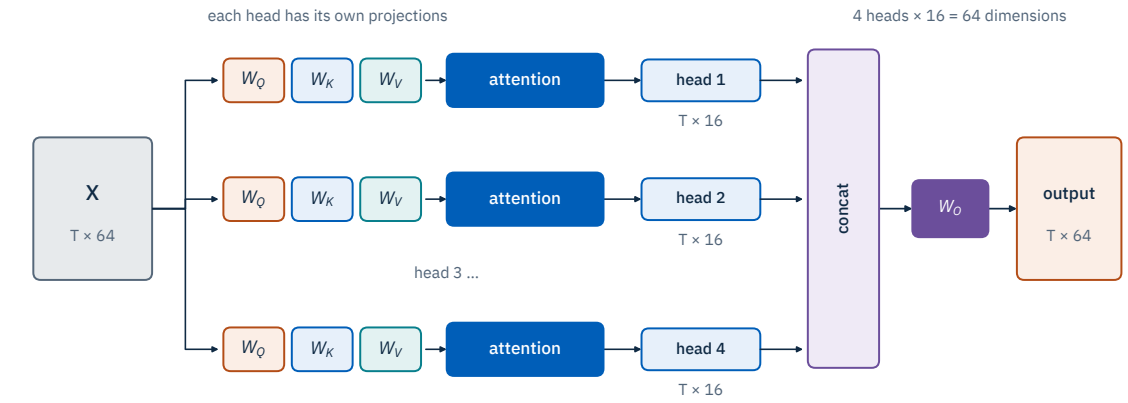
Distinguish causal masks from padding masks. A padding mask hides positions that do not exist, for example the padded end of a short sequence; a causal mask hides positions that exist but lie in the future.

In library APIs boolean mask conventions differ, as noted with the attention formula, so verify them with a tiny example like this one. [2 min]

Multiple heads attend in different subspaces

TEMPORAL CONVOLUTIONS AND ATTENTION

Multiple heads attend in different subspaces



Schematic of multi-head self-attention with model width 64 and 4 heads of width 16.

Neural Networks / Day 5 · Sequence models and attention 64

Each head builds its own projected queries, keys and values, computes attention in its own lower-dimensional subspace, and produces a T × 16 output. The heads run in parallel. Their outputs are concatenated back to width 64 and passed through an output projection W_O.

The model dimension commonly divides evenly across heads: 64 / 4 = 16. In PyTorch: nn.MultiheadAttention(embed_dim=64, num_heads=4, batch_first=True).

Why several heads? One softmax produces one weighting per query; several heads let the model attend, for example, to the same hour yesterday and to the last few hours at the same time.

The number of heads is an architectural choice, not a count of interpretable concepts. More heads are not automatically better. Avoid assigning human-readable meanings to heads without evidence from a specific trained model. [3 min]

Why divide by the square root of dk?

PRACTICE

4 min · Pairs

Why divide by the square root of d_k?

Query and key components are independent, with mean 0 and variance 1; $d_k = 64$.

1. What is the standard deviation of q · k before and after the scaling?
2. Compare softmax([8, 0, −8]) with softmax([1, 0, −1]).
3. Why would near one-hot weights slow down learning?

Answers.

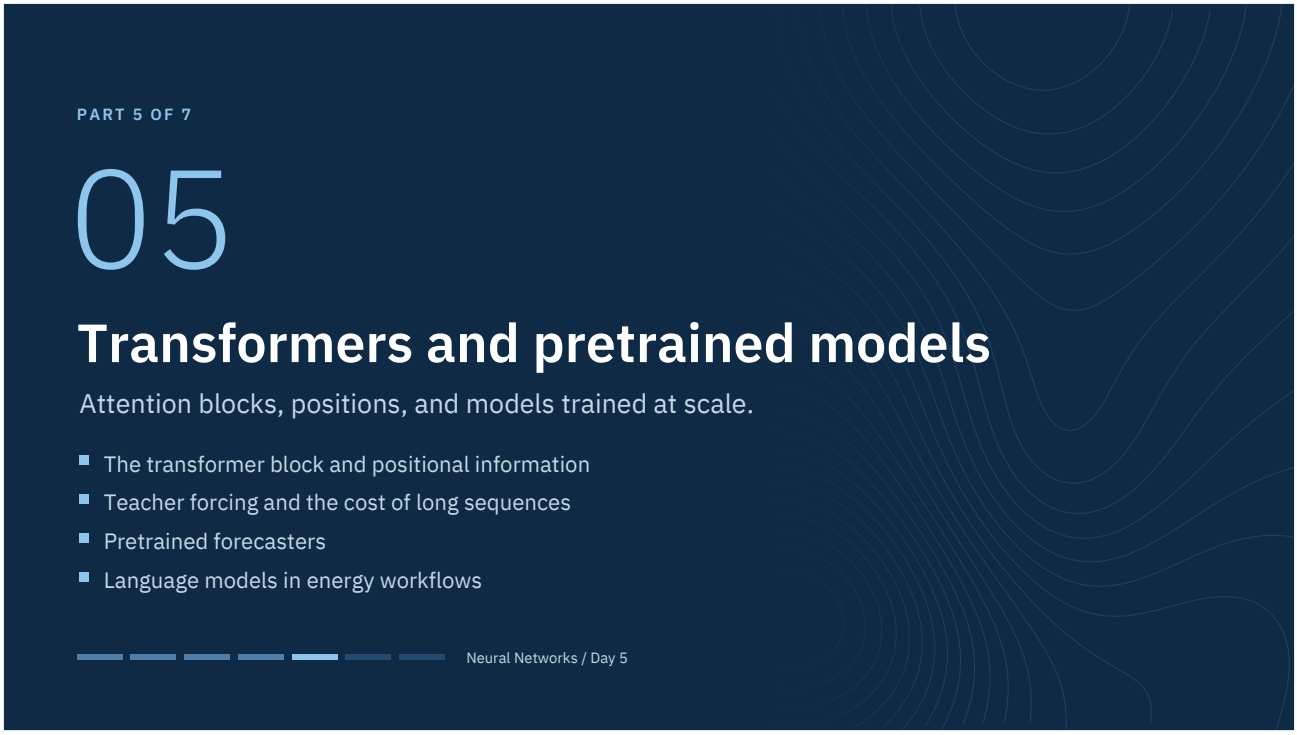
1. q · k is a sum of 64 products, each with mean 0 and variance 1, so its variance is 64 and its standard deviation is 8. After dividing by √64 = 8 the standard deviation is 1, whatever the head width.

2. softmax([8, 0, −8]) = [0.99966, 0.00034, 0.0000001]: essentially one-hot. softmax([1, 0, −1]) = [0.665, 0.245, 0.090]: a genuine weighting.

3. When the softmax saturates, its derivatives for all but the largest score are nearly zero, so gradients barely change which keys are attended; learning stalls, much like a saturated sigmoid on Day 1.

This is the motivation given by Vaswani and colleagues (2017) for the scaling. Some newer models also normalize queries and keys, for the same reason. [4 min]

Transformers and pretrained models



We now assemble attention into the transformer block, see how order enters the model, and quantify the cost of long sequences.

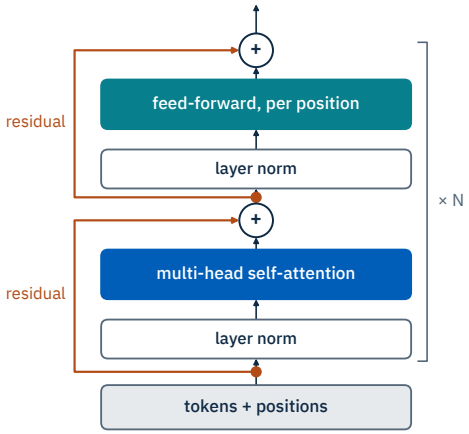
The second half of the part is orientation rather than implementation: pretrained time-series models and large language models are changing fast, so we focus on durable questions, how to evaluate them and when to trust them, rather than on a catalogue of products. No performance claims are made for any named model. [1 min]

A transformer block alternates mixing and processing

TRANSFORMERS AND PRETRAINED MODELS

A transformer block alternates mixing and processing

- Self-attention mixes information between positions.
- The feed-forward network transforms each position alone.
- Residual additions keep a direct update path.
- Layer norm controls scale; its placement varies.



Schematic of a pre-norm encoder block; the 2017 original applied layer norm after each residual addition.

Neural Networks / Day 5 · Sequence models and attention 67

Read the block from bottom to top: normalize, attend, add the input back; normalize, apply a small two-layer network to each position separately, add again. A model stacks N such blocks.

Describe this as a pre-normalization teaching order, the one used by many current implementations, not the exact original 2017 transformer, which placed layer normalization after each residual addition.

The residual paths play the role the cell-state lane played in the LSTM: an additive route along which gradients pass without being multiplied at every block.

An encoder–decoder model adds a cross-attention sublayer in the decoder blocks. Modern variants change activations, position handling, normalization and the attention implementation, but the block structure is recognizable in almost all of them. In PyTorch: nn.TransformerEncoderLayer(d_model=64, nhead=4, batch_first=True, norm_first=True). [3 min]

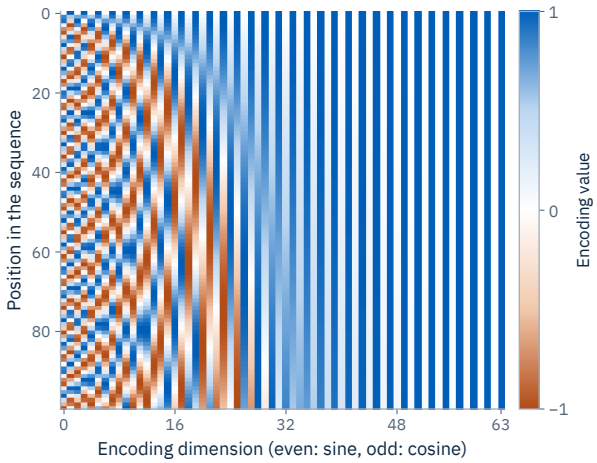
Position must enter the representation

TRANSFORMERS AND PRETRAINED MODELS

Position must enter the representation

$$PE_{p,\,2i} = \sin\left(p \,/\, 10000^{2i/d}\right)$$
$$PE_{p,\,2i+1} = \cos\left(p \,/\, 10000^{2i/d}\right)$$

- Self-attention alone ignores the order of positions.
- Options: sinusoidal, learned, relative, rotary.
- Time series also need elapsed time and calendar.



Exact sinusoidal encoding for model width 64 and positions 0–99.

Neural Networks / Day 5 · Sequence models and attention 68

Plain self-attention is permutation-equivariant: shuffle the input positions and the outputs are shuffled the same way, unless an order signal or an order-specific mask is supplied. A forecasting model that does not know which value came last is useless, so position must be added.

The sinusoidal encoding adds a fixed vector to each position: fast oscillations in the first dimensions, slow ones in the last. Every position gets a unique pattern, and relative offsets correspond to rotations, which attention can exploit.

Alternatives: learned position embeddings, relative position biases, and rotary embeddings (Su and colleagues, 2021).

For time series, the position index is not elapsed time when sampling is irregular, and it carries no calendar meaning. Add hour, weekday and holiday features, or time deltas. Extrapolation beyond the sequence lengths seen in training needs evaluation. [3 min]

Transformer families match different tasks

TRANSFORMERS AND PRETRAINED MODELS

Transformer families match different tasks

Family	Attention pattern	Example use
Encoder	Often bidirectional over available input	Representation or classification
Decoder-only	Causal over generated positions	Next-token prediction
Encoder–decoder	Input encoding plus cross-attention	Translation or sequence generation

All three can forecast if the masks respect the forecast origin.

◆ Many large language models are decoder-only transformers trained to predict the next token.

An encoder reads the whole input with bidirectional attention: fine for classifying a completed window or encoding history up to the origin. A decoder-only model uses causal attention and generates one position at a time. An encoder–decoder encodes the input and generates the output with cross-attention to it.

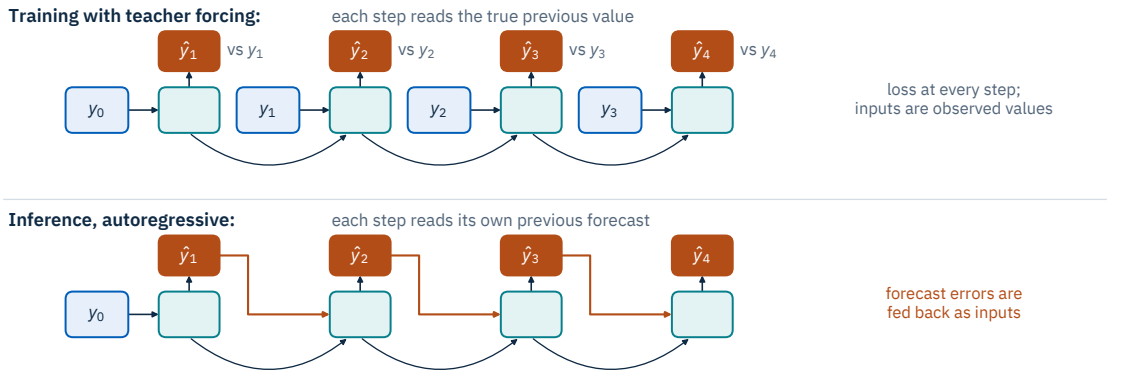
Large language models commonly use decoder-only transformers, but the broader category includes other designs. Model scale, training data, objectives and post-training also matter; avoid defining a large language model solely by its parameter count.

For forecasting, each family can be used; what matters is that no attention path reads beyond the forecast origin, and that the evaluation follows the rolling-origin design from Part 2. [2 min]

Teacher forcing and inference see different histories

TRANSFORMERS AND PRETRAINED MODELS

Teacher forcing and inference see different histories



Schematic of a recurrent decoder in training (teacher forcing) and in inference (autoregressive rollout); y_0 is the last observed value.

Neural Networks / Day 5 · Sequence models and attention 70

During training, the decoder at step t can be given the true previous value y(t−1): teacher forcing (Williams and Zipser, 1989). Training is fast and stable, and the loss is computed at every step.

At inference the true values are unknown, so the decoder reads its own previous forecast. Errors made early become inputs later and may compound, exactly the recursive-strategy problem from Part 2. The model was never trained on its own mistakes.

Make sure teacher-forcing targets are shifted so the model cannot see the target it is currently asked to predict; an unshifted decoder input is one of the most common leakage bugs in sequence-to-sequence code.

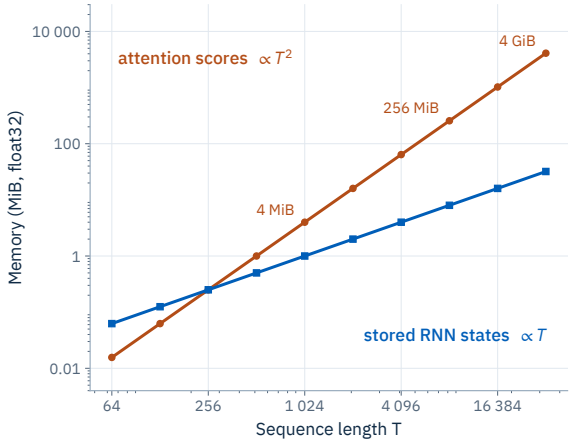
Remedies: evaluate the full rollout required by the application; scheduled sampling, which gradually replaces true inputs by predictions during training (Bengio and colleagues, 2015); or direct multi-horizon heads that avoid feedback. Report the actual forecast horizon. [3 min]

Dense attention cost grows with the square of length

TRANSFORMERS AND PRETRAINED MODELS

Dense attention cost grows with the square of length

- The score matrix has T^2 entries per head.
- One year of hourly data: T = 8,760, about 293 MiB per head.
- Efficient kernels cut memory traffic; compute stays quadratic.
- Measure memory and latency at your real input lengths.



Exact arithmetic for float32 scores, one head and one sequence, against stored RNN states of width 256.

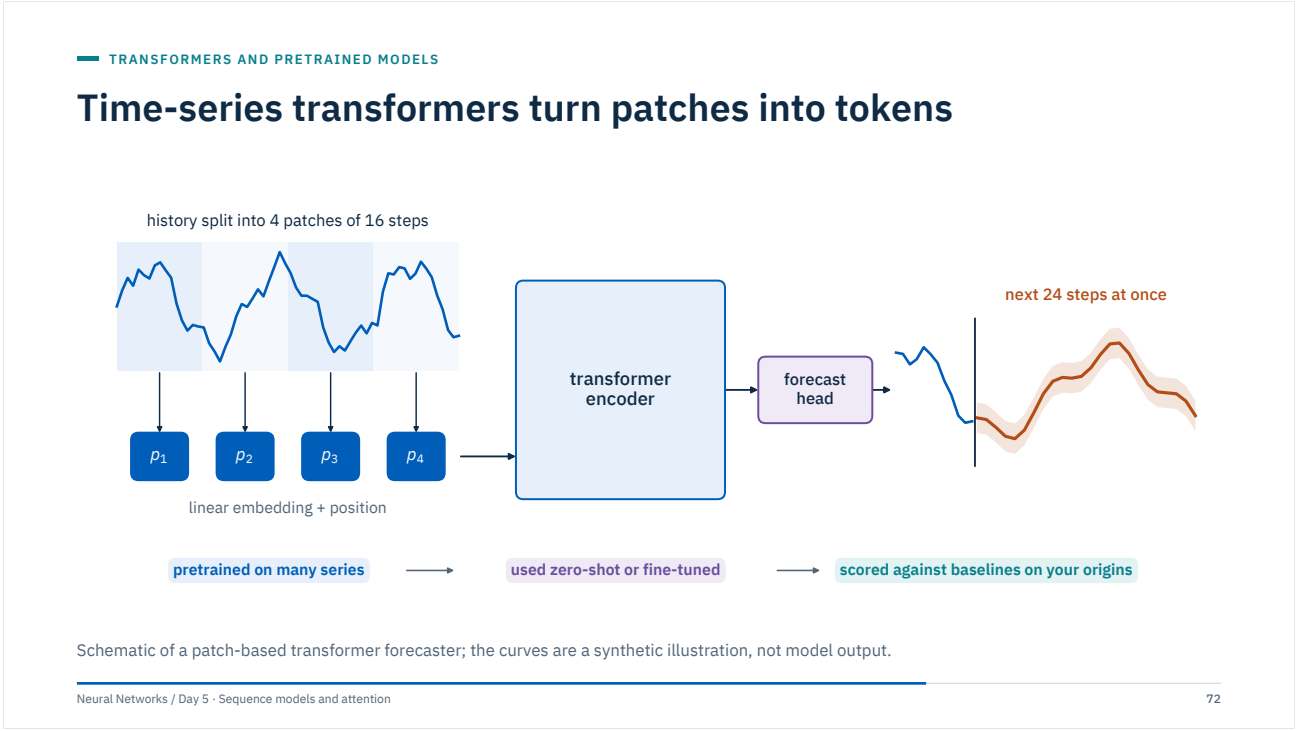
Neural Networks / Day 5 · Sequence models and attention 71

Dense self-attention builds pairwise interactions across positions, so the score matrix grows quadratically with sequence length. The arithmetic: 4 bytes × 8,760² = 307 million bytes, about 293 MiB for one head, one sequence and one layer, before multiplying by heads, batch size and layers. At 1,024 steps it is 4 MiB; at 8,192, 256 MiB.

Do not claim that all modern transformers materialize the full matrix. Fused implementations such as FlashAttention (Dao and colleagues, 2022) compute attention in tiles without storing the full matrix, which cuts memory, but the number of pairwise interactions is still quadratic in the standard dense form. Sparse, linear or state-space alternatives change the assumptions.

Practical options for long sensor histories: downsample, aggregate into patches (next slide), or restrict attention to a local window. Compare real memory and latency under the intended input lengths rather than trusting asymptotics. [3 min]

Time-series transformers turn patches into tokens



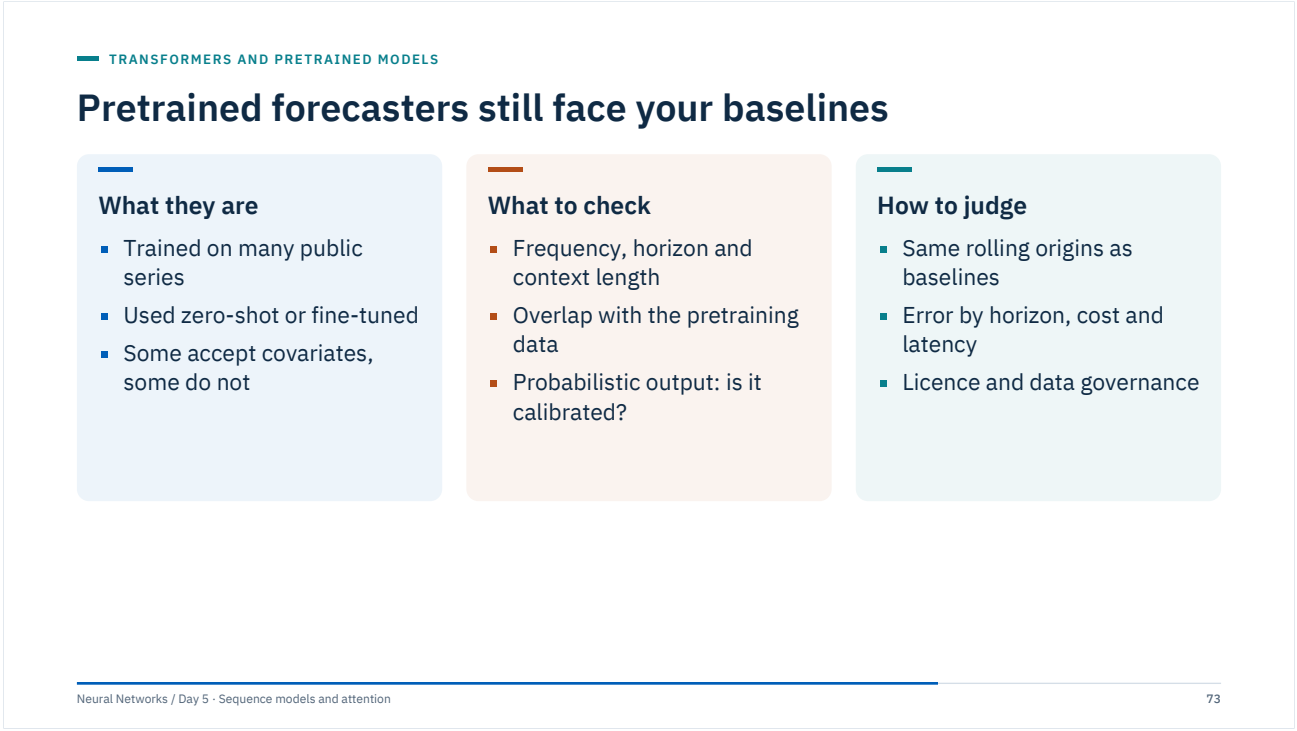
Treating every time step as a token makes sequences long and each token carries little information. Patch-based models cut the history into short segments, here 4 patches of 16 steps, embed each patch as one token, and let the transformer attend between patches. The attention cost falls with the square of the patch length: 64 steps as 4 tokens need 16 attention scores instead of 4,096. PatchTST (Nie and colleagues, 2023) popularized this design for long-horizon forecasting.

The bottom row describes pretrained forecasting models: networks trained on large collections of public series, then applied to a new series without training (zero-shot) or after fine-tuning. Examples in the literature include Chronos, TimesFM and Moirai.

We make no performance claims for any of them. The only evidence that counts is a comparison on your own series, with your origins and horizons, against the baselines from Part 2. [3 min]

Reference: https://arxiv.org/abs/2211.14730

Pretrained forecasters still face your baselines



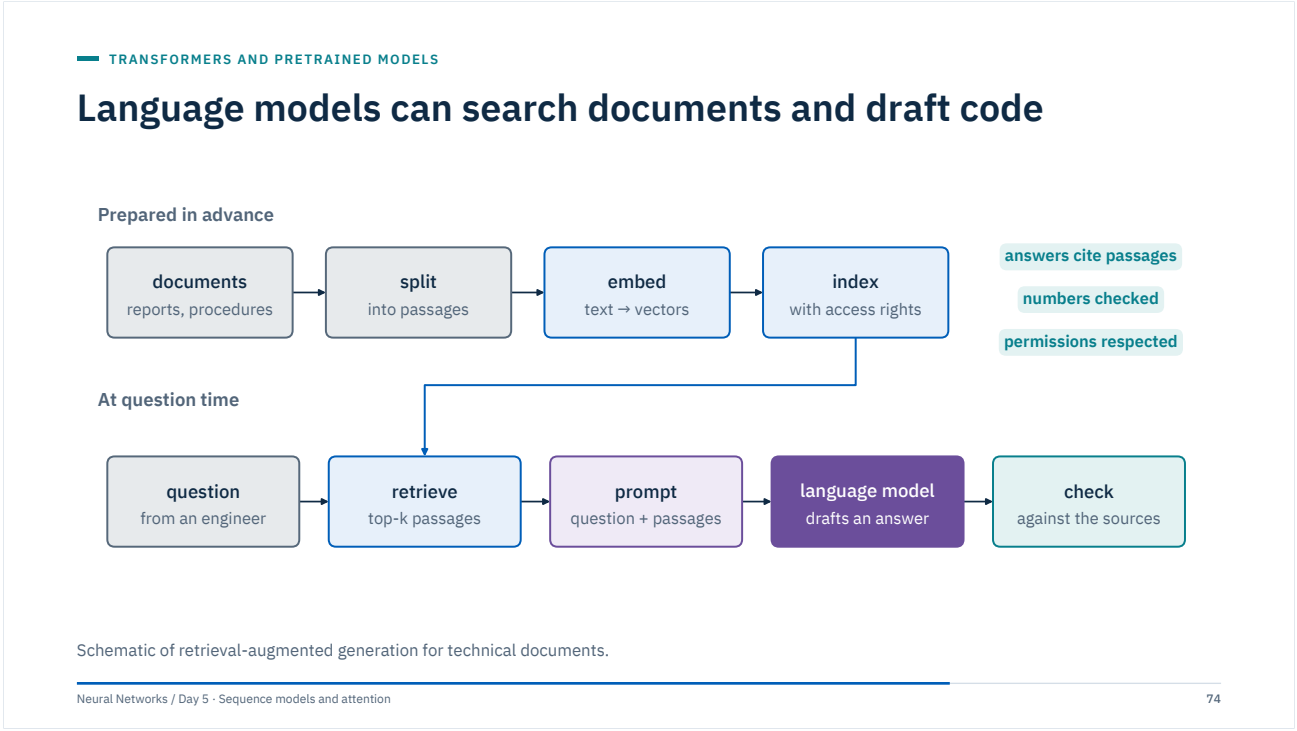
Pretrained forecasters are attractive for many short series, new assets and quick prototypes. Treat them like any other candidate model.

What to check: whether the model supports your sampling frequency, horizon and context length; whether it can use known covariates such as weather forecasts and calendars; and whether its public benchmark results could be inflated because benchmark series overlapped with its pretraining data.

How to judge: run it on the same rolling origins as the seasonal naive and your lag model, report error by horizon, check interval coverage if it produces quantiles, and record inference cost and latency. For hosted models, confirm what data leave your network and under which licence.

Ask: which of your own forecasting problems has many similar series and short histories? That is where a pretrained model is most worth testing. [2 min]

Language models can search documents and draft code



Large language models enter energy workflows mostly through text, not through sensor forecasting. Three uses are common.

Document search and question answering: retrieval-augmented generation (Lewis and colleagues, 2020) splits reports, procedures and standards into passages, embeds them as vectors, retrieves the passages most relevant to a question and gives them to the model, which drafts an answer that cites them.

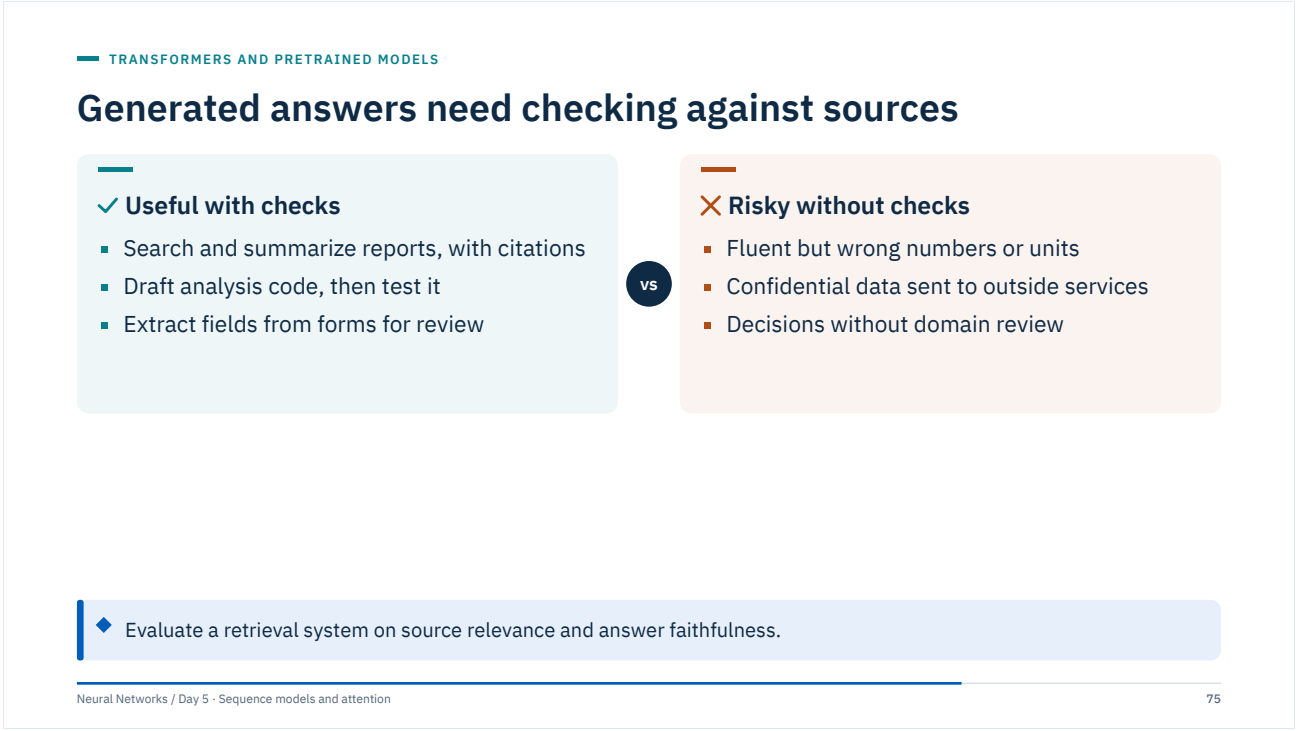
Summarization and extraction: daily drilling reports, maintenance logs and inspection notes into structured fields, reviewed by a person.

Code assistance: drafting data-preparation and plotting code, which must then be tested like any other code.

The retrieval step respects access rights, so users only see passages they are allowed to read. Keep this as orientation, not a catalogue of commercial models: products change monthly, the architecture of the workflow does not. Vision models can similarly represent images or patches for classification and other tasks, and still need task-specific evaluation. [3 min]

Reference: https://arxiv.org/abs/2005.11401

Generated answers need checking against sources



Language models produce fluent text whether or not it is correct. In engineering, the dangerous errors are plausible numbers: a wrong unit conversion, a pressure rating from the wrong standard, a depth from the wrong well.

A retrieval system should be evaluated on two separate questions: did it retrieve the relevant passages, and is the answer faithful to them? Build a small test set of questions with known answers from your own documents before deployment.

Foundation-model adaptation still requires task-specific evaluation, and generated outputs and explanations need verification against source evidence. A generated explanation of an engineering decision must not substitute for a validated model and a domain review.

Data governance: check what leaves your network, who can see prompts and outputs, and how long they are retained. [2 min]

Match the architecture to the constraint

PRACTICE

5 min Pairs

Match the architecture to the constraint

Choose a starting point after the baselines, and name one risk.

1. ESP sensors every 10 minutes; a real-time alarm from the last 2 hours.
2. Hourly load for 300 feeders; one-week context; day-ahead horizon.
3. 5,000 well reports; "which wells had stuck pipe after a mud change?"

Neural Networks / Day 5 · Sequence models and attention 76

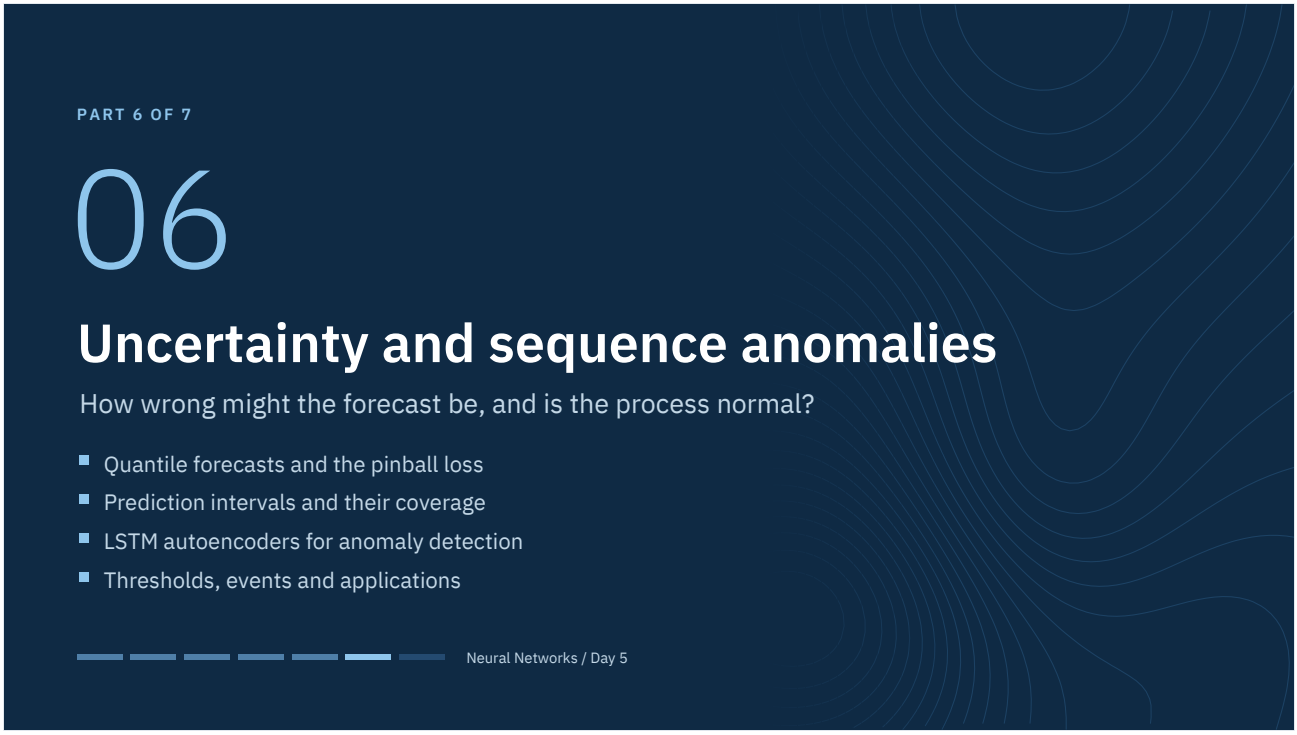
Model answers; accept alternatives with good reasons.

1. Two hours of 10-minute data is only 12 steps. Lag features with gradient boosting, or a small unidirectional GRU or causal TCN, are enough; a GRU can be updated step by step in streaming use. No bidirectional layers. Risk: labels describe the current state rather than a future event, and alarms must be scored as events with lead time.

2. A global model trained across all feeders: lag and calendar features with boosting first, then an LSTM, TCN or transformer with a feeder embedding, or a pretrained forecaster tested zero-shot. Baseline: the weekly seasonal naive per feeder. Risk: holidays and feeders reconfigured during the record.

3. Not a forecasting problem: retrieval over the reports plus a language model that cites passages, with an engineer checking each cited source. Risk: missed reports and invented connections between mud changes and stuck pipe. [5 min]

Uncertainty and sequence anomalies



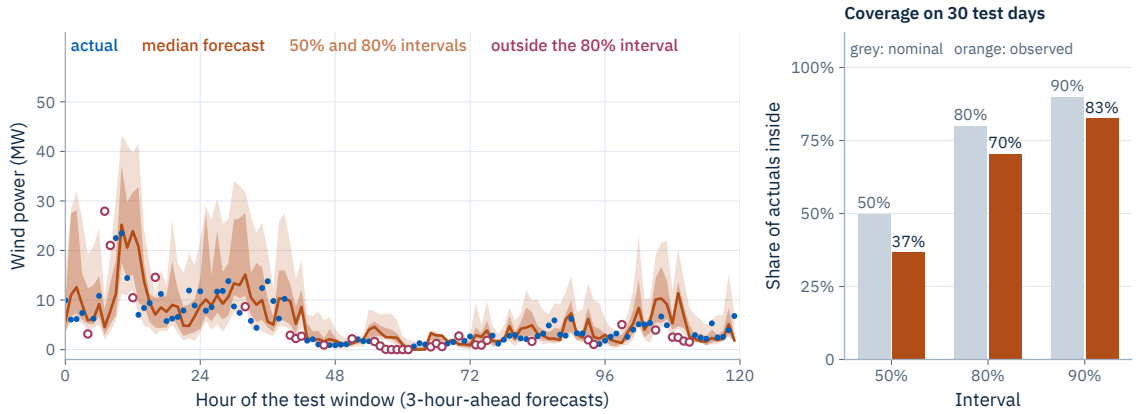
Two practical extensions of everything so far. A point forecast of 30 MW is less useful to a dispatcher than "between 18 and 41 MW with 80% probability". And many sequence problems in operations are not forecasts at all but warnings: is this pump, compressor or process behaving normally?

Both extensions reuse the same evaluation discipline: calibration and coverage checked on held-out time periods, thresholds fitted on past data only, and events scored as events. The part ends with Lab 2. [1 min]

A forecast should state its uncertainty

UNCERTAINTY AND SEQUENCE ANOMALIES

A forecast should state its uncertainty



Synthetic 50 MW wind farm (seed 3). Quantile gradient boosting, 3 hours ahead, on the last six hourly values and hour of day; trained on days 1–90, scored on days 91–120.

Neural Networks / Day 5 · Sequence models and attention 78

Left: five test days with the median forecast and 50% and 80% intervals. The intervals widen in the middle of the power curve, where small changes in wind speed change output a lot, and narrow near zero and near rated power. Red circles mark actuals outside the 80% interval.

Right: coverage on the 717 test hours. The 80% interval contains only 70% of the actuals; the 50% interval 37%; the 90% interval 83%. The intervals are too narrow.

Why? On the training period the same models are close to nominal: 10.7% of actuals fall below the 10% quantile. The test month is calmer, with mean output 11 MW against 21 MW in training, a regime the models saw less often. This is typical: coverage measured in-sample is optimistic. Training targets stop before day 91, so no target crosses the split.

Remedies: check coverage on a recent validation period before use, recalibrate intervals on that period (for example with conformal methods), and report coverage by regime and horizon. [3 min]

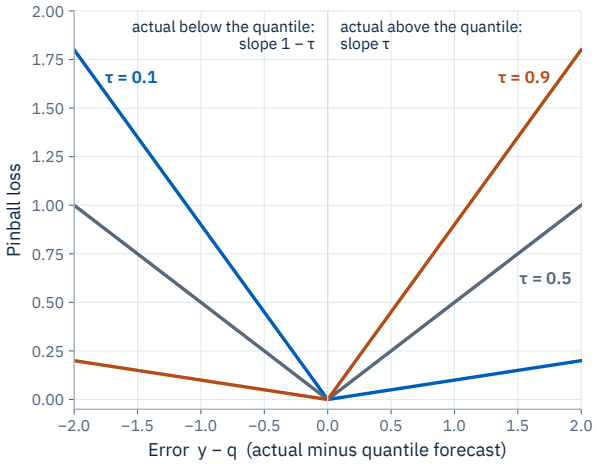
Quantile forecasts minimize the pinball loss

UNCERTAINTY AND SEQUENCE ANOMALIES

Quantile forecasts minimize the pinball loss

$$L_\tau(y, q) = \max(\tau(y - q),\ (\tau - 1)(y - q))$$

- τ = 0.9 penalizes under-forecasts nine times more than over-forecasts.
- The minimizer is the τ-quantile of the outcome.
- One output per quantile, trained with one loss each.



Exact quantile (pinball) loss for τ = 0.1, 0.5 and 0.9.

Neural Networks / Day 5 · Sequence models and attention 79

The pinball loss, from quantile regression (Koenker and Bassett, 1978), is a tilted absolute error. For τ = 0.9, an actual 10 MW above the forecast costs 9 units, while an actual 10 MW below costs only 1. The forecast that minimizes the expected loss is therefore the value exceeded 10% of the time: the 90% quantile.

τ = 0.5 gives half the absolute error, so the median forecast minimizes MAE.

For a neural network, give the final layer one output per quantile and sum the pinball losses: loss = torch.maximum(tau * (y - q), (tau - 1) * (y - q)).mean() for each τ. Separately trained quantiles can cross, for example a 90% quantile below the 80% one; sort them or add a monotonic parameterization.

Averaged over many quantile levels, twice the pinball loss approximates the continuous ranked probability score, a proper scoring rule for probabilistic forecasts (Gneiting and Raftery, 2007). [3 min]

Pinball loss and coverage by hand

PRACTICE

4 min · Individual

Pinball loss and coverage by hand

1. For τ = 0.9 and an actual of 100 MW, compute the loss for q = 90 and for q = 110.
2. A nominal 80% interval contains 150 of 200 test hours. What is its coverage?
3. Is the interval too narrow or too wide, and what would you do?

Answers.

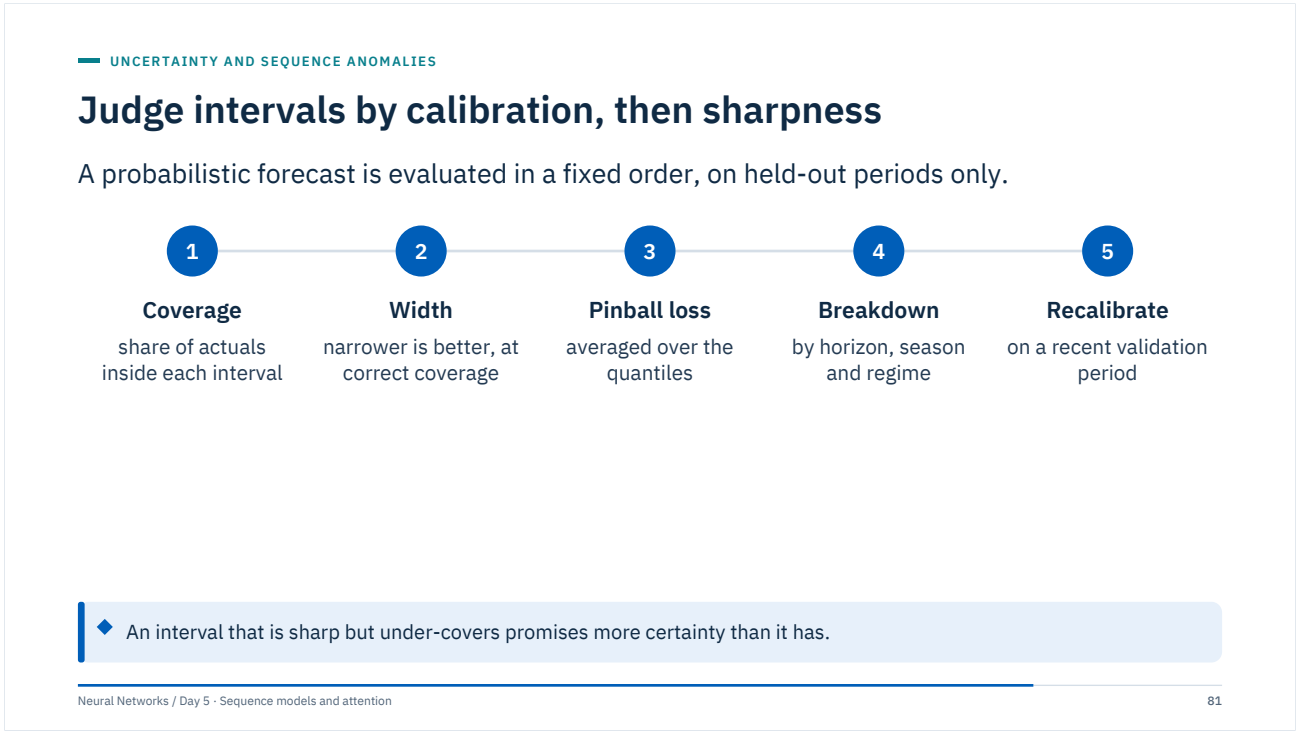
1. q = 90: y − q = 10 ≥ 0, loss = 0.9 × 10 = 9. q = 110: y − q = −10, loss = (0.9 − 1) × (−10) = 1. Under-forecasting the 90% quantile is nine times more expensive.

2. Coverage = 150 / 200 = 75%.

3. Too narrow: 75% is below the nominal 80%. Before acting, consider sampling noise: with 200 independent hours the standard error of a coverage estimate near 80% is √(0.8 × 0.2 / 200) ≈ 2.8 percentage points, so 75% is 1.8 standard errors low. Hours are autocorrelated, so the effective sample is smaller and the evidence weaker. Collect more test hours, check coverage by regime, and if the shortfall persists, widen or recalibrate the interval on a recent validation period.

Common error: judging an interval only by its width. Narrow intervals are good only when coverage is right: calibration first, then sharpness. [4 min]

Judge intervals by calibration, then sharpness



This is the checklist to apply to any interval forecast, including those from pretrained models.

Coverage first: an 80% interval should contain about 80% of actuals on data not used for fitting. Then width, often called sharpness: among calibrated forecasts, narrower is more useful. The pinball loss combines both in one proper score and is the right number for comparing models.

Breakdowns matter as much as for point forecasts: intervals are often well calibrated overall and badly calibrated at long horizons or in rare regimes, such as storms for wind or holidays for load.

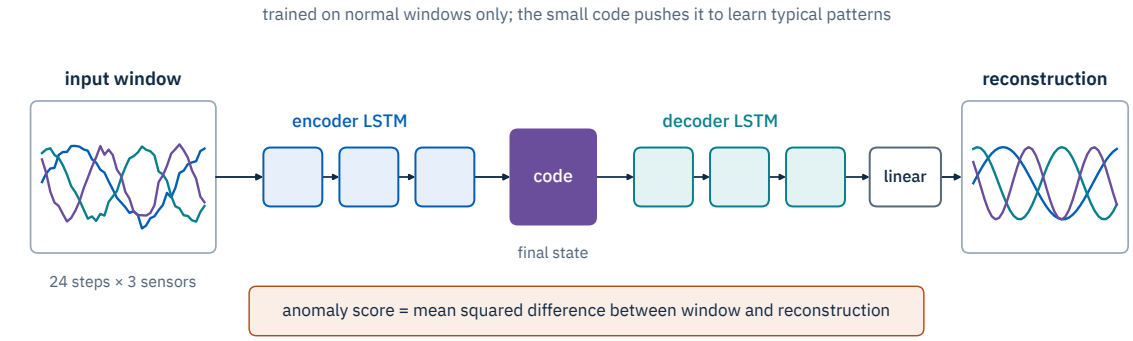
Recalibration adjusts interval widths using recent errors on a validation period; conformal prediction is one family of methods with coverage guarantees under exchangeability, an assumption that time series only approximate. [2 min]

An LSTM autoencoder learns to reconstruct normal windows

UNCERTAINTY AND SEQUENCE ANOMALIES

An LSTM autoencoder learns to reconstruct normal windows

Train on normal operation only; windows that differ from normal reconstruct badly.



Schematic (curves illustrative). Notebook 04 uses 30-step windows of the sensor columns.

Neural Networks / Day 5 · Sequence models and attention 82

An autoencoder, introduced on Day 2, learns to reproduce its input through a smaller internal code. An LSTM autoencoder does this for multivariate windows: the encoder reads the window, its final state is the code, and the decoder unrolls that code back into a window.

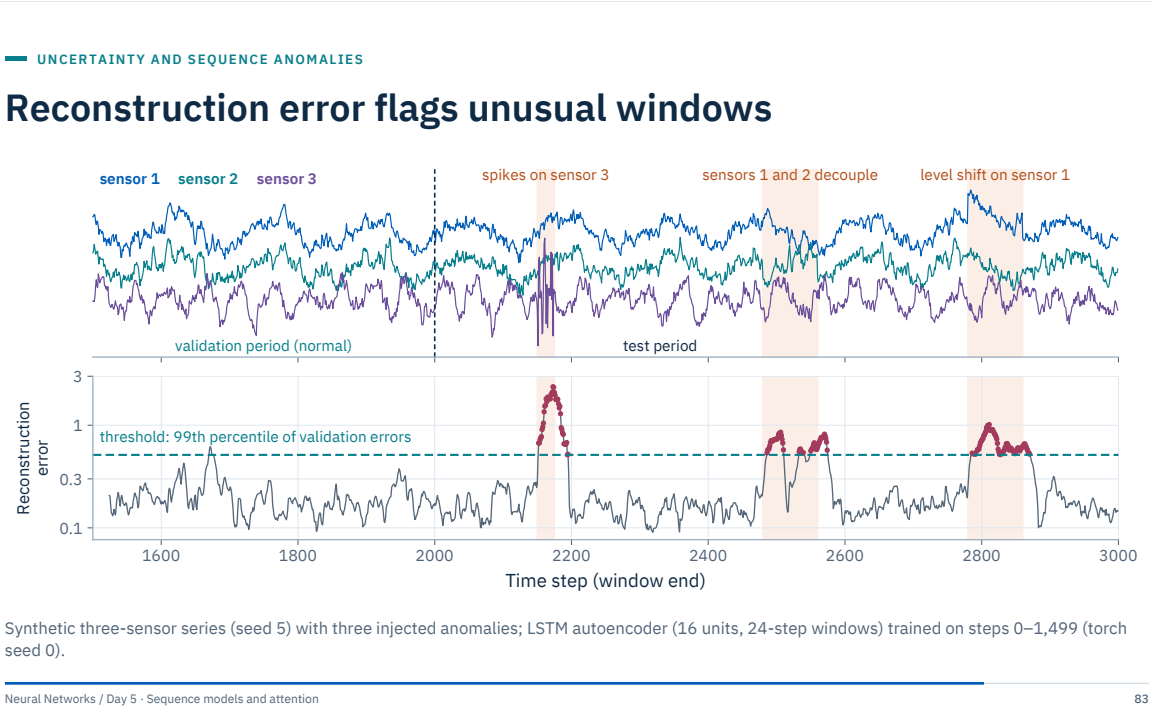
Trained only on normal operation, it learns the typical patterns and correlations between sensors. A window with an unusual pattern, such as a spike, a level shift or two sensors that stop moving together, reconstructs poorly, and its reconstruction error becomes the anomaly score.

Notebook 04 uses a decoder that starts from the last input and feeds back its own output step by step, then reverses the sequence; the figure on the next slide uses a simpler repeat-the-code decoder. Both are common. See Malhotra and colleagues (2016) for an early LSTM encoder–decoder applied to multi-sensor anomaly detection.

Cross-reference: the machine learning course covers reconstruction error alongside Isolation Forest and other detectors. [2 min]

Reference: https://arxiv.org/abs/1607.00148

Reconstruction error flags unusual windows



This is a real, if small, computation. The autoencoder was trained on the first 1,500 steps of normal data. The threshold is the 99th percentile of reconstruction errors on a later normal validation period, steps 1,500 to 1,999, never used for training. Errors on the test period, steps 2,000 to 2,999, are compared with it.

All three injected events exceed the threshold, and in this synthetic test no window outside the events does. Note the middle event: sensors 1 and 2 stay within their usual ranges but stop moving together. A per-sensor limit would miss it; the multivariate reconstruction does not.

Also note that 179 of the 1,000 test windows are flagged, for only three events: overlapping windows repeat one event many times. Count events, their lead time and false alarms per week, not windows.

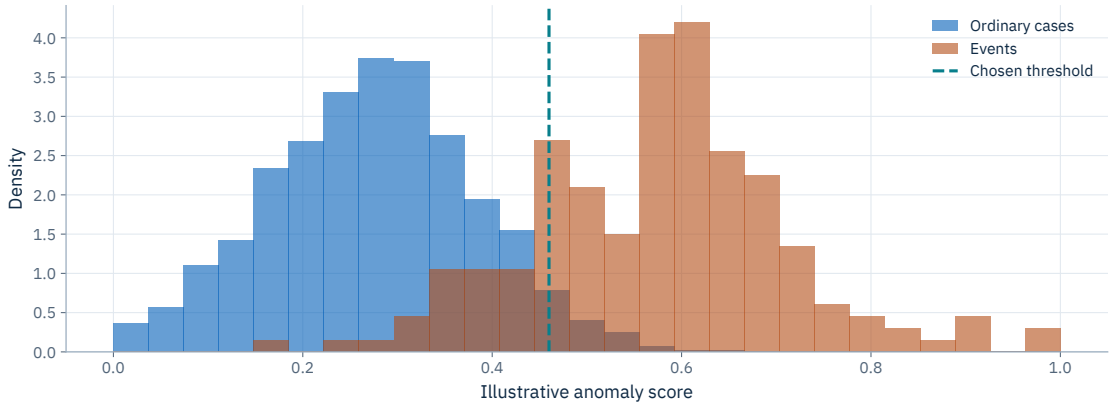
Real anomalies are subtler than injected ones; do not read this as field performance. [3 min]

Any threshold trades missed events for false alarms

UNCERTAINTY AND SEQUENCE ANOMALIES

Any threshold trades missed events for false alarms

Moving the line right removes false alarms and misses more events; no position removes both.



Simulated score distributions (seed 42), not a fitted detector or calibrated probabilities. The machine learning course uses the same example.

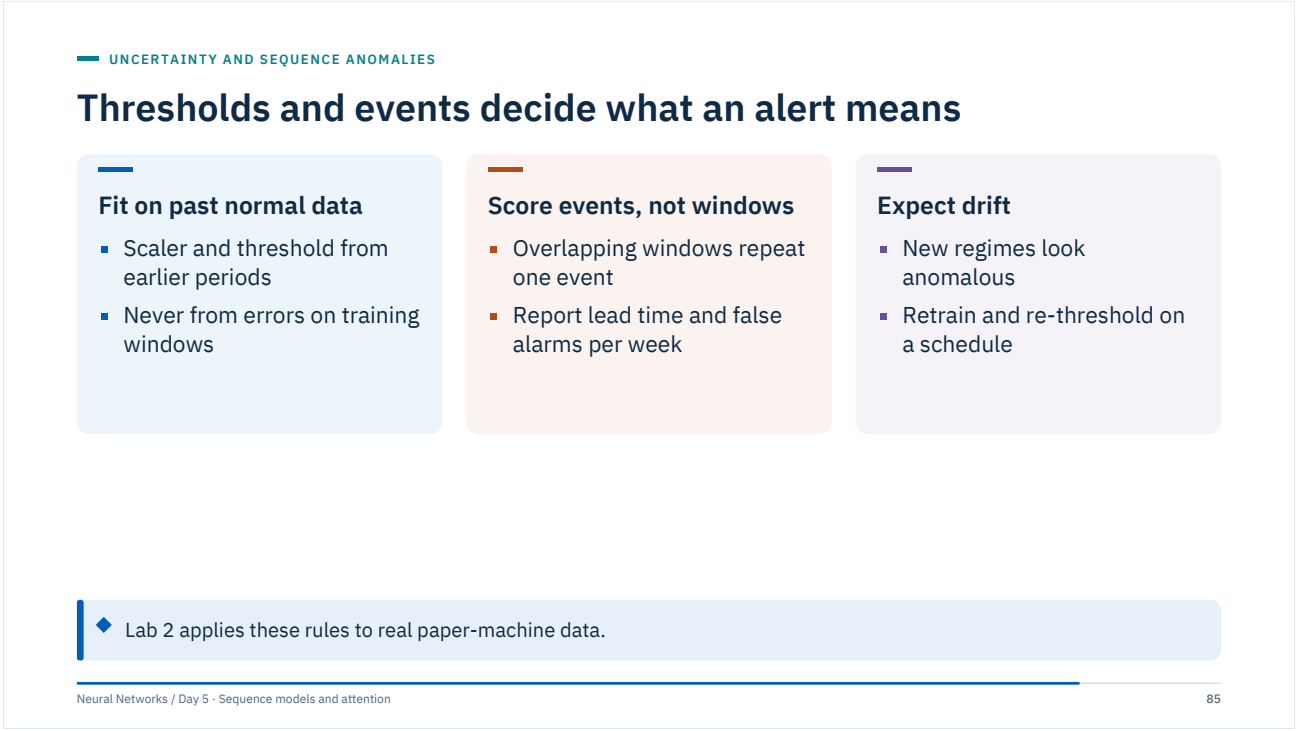
Neural Networks / Day 5 · Sequence models and attention 84

Scores of ordinary periods and of events overlap for any realistic detector, including the reconstruction error of an autoencoder. The threshold decides the trade-off, and choosing it is an operational decision: how many false alarms per week will operators tolerate, and how costly is a missed event?

Notebook 04 sets the threshold at the 99th percentile of reconstruction errors on normal validation windows, which by construction flags about 1% of normal validation windows. With one window every 2 minutes, that is still about 50 flagged windows per week of normal operation; grouped into alerts, the solution's test period gives 6 false alarms per week.

Ask: how would you choose the threshold instead? Answer: fix an alarm budget, for example at most two false alarms per week on a normal validation period, and report the event recall and lead time that this budget buys. The machine learning course's rare-events module treats costs, thresholds and precision–recall curves in depth. [2 min]

Thresholds and events decide what an alert means



These three rules turn a reconstruction score into an alert policy.

Notebook 04 follows the first two, and Lab 2 asks learners to find them in the code. It fits the scaler on normal rows of the training period only. It builds windows inside contiguous stretches, so no window joins data across a downtime gap or a split. It sets the threshold on normal windows of a later validation period, never on training windows, and it scores the test period as events. The third rule is visible in these data: several sensors shift by many training standard deviations between periods, and the notebook prints the largest shifts. Such a level shift would dominate the reconstruction error, so the notebook centres each window on its own mean and the detector looks at how the sensors move within the hour. In operation, retraining and re-thresholding on a schedule would still be needed.

The pattern to remember: an early normal period for training and the scaler, a later normal period for the threshold, and a final period for evaluation, with events scored as events. The machine learning course's module on rare events covers event-level recall, lead time and alert budgets in detail. [2 min]

Each application needs its own target and evaluation

UNCERTAINTY AND SEQUENCE ANOMALIES

Each application needs its own target and evaluation

Application	Possible target	Evaluation priority
ESP monitoring	Future event within a defined horizon	Event recall, lead time, false alerts
Production forecasting	Rate at one or more future horizons	Error by well and horizon
Drilling ROP	Future rate of penetration	Covariates available at the origin
Load and renewables	Hourly values, day ahead	Error by horizon, interval coverage
Sequence autoencoder	Reconstruction or prediction residual	Threshold validation and regime effects

These are task definitions to evaluate, not established capabilities.

These are task definitions to evaluate, not established capabilities.

A model trained on contemporaneous labels may detect the current state rather than predict a future event: an ESP gas-lock label at time t predicted from sensors up to t is a nowcast. To warn in advance, shift the label so it describes the next hour, and score lead time.

Production forecasting needs errors by well and by horizon, and splits that match whether the forecast is for existing wells or new ones. Drilling rate of penetration forecasts must use only covariates known at the origin, such as planned weight on bit, not measured future values.

For anomaly autoencoders, fit the intended normal reference, evaluate on separate events and check how the threshold behaves when the operating regime changes.

Ask: which row matches the Day 3 capstone problems? The ESP gas-lock project, with one difference: its labels describe the current state of unseen pumps, a nowcast, while this row asks for a warning within a defined horizon, which needs shifted labels and lead-time scoring. [2 min]

Lab 2: sequence anomalies or a retail forecast

LABORATORY

35 min

Lab 2: sequence anomalies or a retail forecast

- ☐ Option A: train the LSTM autoencoder on an early normal period only.
- ☐ Set the threshold on a later normal period; score the remaining rows.
- ☐ Report events found, events missed and false alarms, not only F1.
- ☐ Option B: retail forecast for one item; beat the weekly seasonal naive.
- ☐ Either option: one plot, one table and one caveat.

NOTEBOOK

`exercises/04_timeseries_autoencoder/TS_Autoencoder_Student.ipynb`

EVIDENCE TO SUBMIT

- Error-over-time plot with the threshold
- Event or baseline comparison table
- One caveat about the evaluation

Neural Networks / Day 5 · Sequence models and attention 87

Timing: 30 minutes of work, then the five-minute debrief. Option A reads an Excel file with openpyxl (listed in requirements-labs.txt). The notebook fits the scaler on normal rows of 1–14 May, trains on normal windows built inside contiguous stretches, sets the threshold at the 99th percentile of normal validation errors (15–19 May) and scores the test period (20–29 May, 41 breaks) once, as events: a break is found when an alert starts at most 30 minutes before it or at it. Training stops after at most 20 epochs, about a minute on an Apple M3 Max laptop CPU; use 10 on slow laptops. Learners fill in the split, the windows, the threshold and the event counts.

Reference result from the solution: 5 of the 41 test breaks found, only 2 of them before the break itself; 36 missed; 6 false alarms, about 6 per week; a window-level F1 of 0.03. The threshold, 2.29, sits far above the median normal validation error of about 0.07, because windows just after restarts reconstruct badly. That is the natural caveat for this option.

Option B uses exercises/03_walmart_timeseries/Walmart_Forecasting_Student.ipynb. It reads only the first three rows of the 120 MB sales file (nrows=3), which are exactly the three store CA_1 items it selects. Each item has 1,913 daily values with many zero-sales days, so use MAE, not MAPE. Windows are split by target day, 70/10/20; early stopping and all comparisons use validation, and the 377 test days are scored once. Solution test MAE: weekly seasonal naive 0.88, 28-day mean 0.70, zero forecast 0.68, LSTM 0.72 units.

Notebook 05 (stock prediction) is optional homework on methodology: compare with persistence and evaluate daily changes, as on the copy-illusion slide. The solution does both: persistence beats the LSTM on price levels (RMSE $2.78 against $7.16), and on daily changes the LSTM has an R^2 of −5.6 and predicts a rise on every test day. [35 min]

What did the alerts, or the forecasts, really show?

DISCUSSION

What did the alerts, or the forecasts, really show?

- How many distinct events were detected, and how early?
- How many alerts per week would operators receive?
- Did the threshold come from data the model never trained on?
- Retail: did the LSTM beat the weekly seasonal naive on MAE?

◆ Report events and alarms per week; an F1 score on rows hides both.

Neural Networks / Day 5 · Sequence models and attention 88

Take one answer per group; five minutes in total.

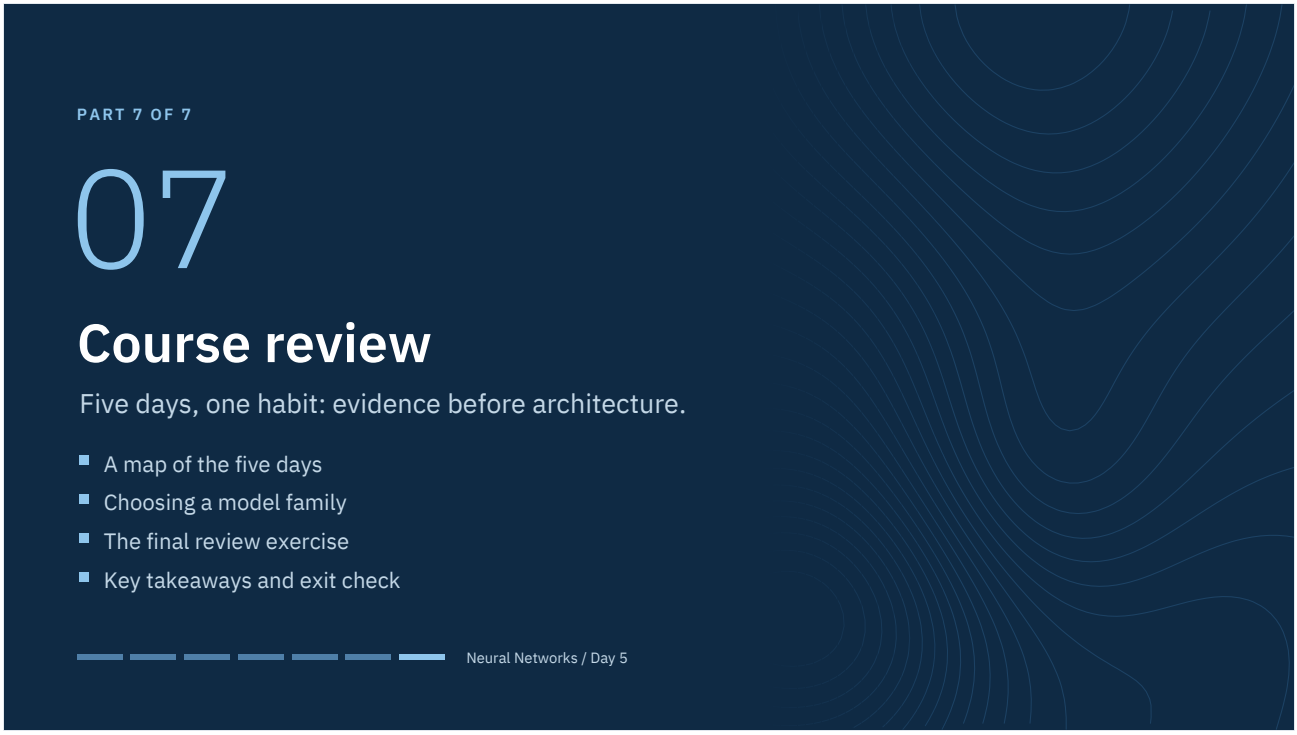
For option A, the useful answers are counts of events, lead time before each event, and false alarms per week, together with how the threshold was chosen. An F1 score on rows mixes all three and depends on how many rows each event contains. In the solution, 5 of 41 breaks are found but only 2 before the break itself, with about 6 false alarms per week: a detector that mostly recognizes a break as it happens gives no warning lead time.

Ask how the result changes if the threshold is set on training-window errors instead of a separate normal period; most groups see more flagged windows and a lower threshold. In the solution the same percentile of training-window errors is 0.88 instead of 2.29.

For option B, expect that a simple mean or even a zero forecast is hard to beat on intermittent single-item sales; say so plainly when it happens. In the solution the LSTM beats the weekly seasonal naive but loses to both on MAE: MAE rewards the median, which is zero for this item, while a network trained with MSE aims at the mean. The notebook's own summary suggests adding calendar and price features and ensembling with statistical methods; those are reasonable next steps.

Collect the caveats: they are the most valuable output of the lab. [5 min]

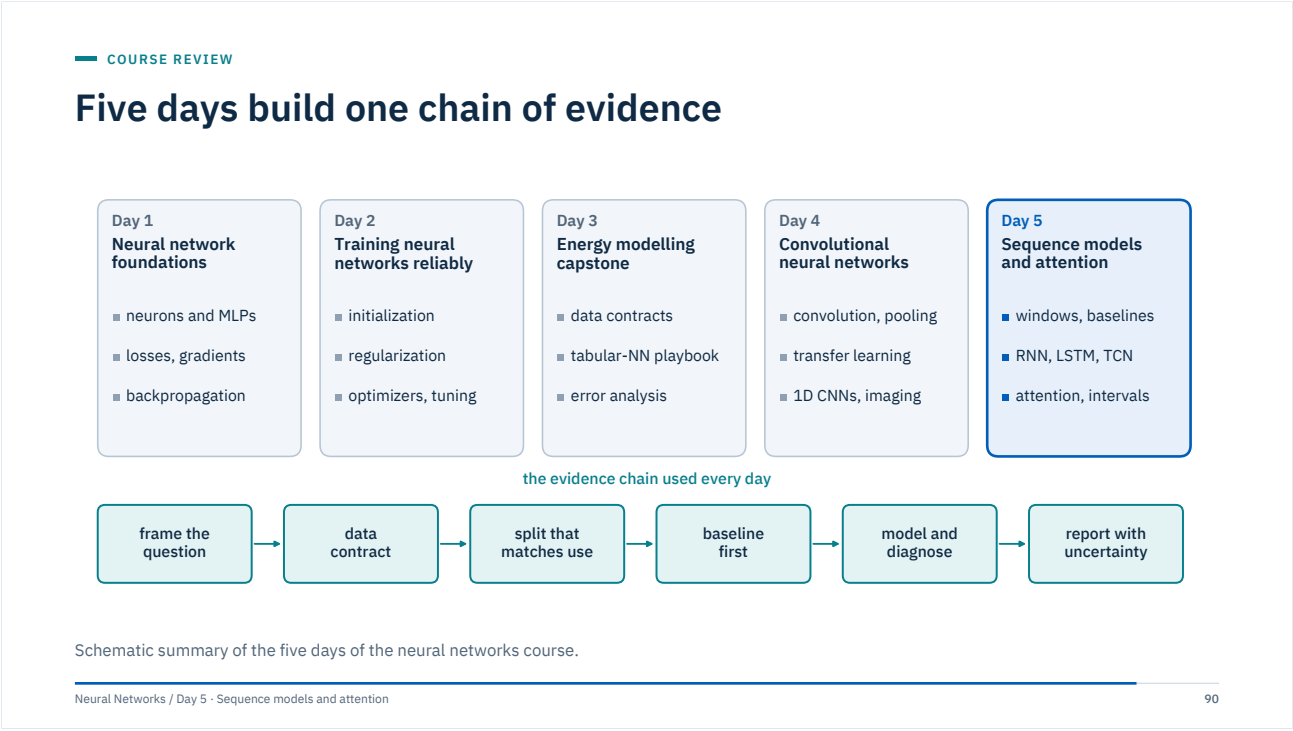
Course review



The last half hour steps back from sequences to the whole neural networks course. The goal is not to memorize architectures but to leave with a decision process that works for the next problem, which will not look like any of our labs.

Keep this part interactive: the final review exercise is the centrepiece, and the discussion matters more than the slides. [1 min]

Five days build one chain of evidence

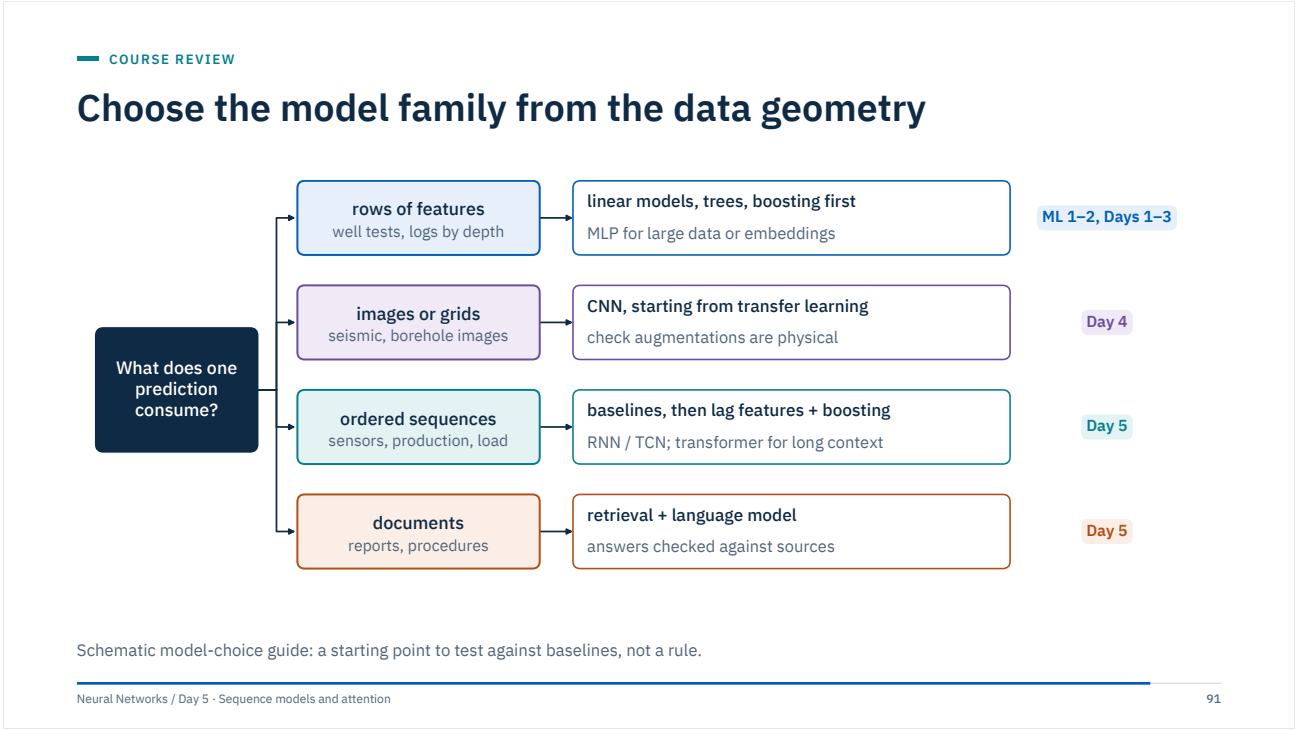


Walk across the days in one sentence each. Day 1, Neural network foundations, built the neuron, the multilayer perceptron, losses and backpropagation. Day 2, Training neural networks reliably, covered initialization, normalization, regularization, optimizers and tuning, and introduced autoencoders. Day 3, the Energy modelling capstone, applied everything to supplied project data with data contracts and error analysis. Day 4, Convolutional neural networks, added convolutions for images and grids. Today, Sequence models and attention, added sequences, recurrence, attention, uncertainty and anomalies.

The bottom row is the part that does not change between architectures: frame the question, write the data contract, choose a split that matches use, beat a baseline, model and diagnose, report with uncertainty.

Ask the room: which step of the chain has failed most often in your own projects? The usual answers are the data contract and the split, not the model. [2 min]

Choose the model family from the data geometry



Start from what one prediction consumes.

Rows of features, such as well tests or log values per depth: start with linear models, trees and gradient boosting from the machine learning course. Use an MLP when data are large, when you need embeddings for many categories, or when the model must be part of a larger network.

Images and grids, such as seismic sections or borehole images: convolutional networks, usually starting from transfer learning, with augmentations that respect the physics.

Ordered sequences, such as sensors, production or load: baselines first, then lag features with boosting, then recurrent networks or TCNs for moderate context and transformers or pretrained forecasters for long context or many related series.

Documents: retrieval with a language model, answers checked against sources.

Every branch shares the same evaluation habits; the next slide makes them explicit. [2 min]

Every model family shares the same evaluation habits

COURSE REVIEW

Every model family shares the same evaluation habits

Data	Split that matches use	First baseline	Report
Rows per well or asset	Grouped by well, often also by time	Mean or linear model	Error by well
Images or grids	By survey, well or field	Pretrained features + linear	Recall per class
Sequences	Rolling origins in time	Persistence, seasonal naive	Error by horizon
Documents	By document set	Keyword search	Faithfulness to sources

Uncertainty and monitoring apply to all four.

Neural Networks / Day 5 · Sequence models and attention 92

Read the table as a checklist for the final exercise. Each data type has a natural unit of independence that the split must respect: the well, the survey, the time period, the document collection. Each has a simplest defensible baseline. Each has a breakdown that hides failures when left out.

Point out that the "first baseline" column is deliberately modest. A baseline is not a straw man; it is the level that a more complex model must beat on identical data to justify its cost.

The last line matters: every deployed model needs a statement of uncertainty and a plan for monitoring drift, whatever its architecture. [1 min]

Defend a model choice for one energy task

PRACTICE

10 min Pairs

Defend a model choice for one energy task

Pick one: ESP gas-lock warning 1 hour ahead; facies from well-log curves; day-ahead feeder load.

1. Choose a dense model, CNN, RNN or attention model and defend the choice.
2. Define the tensor shapes, target timing, baseline and evaluation split.
3. Explain one plausible failure and the evidence that would reveal it.

Neural Networks / Day 5 · Sequence models and attention 93

Use a ten-minute pair exercise, then two or three pairs present in one minute each. Full credit requires reasoning about data geometry and evaluation, not choosing the most complex architecture. Relate answers back to the Day 3 capstone and propose an extension only if the data support one.

Worked example for day-ahead feeder load. Origin 10:00 daily; target 24 hourly values, horizons 14 to 37 hours. Inputs: the history (batch, 168, F) up to the origin, with F features such as load, observed temperature and calendar; plus the known future covariates for the 24 target hours, (batch, 24, F′): the temperature forecast issued before 10:00 and the calendar. Output (batch, 24) from a multi-output LSTM or TCN head. Baseline: weekly seasonal naive, plus a boosted lag model. Split: rolling origins over the last year, final three months as test, windows purged at boundaries, scalers fitted per fold. Failure: public holidays, which are rare in training; evidence: error by day type, with holidays reported separately.

For facies from logs, the natural answer is a dense or boosted model on depth rows, or a one-dimensional CNN along depth, with wells grouped in the split.

For the ESP gas-lock warning, a small causal model is enough: lag features with boosting, or a unidirectional GRU or TCN over the last few hours of sensor readings, (batch, time, sensors) in and one probability out. The label must be shifted to "gas lock within the next hour"; the Day 3 labels describe the current state, so as supplied they support only a nowcast. Baseline: the Day 3 classifier or a single-sensor threshold. Split: by pump and by time. Failure: alerts that fire only once the gas lock has started; evidence: lead time for each event and false alarms per week. [10 min]

A strong answer names the timing before the network

DISCUSSION

A strong answer names the timing before the network

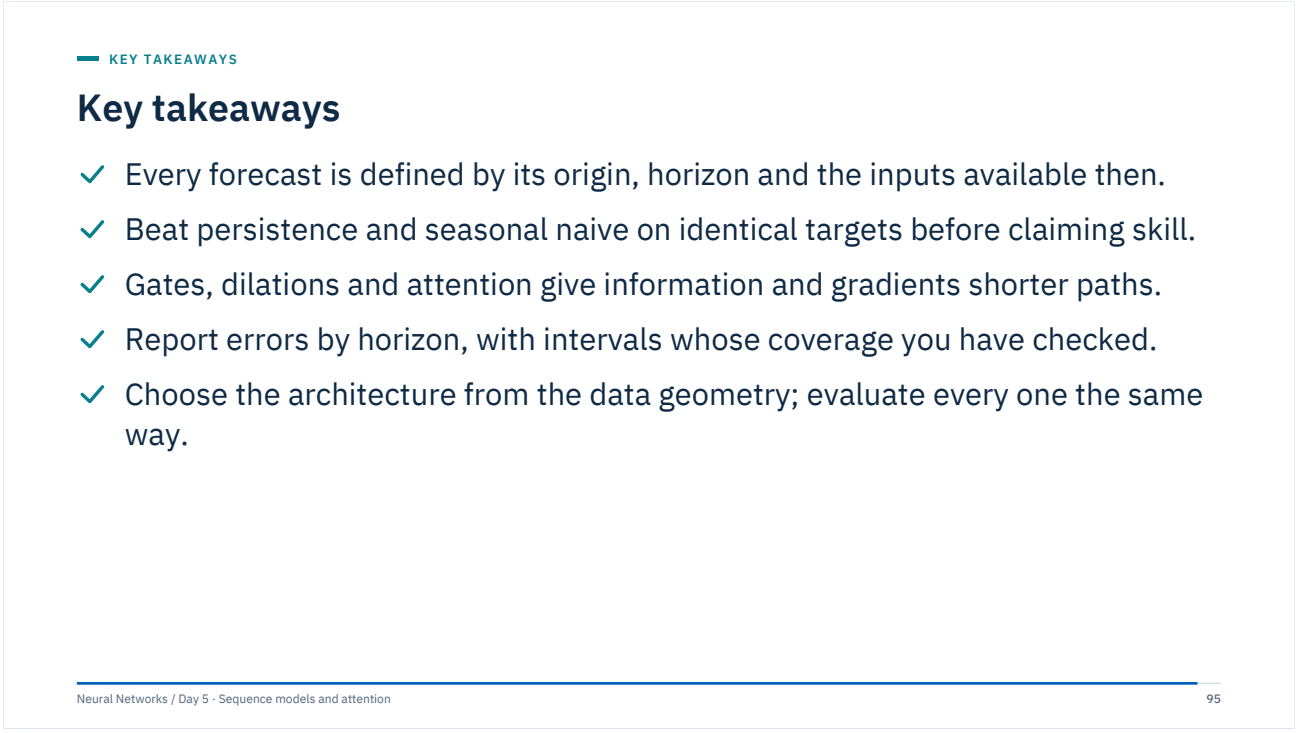
1. **Unit and timing**: what one prediction consumes, and when it is made
2. **Shapes**: (batch, time, features) in; the target shape out
3. **Baseline**: the simplest method with exactly the same timing
4. **Split**: grouped or rolling origin, matching deployment
5. **Failure evidence**: the breakdown that would expose the weakness

Use this list to give feedback on the presentations. The order is deliberate: an answer that starts with "we use a transformer" and never states the origin, the horizon or the split has skipped the steps that decide whether the result can be trusted.

Point to strong answers heard in the room. Typical gaps: no statement of when the prediction is made, a baseline that uses different timing from the model, a random split for data ordered in time, and a failure analysis that restates the aggregate metric instead of naming a breakdown.

Encourage learners to reuse this five-line template at the start of their next project at work. [2 min]

Key takeaways



Read each takeaway and connect it to a moment of the day. The origin: the 10:00 day-ahead case and the lag-24 trap. Baselines: the weekly seasonal naive at 19.7 MW and the Jena seasonal naive at 2.38 °C. Shorter paths: the LSTM cell-state lane, the TCN's 16-step receptive field in four layers, and attention's single step between any two positions. Intervals: the 80% interval that covered only 70% of a calmer month. Architecture choice: the model-choice guide.

Ask each learner to write down one habit they will apply in their next project, then take two or three examples. [2 min]

Exit check: five quick questions

PRACTICE

4 min · Individual

Exit check: five quick questions

1. A forecast issued at 10:00 targets 18:00 tomorrow. What is the horizon?
2. Which baseline would you compute first for hourly load, and why?
3. Which quantity does the LSTM forget gate multiply?
4. Which positions does a causal mask hide from a query?
5. An 80% interval covers 64% of test hours. What do you conclude?

Answers.

1. 32 hours: from 10:00 today to 10:00 tomorrow is 24 hours, plus 8 more to 18:00.

2. The weekly seasonal naive, the same hour one week earlier, because hourly load has strong daily and weekly cycles; also persistence for the shortest horizons. Both must be scored on the same origins as the model.

3. The previous cell state, c(t−1), elementwise.

4. All key positions after the query's own position: the upper triangle of the attention matrix.

5. The interval is too narrow and miscalibrated: it promises more certainty than it delivers. Check coverage by regime and horizon, and widen or recalibrate on a recent validation period before anyone uses it.

Collect answers anonymously if possible; the most frequently missed question tells you what to revisit in the course follow-up. [4 min]

References and further study

FURTHER STUDY

References and further study

- Hyndman and Athanasopoulos, *Forecasting: Principles and Practice*, 3rd edition (online); Hyndman and Koehler (2006) on MASE.
- Hochreiter and Schmidhuber (1997), long short-term memory; Cho and colleagues (2014), GRU; Bai, Kolter and Koltun (2018), TCN.
- Bahdanau, Cho and Bengio (2014), attention; Vaswani and colleagues (2017), *Attention Is All You Need*; Nie and colleagues (2023), PatchTST.
- Koenker and Bassett (1978), quantile regression; Gneiting and Raftery (2007), proper scoring rules.
- PyTorch documentation (LSTM, GRU, MultiheadAttention) and the Keras weather-forecasting example.

Neural Networks / Day 5 · Sequence models and attention 97

Forecasting: Principles and Practice is the best free introduction to baselines, decomposition, evaluation and prediction intervals; its examples use R, but the ideas transfer directly. The original LSTM and GRU papers are readable; the PyTorch documentation gives the exact gate conventions used in code, which differ in small ways from some papers.

For attention and transformers, read Bahdanau and colleagues for the idea and Vaswani and colleagues for the architecture. PatchTST is a readable entry point to transformer forecasters.

Check the exact framework's mask and gate conventions before translating equations into code.

End by asking learners which evaluation habit they will use on their next project, and thank them for the five days. [1 min]

Reference: https://otexts.com/fpp3/ ; https://doi.org/10.1016/j.ijforecast.2006.03.001 ; https://doi.org/10.1162/neco.1997.9.8.1735 ; https://arxiv.org/abs/1406.1078 ; https://arxiv.org/abs/1803.01271 ; https://arxiv.org/abs/1409.0473 ; https://arxiv.org/abs/1706.03762 ; https://arxiv.org/abs/2211.14730 ; https://docs.pytorch.org/docs/stable/generated/torch.nn.LSTM.html ; https://keras.io/examples/timeseries/timeseries_weather_forecasting/